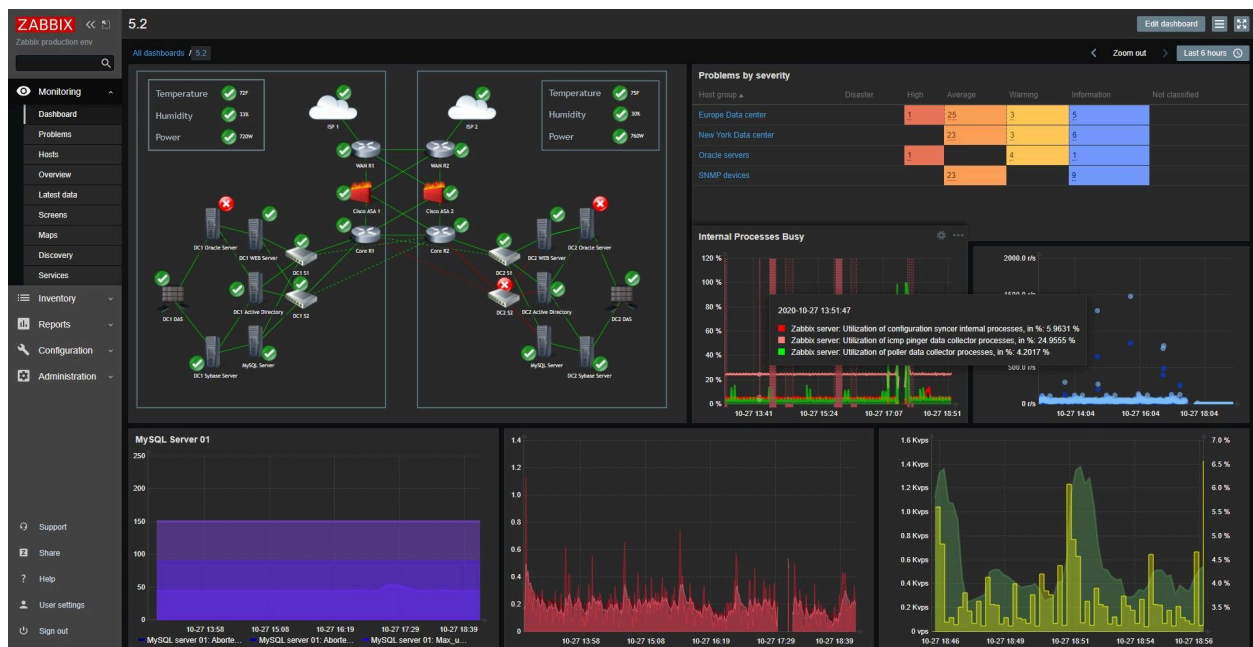


PROYECTO MONITORIZACIÓN CON ZABBIX

Instalación, Configuración y Administración de un Sistema de Monitorización para el Departamento de Informática del IES Guadalpeña



Eugenio Rondán Bermúdez

26/03/2026

2º C.F. G.M. Sistemas Microinformáticos y Redes | Proyecto Integrado

Tutor: Cristóbal Ángel Fernandez Ramírez

RESUMEN

Zabbix es una plataforma de monitorización de código abierto diseñada para supervisar en tiempo real el estado, rendimiento y disponibilidad de infraestructuras informáticas.

Este proyecto desarrolla la instalación, configuración y administración completa de un sistema de monitorización basado en Zabbix 7.0 LTS sobre Debian 12, orientado al Departamento de Informática del IES Guadalpeña.

El proyecto incluye la preparación de máquinas virtuales, la configuración del servidor Zabbix, la integración de agentes, la creación de alertas, la automatización de acciones y la monitorización de servidores Proxmox, máquinas Debian 12 y dispositivos de red. Además, se realiza un estudio de viabilidad técnica y económica, un análisis de requisitos, un diseño detallado de la arquitectura del sistema y una evaluación de riesgos.

El proyecto se gestiona mediante ClickUp y GitHub, garantizando trazabilidad, control de versiones y una metodología profesional. El resultado final es un sistema funcional, escalable y seguro que permite detectar incidencias, generar alertas y visualizar métricas mediante paneles avanzados, demostrando la aplicabilidad real de Zabbix en entornos educativos y empresariales.

¡ALERTA!

Algunos de los detalles dados en el informe no han sido añadidos aún, ya que la fase correspondiente no se ha publicado.

ÍNDICE

RESUMEN	1
ÍNDICE	2
1. INTRODUCCIÓN	7
2. ¿QUÉ ES ZABBIX?	8
2.1. CARACTERÍSTICAS PRINCIPALES DE ZABBIX	8
2.2. COMPONENTES PRINCIPALES DE ZABBIX	9
2.3. ARQUITECTURA GENERAL DE ZABBIX	11
3. ARQUITECTURA DE ZABBIX	12
3.1. COMPONENTES PRINCIPALES DE LA ARQUITECTURA DE ZABBIX	12
3.1.1. ZABBIX SERVER	12
3.1.2. BASE DE DATOS	13
3.1.3. FRONTEND WEB	13
3.1.4. AGENTES ZABBIX	13
3.1.5. PROXIES (OPCIONALES)	14
3.1.6. INTEGRACIONES Y APIS	14
3.2. FLUJO DE FUNCIONAMIENTO	14
3.3. VENTAJAS DE LA ARQUITECTURA	15
4. REQUISITOS DEL SISTEMA	16
4.1. REQUISITOS HARDWARE	16
4.1.1. REQUISITOS HARDWARE DE NUESTRO PROYECTO	16
4.2. REQUISITOS SOFTWARE	17
4.2.1. SISTEMA OPERATIVO	17
4.2.2. SERVIDOR WEB	17
4.2.3. PHP	17
4.2.4. BASE DE DATOS	17
4.2.5. AGENTES	17
4.3. REQUISITOS DEL FRONTEND	18
4.4. REQUISITOS ADICIONALES	18
4.4.1. SINCRONIZACIÓN HORARIA	18
4.4.2. RED ESTABLE	18
4.4.3. ACCESO SSH	18
4.4.4. VIRTUALBOX GUEST ADDITIONS	18
5. VENTAJAS E INCONVENIENTES DE ZABBIX	19
5.1. VENTAJAS	19
5.1.1. ESCALABILIDAD	19
5.1.2. FIABILIDAD Y ESTABILIDAD	19

5.1.3. RENTABILIDAD	19
5.1.4. VISIBILIDAD TOTAL	20
5.1.5. FLEXIBILIDAD Y PERSONALIZACIÓN	20
5.1.6. INTEGRACIÓN CON HERRAMIENTAS EXTERNAS	20
5.1.7. COMUNIDAD ACTIVA Y SOPORTE	21
5.2. INCONVENIENTES	21
5.2.1. CURVA DE APRENDIZAJE ELEVADA	21
5.2.2. INTERFAZ TRADICIONAL	21
5.2.3. ESCALABILIDAD MANUAL	21
5.2.4. DEPENDENCIA DE LA BASE DE DATOS	22
5.2.5. ALERTAS AVANZADAS REQUIEREN SCRIPTING	22
5.2.6. ACTUALIZACIONES DELICADAS	22
6. SEGURIDAD EN ZABBIX	23
6.1. CIFRADO Y COMUNICACIONES SEGURAS	23
6.1.1. TIPOS DE CIFRADO SOPORTADO	23
6.1.2. BENEFICIOS DEL CIFRADO	23
6.2. AUTENTICACIÓN Y CONTROL DE ACCESO	23
6.2.1. MÉTODOS DE AUTENTICACIÓN DISPONIBLES	24
6.2.2. CONTROL DE ACCESO BASADO EN ROLES (RBAC)	24
6.3. GESTIÓN DE CONTRASEÑAS	24
6.4. SEGURIDAD EN LA BASE DE DATOS	25
6.5. SEGURIDAD EN EL FRONTEND	25
6.6. SEGURIDAD EN AGENTES Y PROXIES	26
6.7. AUDITORÍA Y REGISTRO DE EVENTOS	26
7. INTEGRACIONES DE ZABBIX	27
7.1. INTEGRACIÓN MEDIANTE API REST	27
7.2. INTEGRACIÓN CON GRAFANA	28
7.3. INTEGRACIÓN CON SISTEMAS DE TICKETS	28
7.4. INTEGRACIÓN CON SISTEMAS CLOUD	29
7.5. INTEGRACIÓN CON CONTENEDORES Y KUBERNETES	29
7.6. INTEGRACIÓN CON PROXMOX	30
7.7. INTEGRACIÓN CON DISPOSITIVOS DE RED	30
7.8. INTEGRACIÓN CON HERRAMIENTAS DE AUTOMATIZACIÓN	31
7.9. INTEGRACIÓN CON SISTEMAS DE NOTIFICACIONES	31
8. AUTOMATIZACIÓN EN ZABBIX	32
8.1. ACCIONES AUTOMÁTICAS	32
8.2. SCRIPTS REMOTOS	33
8.3. MACROS	34

8.4. PLANTILLAS	34
8.5. DESCUBRIMIENTO AUTOMÁTICO (LLD)	35
8.6. DESCUBRIMIENTO DE RED	35
8.7. AUTOMATIZACIÓN MEDIANTE API	36
9. MÉTRICAS EN ZABBIX	37
9.1. TIPOS DE MÉTRICAS EN ZABBIX	37
9.2. FRECUENCIA DE ACTUALIZACIÓN (INTERVALOS)	39
9.3. TRIGGERS BASADOS EN MÉTRICAS	40
9.4. VISUALIZACIÓN DE MÉTRICAS	40
9.5. IMPORTANCIA DE LAS MÉTRICAS EN LA MONITORIZACIÓN	41
10. ESTADO DEL ARTE	42
10.1. ZABBIX	42
10.2. PROMETHEUS	43
10.3. NAGIOS	44
10.4. PRTG NETWORK MONITOR	45
11. ESTUDIO DE VIABILIDAD	46
11.1. VIABILIDAD TÉCNICA	46
11.2. VIABILIDAD ECONÓMICA	47
11.2.1. COSTE DEL HARDWARE	48
11.2.2. COSTE DEL SOFTWARE	50
11.2.3. COSTE DE MANO DE OBRA	51
11.3. VIABILIDAD OPERATIVA	54
11.3.1. CAPACIDAD DEL PERSONAL TÉCNICO	55
11.3.2. FACILIDAD DE USO DEL SISTEMA	55
11.3.3. MANTENIMIENTO DEL SISTEMA	55
11.3.4. INTEGRACIÓN CON EL ENTORNO EDUCATIVO	55
11.4. VIABILIDAD TEMPORAL	56
12. ANÁLISIS DE REQUISITOS	59
12.1. REQUISITOS FUNCIONALES	59
12.2. REQUISITOS NO FUNCIONALES	60
12.3. REQUISITOS HARDWARE	61
12.4. REQUISITOS SOFTWARE	62
12.5. REQUISITOS DE RED	62
12.7. ANÁLISIS DE RIESGOS Y VULNERABILIDADES	65
13. DISEÑO DEL PROTOTIPO	70
13.1. VISIÓN GENERAL DEL PROTOTIPO	70
13.2. ROL DE CADA COMPONENTE DEL PROTOTIPO	70
13.2.1. SERVIDOR ZABBIX (DEBIAN 12)	70

13.2.2. SERVIDOR PROXMOX	71
13.2.3. SWITCH GESTIONABLE	71
13.2.4. CLIENTE DEIBAN 13	71
13.2.5. CLIENTE MX LINUX	71
13.2.6. CLIENTE WINDOWS SERVER 2022 (DHCP)	71
13.2.7. CLIENTE WINDOWS 10	71
13.3. TOPOLOGÍA DEL PROTOTIPO	72
13.4. FLUJO DE FUNCIONAMIENTO	72
13.5. DISEÑO LÓGICO	72
13.6. JUSTIFICACIÓN DEL PROTOTIPO	72
14. DISEÑO DEL PROTOTIPO	74
14.1. DISEÑO DE LA ARQUITECTURA DE RED	74
14.2. DISEÑO DE LA TOPOLOGÍA DE RED	75
14.3. DISEÑO DE LAS SOLUCIONES DE SEGURIDAD	75
14.4. DIAGRAMA DE DESPLIEGUE	76
14.5. DIAGRAMA DE INFRAESTRUCTURA	76
14.6. DIAGRAMA DE INTEGRACIÓN	77
15. IMPLEMENTACIÓN	78
15.1. INSTALACIÓN Y CONFIGURACIÓN DE LOS DISPOSITIVOS DE RED	78
15.1.1. SERVIDOR ZABBIX	78
15.1.2. AGENTE ZABBIX	79
15.1.3. PROTOCOLO SNMP	79
15.2. CONFIGURACIÓN DE LAS SOLUCIONES DE SEGURIDAD	80
15.2.1. CONTROL DE ACCESO	80
15.2.2. COMUNICACIÓN CIFRADA	81
15.2.3. PROTECCIÓN DE LA BASE DE DATOS	81
15.3. CONFIGURACIÓN DE LAS SOLUCIONES DE VIRTUALIZACIÓN	81
15.4. CONFIGURACIÓN DE LAS SOLUCIONES DE ALMACENAMIENTO EN RED	83
15.5. CONFIGURACIÓN DE LAS COPIAS DE SEGURIDAD	86
15.6. CONFIGURACIÓN DE LOS UMBRALES DE AVISO	89
15.6.1. GRUPO DE TELEGRAM CON BOTFATHER	89
16. ADMINISTRACIÓN	97
16.1. GESTIÓN DE USUARIOS Y PERMISOS	97
16.2. MONITOREO Y MANTENIMIENTO EN RED	97
16.3. MONITOREO DE DISPONIBILIDAD	97
16.4. POLÍTICAS DE SEGURIDAD	100
16.5. PLAN DE MANTENIMIENTO PREVENTIVO Y CORRECTIVO	104
16.5.1. MANTENIMIENTO PREVENTIVO	104

16.5.2. MANTENIMIENTO CORRECTIVO	108
16.6. IMPLEMENTACIÓN DE BACKUPS Y RECUPERACIÓN ANTE DESASTRES	112
16.7. PLAN DE CONTINGENCIA	115
16.7.1. IDENTIFICACIÓN Y CLASIFICACIÓN DEL PROBLEMA	115
16.7.2. ACTUACIÓN ANTE ERROR	115
16.7.3. TIEMPOS DE RECUPERACIÓN	116
16.8. SOPORTE A LA APLICACIÓN	116
16.8.1. HELPDESK	116
17. MONITORIZAR UN SERVIDOR PROXMOX EN ZABBIX	117
17.1. Configuración de los umbrales de aviso (Proxmox)	127
17.1.1. Activación de triggers de aviso	129
17.2. Uso de mapas para aumentar la facilidad de gestión de los equipos	135
18. Herramientas de apoyo	140
18.1. Control de versiones (GIT)	140
18.2. Gestión de pruebas	140
18.2.1. Pruebas de funcionamiento (Alertas)	140
18.2.2. Pruebas de rendimiento (Umbrales de aviso)	140
19. Conclusiones	141
19.1. Conclusiones sobre el trabajo realizado	141
19.1.1. Monitorización de virtualización	141
19.1.2. Control de la red con el protocolo SNMP	141
19.1.3. Aplicación de políticas de seguridad	141
19.1.4. Sistema de alertas redundante	141
19.2. Conclusiones personales	142
19.3. Posibles ampliaciones y mejoras	142
19.3.1. Configuración de Zabbix en un Clúster de alta disponibilidad	142
19.3.2. Integración con un sistema de Tickets HelpDesk	143
BIBLIOGRAFÍA	144
Eugenio Rondán Bermúdez	2

1. INTRODUCCIÓN

La monitorización de sistemas es un componente esencial en la administración de infraestructuras informáticas modernas. Permite supervisar en tiempo real el estado de servidores, redes, aplicaciones y servicios, anticiparse a fallos y optimizar el rendimiento global de la organización.

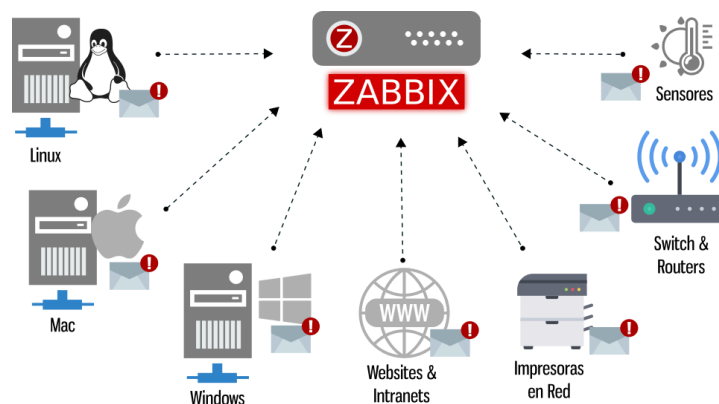
En este proyecto se desarrolla un sistema completo de monitorización basado en Zabbix 7.0 LTS, una de las plataformas más robustas y extendidas en entornos profesionales. El objetivo principal es implementar, configurar y administrar un entorno de monitorización funcional para el Departamento de Informática del IES Guadalpeña, utilizando máquinas virtuales con Debian 12 y agentes distribuidos.

El proyecto abarca desde la instalación inicial del servidor Zabbix hasta la creación de dashboards, alertas, automatizaciones y monitorización de infraestructura real como servidores Proxmox, máquinas Linux y dispositivos de red.

Además, se incluye un estudio de viabilidad, un análisis de requisitos, un diseño detallado de la arquitectura, un análisis de riesgos y una documentación completa del proceso.

El trabajo se gestiona mediante ClickUp y GitHub, garantizando una metodología profesional basada en control de versiones, planificación y trazabilidad.

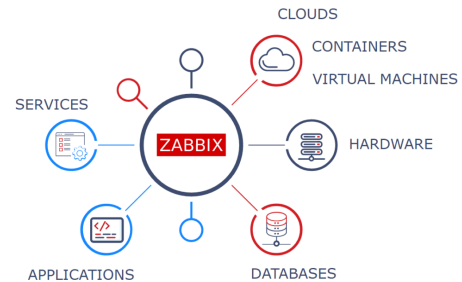
El resultado final es un sistema estable, escalable y seguro, capaz de ofrecer una visión completa del estado de la infraestructura y de reaccionar automáticamente ante incidencias.



2. ¿QUÉ ES ZABBIX?

Zabbix es una plataforma de monitorización de código abierto diseñada para supervisar en tiempo real el estado, rendimiento y disponibilidad de infraestructuras informáticas. Permite monitorizar servidores físicos y virtuales, dispositivos de red, aplicaciones, servicios, contenedores, entornos cloud y sistemas distribuidos.

Su enfoque integral combina la recopilación de métricas, la generación de alertas, la visualización avanzada y la automatización de acciones, convirtiéndolo en una herramienta esencial para administradores de sistemas y equipos de operaciones.



Zabbix destaca por su estabilidad, escalabilidad y flexibilidad, siendo ampliamente utilizado tanto en entornos educativos como empresariales. La versión utilizada en este proyecto, **Zabbix 7.0 LTS**, incorpora mejoras en rendimiento, seguridad, visualización y soporte para nuevas tecnologías.

2.1. CARACTERÍSTICAS PRINCIPALES DE ZABBIX

Zabbix ofrece un conjunto de funcionalidades avanzadas que permiten monitorizar infraestructuras de cualquier tamaño:

- Monitorización en tiempo real:

Recopila métricas de CPU, RAM, disco, red, procesos, servicios, bases de datos, contenedores, etc.

- Alertas inteligentes:

Permite configurar triggers complejos, umbrales dinámicos, escalado de alertas y notificaciones por correo, SMS, webhooks o integraciones externas.

- Visualización avanzada:

Incluye dashboards, gráficos, mapas de red, informes, pantallas personalizadas y vistas de problemas.

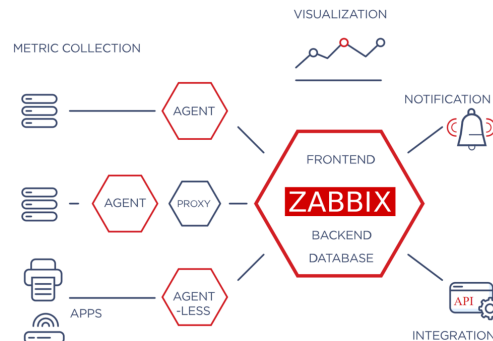
- Automatización:

Puede ejecutar scripts remotos, reiniciar servicios, generar tickets o realizar acciones correctivas sin intervención humana.

- Integración con múltiples tecnologías:

Compatible con:

- SNMP
- IPMI
- JMX
- HTTP/HTTPS
- SSH/Telnet
- VMware
- Docker/Kubernetes
- APIs REST
- Sistemas cloud (AWS, Azure, GCP)



- Escalabilidad:

Permite monitorizar desde pequeñas redes hasta infraestructuras con millones de métricas mediante proxies distribuidos.

- Código abierto:

Totalmente gratuito, con una comunidad activa y soporte profesional opcional.

2.2. COMPONENTES PRINCIPALES DE ZABBIX

Zabbix se compone de varios elementos que trabajan de forma conjunta:

- Zabbix Server:

Es el núcleo del sistema. Recibe, procesa y almacena las métricas. Gestiona alertas, acciones y comunicación con agentes y proxies.

- Base de datos:

Almacena:

- Configuración
- Históricos

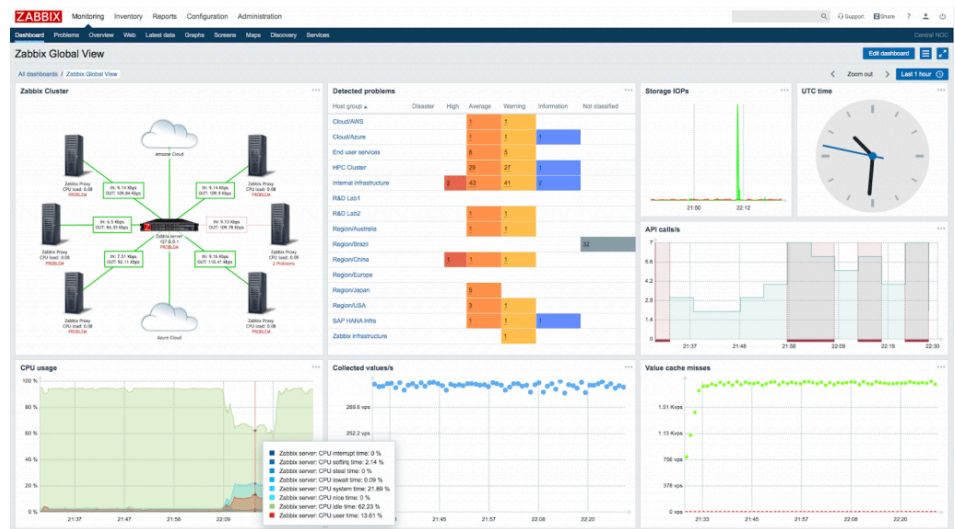
- Eventos
- Métricas
- Usuarios
- Dashboards

Compatible con MySQL, MariaDB, PostgreSQL y Oracle.

- Frontend Web:

Interfaz gráfica desarrollada en PHP. Permite:

- Configurar hosts
- Crear dashboards
- Gestionar alertas
- Consultar métricas
- Administrar usuarios



- Agentes Zabbix:

Instalados en los hosts monitorizados. Recogen métricas como:

- Uso de CPU
- Memoria
- Espacio en disco
- Procesos
- Servicios activos

- Proxies:

Permiten monitorizar redes remotas o distribuidas sin sobrecargar el servidor principal.

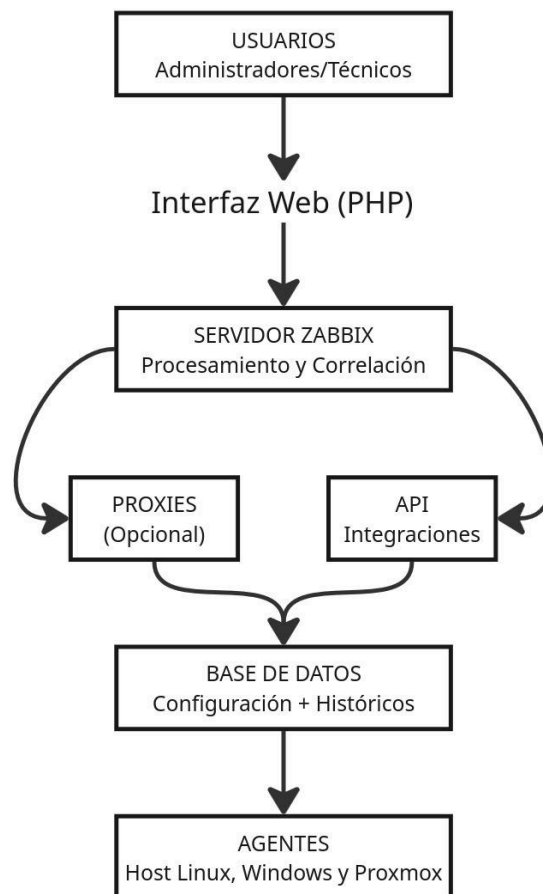
- Integraciones y APIs:

Zabbix dispone de una API REST que permite automatizar configuraciones, exportar datos o integrarlo con otras herramientas.

2.3. ARQUITECTURA GENERAL DE ZABBIX

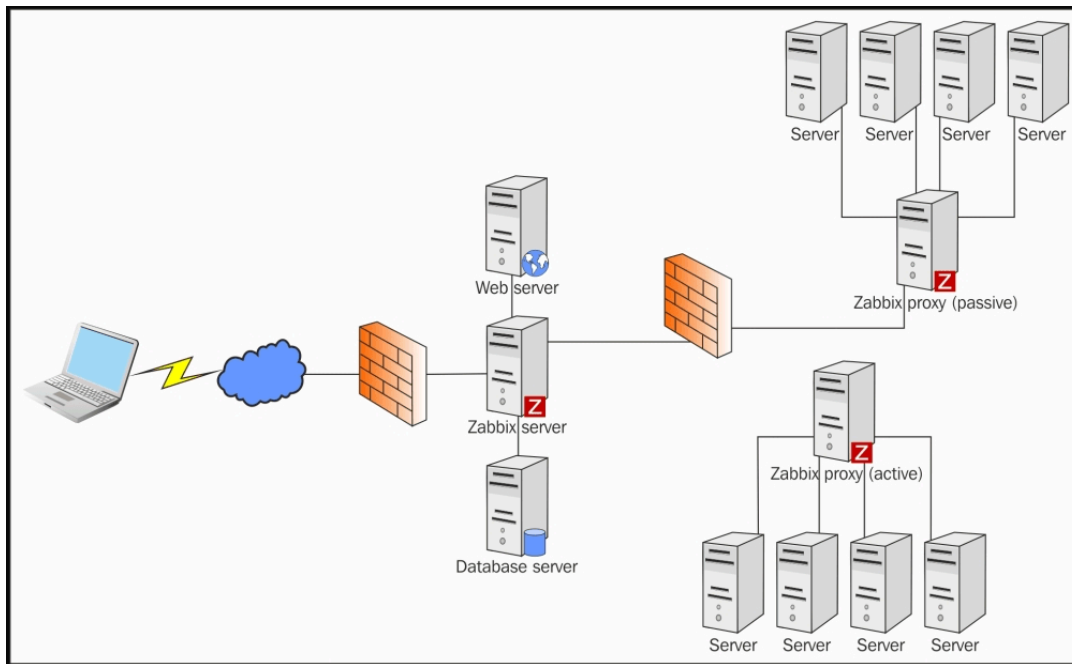
Zabbix se basa en una arquitectura distribuida compuesta por los siguientes componentes:

Esta arquitectura permite escalar horizontalmente, distribuir la carga y garantizar alta disponibilidad.



3. ARQUITECTURA DE ZABBIX

Zabbix se basa en una arquitectura distribuida diseñada para ofrecer un sistema de monitorización escalable, seguro y eficiente. Su estructura modular permite supervisar infraestructuras de cualquier tamaño, desde pequeños laboratorios educativos hasta grandes centros de datos empresariales.



3.1. COMPONENTES PRINCIPALES DE LA ARQUITECTURA DE ZABBIX

La arquitectura de Zabbix está formada por los siguientes elementos:

3.1.1. ZABBIX SERVER

Es el núcleo del sistema y el componente más importante. Sus funciones incluyen:

- Recopilar y procesar métricas enviadas por agentes y proxies.
- Evaluar triggers y generar alertas.
- Ejecutar acciones automáticas.
- Gestionar la comunicación con la base de datos.
- Coordinar la monitorización distribuida.

El servidor es el responsable de la lógica central del sistema.

3.1.2. BASE DE DATOS

Zabbix almacena toda su información en una base de datos SQL. Esta incluye:

- Configuración del sistema
- Históricos de métricas
- Eventos y alertas
- Usuarios y permisos
- Dashboards y mapas

En este proyecto se utiliza **MariaDB**, totalmente compatible con Zabbix 7.0 LTS.

3.1.3. FRONTEND WEB

El frontend está desarrollado en PHP y permite:

- Configurar hosts, plantillas y agentes
- Crear dashboards y mapas
- Consultar métricas en tiempo real
- Administrar usuarios y permisos
- Revisar alertas y eventos

Es accesible desde cualquier navegador web.

3.1.4. AGENTES ZABBIX

Los agentes se instalan en los hosts monitorizados (Linux, Windows, etc.) y permiten:

- Recoger métricas detalladas del sistema
- Ejecutar comprobaciones activas y pasivas
- Enviar datos al servidor o proxy
- Monitorizar servicios, procesos y recursos locales

En este proyecto se utilizan agentes en:

- Debian 12
- Proxmox
- Otros equipos de red compatibles

3.1.5. PROXIES (OPCIONALES)

Los proxies permiten distribuir la carga de trabajo y monitorizar redes remotas. Sus funciones:

- Recopilar métricas en redes separadas
- Almacenar temporalmente datos
- Enviar información al servidor principal

Aunque no son obligatorios, son muy útiles en entornos grandes.

3.1.6. INTEGRACIONES Y APIS

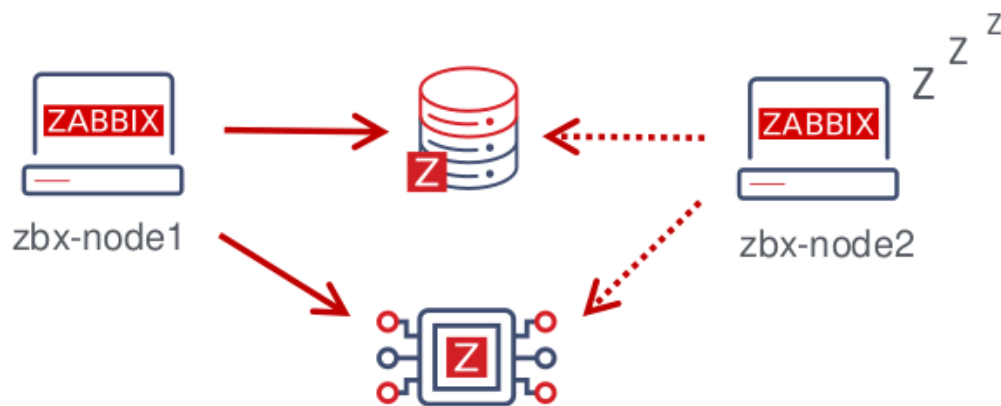
Zabbix dispone de una API REST que permite:

- Automatizar configuraciones
- Integrar herramientas externas
- Crear scripts personalizados
- Conectar con plataformas como ClickUp, GitHub, Grafana, Docker o Kubernetes

3.2. FLUJO DE FUNCIONAMIENTO

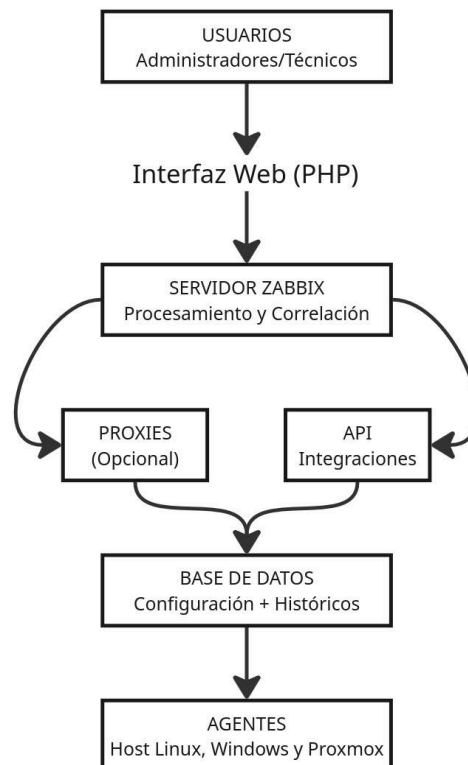
El flujo de trabajo en Zabbix sigue esta secuencia:

1. **El agente** recoge métricas del host.
2. **El servidor** recibe los datos y los almacena en la base de datos.
3. **Los triggers** analizan las métricas y detectan problemas.
4. **El sistema de alertas** genera notificaciones o acciones automáticas.
5. **El frontend** muestra dashboards, gráficos y eventos.



3.3. VENTAJAS DE LA ARQUITECTURA

- Escalabilidad: permite añadir proxies y agentes sin reconfigurar el servidor.
- Modularidad: cada componente cumple una función clara.
- Flexibilidad: compatible con múltiples protocolos y sistemas.
- Alta disponibilidad: puede configurarse en clúster.
- Seguridad: soporta cifrado TLS entre agentes y servidor.



4. REQUISITOS DEL SISTEMA

La instalación y funcionamiento de Zabbix depende de una combinación adecuada de recursos hardware y software. Estos requisitos cambian en función del tamaño de la infraestructura a monitorizar, el número de métricas procesadas por segundo y la carga generada por los agentes, proxies y consultas del frontend.

En este punto se detallan los requisitos mínimos y recomendados para garantizar un rendimiento óptimo del sistema, usando la documentación oficial de Zabbix y en la experiencia práctica obtenida durante el proyecto.

4.1. REQUISITOS HARDWARE

Los requisitos hardware dependen directamente del tamaño de la instalación y del número de métricas que Zabbix debe procesar. La siguiente tabla resume las recomendaciones oficiales:

TAMAÑO	MÉTRICAS	CPU	RAM	BASE DE DATOS	TIPO DE INSTANCIA
Pequeña	1.000	2	8Gb	MySQL, MariaDB, PostgreSQL	m6i.large
Mediana	10.000	4	16Gb		m6i.xlarge
Grande	100.000	16	64Gb		m6i.4xlarge
Muy Grande	1.000.000	32	96Gb		m6i.8xlarge

4.1.1. REQUISITOS HARDWARE DE NUESTRO PROYECTO

Para este proyecto se ha optado por una instalación **mediana**, adaptada a los recursos disponibles en el aula:

- **CPU:** 3 vCPUs
- **RAM:** 10 GB asignados a la máquina virtual
- **Disco:** 200 GB
- **Virtualización:** VirtualBox
- **Red:** Adaptador NAT + Adaptador solo-anfitrión

Aunque Zabbix recomienda 16 GB de RAM para instalaciones medianas, se ha ajustado a 10 GB debido a las limitaciones del equipo físico, manteniendo un rendimiento adecuado.

4.2. REQUISITOS SOFTWARE

Zabbix es compatible con una amplia variedad de sistemas operativos, bases de datos y servidores web. Para este proyecto se ha seleccionado una configuración estable, segura y totalmente soportada.

4.2.1. SISTEMA OPERATIVO

Debian 12 (Bookworm) Es la distribución recomendada por Zabbix por su estabilidad, soporte a largo plazo y compatibilidad con los paquetes oficiales.

4.2.2. SERVIDOR WEB

Apache 2.4 Utilizado para servir el frontend PHP de Zabbix.

4.2.3. PHP

Zabbix requiere una versión moderna de PHP con extensiones específicas.

En este proyecto se utiliza:

- PHP 8.2
- Extensiones necesarias:
 - bcmath
 - mbstring
 - gd
 - xml
 - sockets
 - mysqli
 - json

4.2.4. BASE DE DATOS

MariaDB 10.11 Elegida por su rendimiento, compatibilidad con MySQL y facilidad de administración.

4.2.5. AGENTES

Zabbix Agent 2

Instalado en:

- Debian 12
- Proxmox
- Otros equipos de red compatibles

4.3. REQUISITOS DEL FRONTEND

Durante la instalación del frontend, Zabbix realiza una comprobación automática de los requisitos PHP. Los valores obtenidos en este proyecto fueron:

PARÁMETRO	VALOR ACTUAL	REQUERIDO	ESTADO
PHP version	7.4.3	≥ 7.2	Ok
memory_limit	128M	128M	Ok
post_max_size	16M	16M	Ok
upload_max_filesize	2M	2M	Ok
max_execution_time	300	300	Ok
max_input_time	300	300	Ok
date.timezone	Europe/Madrid		Ok
PHP databases support	MySQL		Ok
PHP bcmath	ON		Ok
PHP mbstring	ON		Ok

4.4. REQUISITOS ADICIONALES

Además de los requisitos principales, Zabbix necesita:

4.4.1. SINCRONIZACIÓN HORARIA

Servicio NTP para evitar inconsistencias en las métricas.

4.4.2. RED ESTABLE

Comunicación constante entre agentes, servidor y base de datos.

4.4.3. ACCESO SSH

Para administración remota y despliegue de agentes.

4.4.4. VIRTUALBOX GUEST ADDITIONS

Para mejorar rendimiento, copiar/pegar y resolución gráfica.

5. VENTAJAS E INCONVENIENTES DE ZABBIX

Zabbix es una de las plataformas de monitorización más completas del mercado. Sin embargo, como cualquier solución tecnológica, presenta ventajas y limitaciones que deben tenerse en cuenta a la hora de planificar su implementación.

5.1. VENTAJAS

Zabbix destaca por una serie de características que lo convierten en una herramienta muy potente y versátil:

5.1.1. ESCALABILIDAD

Zabbix puede crecer desde pequeñas instalaciones con unos pocos hosts hasta infraestructuras con millones de métricas.

Esto se consigue gracias a:

- Proxies distribuidos
- Optimización de consultas
- Almacenamiento eficiente
- Arquitectura modular

Es ideal para entornos que pueden expandirse con el tiempo.

5.1.2. FIABILIDAD Y ESTABILIDAD

Zabbix está diseñado para funcionar de forma continua, incluso en entornos críticos. Su sistema de alertas y triggers permite detectar fallos antes de que afecten al servicio.

5.1.3. RENTABILIDAD

Al ser **software libre**, no requiere licencias de pago. Esto reduce significativamente los costes frente a soluciones comerciales como Datadog o PRTG.

5.1.4. VISIBILIDAD TOTAL

Zabbix permite monitorizar:

- Servidores físicos y virtuales
- Equipos de red
- Bases de datos
- Servicios web
- Contenedores
- Aplicaciones
- Sistemas cloud

Todo desde un único panel centralizado.

5.1.5. FLEXIBILIDAD Y PERSONALIZACIÓN

Zabbix permite:

- Crear plantillas personalizadas
- Definir triggers complejos
- Automatizar acciones
- Integrar scripts externos
- Diseñar dashboards avanzados

Esto facilita adaptarlo a cualquier entorno.

5.1.6. INTEGRACIÓN CON HERRAMIENTAS EXTERNAS

Zabbix se integra con:

- Grafana
- Docker / Kubernetes
- Ansible / Puppet
- AWS / Azure / GCP
- ClickUp, Jira, ServiceNow
- GitHub
- APIs REST

Esto amplía enormemente sus capacidades.

5.1.7. COMUNIDAD ACTIVA Y SOPORTE

Miles de usuarios contribuyen con:

- Documentación
- Plantillas
- Scripts
- Foros
- Soluciones a problemas comunes

Además, Zabbix SIA ofrece soporte empresarial opcional.

5.2. INCONVENIENTES

Aunque Zabbix es una herramienta muy potente, también presenta algunas limitaciones que deben considerarse:

5.2.1. CURVA DE APRENDIZAJE ELEVADA

La configuración inicial puede resultar compleja para usuarios sin experiencia en:

- Administración de sistemas
- Redes
- Bases de datos
- Monitorización avanzada

Requiere tiempo para dominarlo.

5.2.2. INTERFAZ TRADICIONAL

Aunque funcional, la interfaz web puede parecer menos moderna que la de herramientas como Grafana o Datadog. Algunos paneles requieren configuración manual.

5.2.3. ESCALABILIDAD MANUAL

Para entornos muy grandes, es necesario:

- Optimizar la base de datos
- Ajustar parámetros de rendimiento
- Planificar la arquitectura

Esto puede complicar la implementación.

5.2.4. DEPENDENCIA DE LA BASE DE DATOS

El rendimiento de Zabbix depende en gran medida de:

- La optimización de MariaDB/MySQL
- El tamaño del histórico
- La frecuencia de las métricas

Una mala configuración puede afectar al rendimiento global.

5.2.5. ALERTAS AVANZADAS REQUIEREN SCRIPTING

Aunque Zabbix permite automatizar acciones, las más complejas requieren:

- Scripts personalizados
- Uso de la API
- Integraciones externas

Esto implica conocimientos adicionales.

5.2.6. ACTUALIZACIONES DELICADAS

En entornos críticos, actualizar Zabbix puede requerir:

- Pruebas previas
- Copias de seguridad
- Migración de base de datos
- Ajustes de compatibilidad

No siempre es un proceso trivial.

6. SEGURIDAD EN ZABBIX

La seguridad es un aspecto fundamental en cualquier sistema de monitorización, ya que Zabbix gestiona información crítica sobre servidores, redes, servicios y dispositivos. Un fallo de seguridad podría comprometer la integridad de la infraestructura monitorizada o permitir accesos no autorizados a datos sensibles. Zabbix incorpora múltiples mecanismos de protección que garantizan la confidencialidad, integridad y disponibilidad del sistema, tanto en la comunicación entre componentes como en la gestión interna de usuarios y permisos.

6.1. CIFRADO Y COMUNICACIONES SEGURAS

Zabbix permite cifrar las comunicaciones entre:

- Servidor ↔ Agentes
- Servidor ↔ Proxies
- Servidor ↔ Frontend
- API ↔ Aplicaciones externas

El cifrado se realiza mediante **TLS/SSL**, evitando que las métricas viajen sin protección por la red.

6.1.1. TIPOS DE CIFRADO SOPORTADO

- **PSK (Pre-Shared Key)**
 - Clave compartida entre servidor y agente.
 - Fácil de implementar en entornos pequeños.
- **Certificados TLS**
 - Basados en infraestructura de clave pública (PKI).
 - Recomendado para entornos profesionales.

6.1.2. BENEFICIOS DEL CIFRADO

- Evita ataques de interceptación (Man-in-the-Middle).
- Protege credenciales y datos sensibles.
- Garantiza la autenticidad del agente o proxy.

6.2. AUTENTICACIÓN Y CONTROL DE ACCESO

Zabbix incorpora un sistema de autenticación robusto que permite gestionar usuarios y permisos de forma granular.

6.2.1. MÉTODOS DE AUTENTICACIÓN DISPONIBLES

- **Autenticación interna** (por defecto)
- **LDAP / Active Directory**
- **SAML**
- **Autenticación HTTP**

6.2.2. CONTROL DE ACCESO BASADO EN ROLES (RBAC)

Zabbix permite definir:

- Grupos de usuarios
- Roles personalizados
- Permisos por host, grupo o acción
- Acceso de solo lectura o lectura/escritura

Esto garantiza que cada usuario solo pueda ver o modificar aquello que le corresponde.

6.3. GESTIÓN DE CONTRASEÑAS

Zabbix protege la información sensible mediante:

- **Hash SHA-256** para contraseñas
- **Enmascaramiento de secretos** en la interfaz
- **Protección de macros sensibles**
- **Restricción de acceso a scripts remotos**

Esto evita que credenciales o claves queden expuestas accidentalmente.

6.4. SEGURIDAD EN LA BASE DE DATOS

La base de datos es uno de los componentes más críticos del sistema.

Zabbix recomienda:

- Crear un usuario exclusivo para Zabbix
- Restringir el acceso a localhost
- Usar contraseñas robustas
- Activar `log_bin_trust_function_creators` solo cuando sea necesario
- Realizar copias de seguridad periódicas
- Limitar el acceso mediante firewall

Así solventamos riesgos como:

- Acceso no autorizado
- Manipulación de históricos
- Pérdida de datos
- Inyección SQL

6.5. SEGURIDAD EN EL FRONTEND

El frontend de Zabbix incluye medidas de protección como:

- Validación de sesiones
- Protección CSRF
- Restricción de intentos de login
- Auditoría de acciones
- Configuración de HTTPS en Apache o Nginx

Además, permite registrar:

- Cambios de configuración
- Accesos de usuarios
- Acciones ejecutadas

Esto facilita el cumplimiento de normativas y auditorías internas.

6.6. SEGURIDAD EN AGENTES Y PROXIES

Los agentes y proxies deben configurarse correctamente para evitar accesos indebidos.

Zabbix recomienda:

- Limitar direcciones IP permitidas
- Activar cifrado TLS
- Deshabilitar comandos remotos si no son necesarios
- Mantener los agentes actualizados
- Usar firewalls para restringir puertos

Los puertos más importantes de zabbix son:

Componente	Puerto	Protocolo
Zabbix Server	10051	TCP
Zabbix Agent	10050	TCP
Proxy	10051	TCP
Frontend	80/443	HTTP/HTTPS

6.7. AUDITORÍA Y REGISTRO DE EVENTOS

Zabbix registra:

- Accesos de usuarios
- Cambios de configuración
- Ejecución de scripts
- Problemas detectados
- Acciones automáticas

Estos registros permiten:

- Detectar comportamientos sospechosos
- Realizar análisis forense
- Cumplir políticas de seguridad

7. INTEGRACIONES DE ZABBIX

Zabbix es una plataforma altamente extensible que permite integrarse con una amplia variedad de herramientas, servicios y tecnologías. Estas integraciones amplían sus capacidades, facilitan la automatización de tareas, mejoran la visualización de datos y permiten conectar el sistema de monitorización con otros componentes de la infraestructura.

Gracias a su API REST, sus plantillas oficiales y su compatibilidad con protocolos estándar, Zabbix puede adaptarse a prácticamente cualquier entorno, desde redes locales hasta infraestructuras cloud o sistemas de contenedores.

7.1. INTEGRACIÓN MEDIANTE API REST

Zabbix dispone de una API REST completa que permite:

- Crear, modificar o eliminar hosts
- Gestionar plantillas
- Consultar métricas
- Crear dashboards
- Automatizar configuraciones
- Integrar herramientas externas

La API REST utiliza JSON-RPC y permite automatizar tareas que, de otro modo, requerirían intervención manual desde el frontend.

Los usos más habituales de la API son:

- Automatizar el alta de nuevos hosts
- Crear alertas o triggers desde scripts
- Integrar Zabbix con sistemas de tickets
- Exportar métricas a otras plataformas
- Generar informes personalizados

7.2. INTEGRACIÓN CON GRAFANA

Grafana es una de las herramientas de visualización más utilizadas en entornos profesionales.

Zabbix se integra con Grafana mediante un plugin oficial que permite:

- Crear dashboards avanzados
- Combinar métricas de Zabbix con otras fuentes (Prometheus, Loki, InfluxDB...)
- Diseñar paneles interactivos
- Mejorar la visualización de históricos

Las ventajas de usar Grafana con Zabbix son:

- Dashboards más modernos
- Mayor personalización
- Mejor experiencia visual
- Integración con múltiples fuentes de datos

7.3. INTEGRACIÓN CON SISTEMAS DE TICKETS

Zabbix puede integrarse con herramientas de gestión de incidencias como:

- Clickup
- Jira
- ServiceNow
- GLPI
- OTRS

Esto permite que, cuando se detecta un problema, Zabbix:

- Cree un ticket automáticamente
- Asigne la incidencia a un técnico
- Actualiza el estado del ticket según la evolución del problema

En este proyecto, la integración conceptual se realiza con ClickUp, utilizando como herramienta de gestión del proyecto.

7.4. INTEGRACIÓN CON SISTEMAS CLOUD

Zabbix ofrece plantillas oficiales para monitorizar servicios de:

- Amazon Web Services (AWS)
- Microsoft Azure
- Google Cloud Platform (GCP)

Estas integraciones permiten supervisar:

- Máquinas virtuales
- Bases de datos gestionadas
- Balanceadores
- Almacenamiento
- Servicios serverless

La conexión se realiza mediante APIs de cada proveedor.

7.5. INTEGRACIÓN CON CONTENEDORES Y KUBERNETES

Zabbix puede monitorizar entornos basados en contenedores mediante:

- Docker Engine API
- Plantillas oficiales para Docker
- Integración con Kubernetes a través de Zabbix Agent 2
- Exporters compatibles

Permite obtener métricas como:

- Uso de CPU y RAM por contenedor
- Estado de pods y nodos
- Consumo de red
- Eventos del clúster

7.6. INTEGRACIÓN CON PROXMOX

Proxmox es una de las plataformas más utilizadas en entornos educativos y empresariales para virtualización.

Zabbix puede integrarse con Proxmox mediante:

- Plantillas oficiales
- API de Proxmox
- Agentes instalados en nodos y máquinas virtuales

Permite monitorizar:

- Estado de nodos
- Consumo de recursos
- Máquinas virtuales activas
- Almacenamiento
- Redes virtuales

Esta integración es especialmente relevante en este proyecto, ya que Proxmox forma parte de la infraestructura monitorizada.

7.7. INTEGRACIÓN CON DISPOSITIVOS DE RED

Zabbix soporta monitorización mediante:

- SNMP v1/v2c/v3
- ICMP
- SSH/Telnet
- LLDP/CDP

Esto permite supervisar:

- Switches
- Routers
- Puntos de acceso
- Firewalls
- Impresoras de red

Las métricas más habituales incluyen:

- Tráfico por interfaz
- Estado de puertos
- Latencia
- Paquetes descartados
- Temperatura del dispositivo

7.8. INTEGRACIÓN CON HERRAMIENTAS DE AUTOMATIZACIÓN

Zabbix puede trabajar junto a herramientas como:

- Ansible
- Puppet
- Chef
- SaltStack

Esto permite:

- Desplegar agentes automáticamente
- Configurar hosts de forma masiva
- Aplicar plantillas sin intervención manual
- Mantener la infraestructura sincronizada

7.9. INTEGRACIÓN CON SISTEMAS DE NOTIFICACIONES

Zabbix permite enviar alertas mediante:

- Correo electrónico
- SMS
- Telegram
- Discord
- Slack
- Webhooks personalizados

Esto facilita que los técnicos reciban avisos inmediatos ante cualquier incidencia.

8. AUTOMATIZACIÓN EN ZABBIX

La automatización es uno de los pilares fundamentales de Zabbix. Gracias a su sistema de acciones, scripts remotos, macros, plantillas y API, es posible reducir la intervención manual, acelerar la resolución de incidencias y mejorar la eficiencia operativa del sistema de monitorización. En este capítulo se describen los mecanismos de automatización que ofrece Zabbix y cómo se aplican en un entorno real como el desarrollado en este proyecto.

8.1. ACCIONES AUTOMÁTICAS

Las acciones son reglas que permiten ejecutar tareas de forma automática cuando se cumplen determinadas condiciones.

Estas condiciones se basan en:

- Triggers
- Severidad del problema
- Estado del host
- Grupos de hosts
- Tiempo transcurrido
- Valores de métricas

Zabbix permite dos tipos de acciones principales:

- Acciones de notificación
 - Envían avisos a los técnicos mediante:
 - Correo electrónico
 - Telegram
 - Slack
 - Discord
 - SMS
 - Webhooks personalizados

- Acciones remotas
 - Ejecutan comandos en los hosts monitorizados, como:
 - Reiniciar un servicio
 - Limpiar memoria caché
 - Reiniciar un contenedor
 - Ejecutar scripts de mantenimiento
 - Lanzar comprobaciones adicionales

Estas acciones permiten que Zabbix no solo detecte problemas, sino que también actúa para resolverlos.

8.2. SCRIPTS REMOTOS

Zabbix permite ejecutar scripts directamente desde el servidor o desde el agente.

Estos scripts pueden ser:

- Bash
- PowerShell
- Python
- Comandos simples del sistema

Los usos más habituales de estos scripts son:

- Reiniciar Apache o Nginx
- Comprobar el estado de un servicio
- Limpiar logs
- Reiniciar máquinas virtuales
- Ejecutar tareas de mantenimiento

8.3. MACROS

Las macros son variables que permiten personalizar configuraciones sin necesidad de modificar cada host individualmente.

Tipos de macros:

- Macros globales
- Macros de plantilla
- Macros de host
- Macros de usuario
- Macros sensibles (secretas)

Las macros facilitan la automatización al permitir que una misma plantilla funcione en múltiples hosts con configuraciones diferentes.

8.4. PLANTILLAS

Las plantillas son uno de los mecanismos más potentes de automatización en Zabbix. Permiten aplicar configuraciones completas a múltiples hosts sin necesidad de configurarlos uno por uno.

Una plantilla puede incluir:

- Ítems (métricas)
- Triggers
- Gráficos
- Reglas de descubrimiento
- Macros
- Scripts
- Dashboards

Ventajas de usar plantillas:

- Ahorro de tiempo
- Consistencia en la monitorización
- Facilidad de mantenimiento
- Escalabilidad

8.5. DESCUBRIMIENTO AUTOMÁTICO (LLD)

El Low-Level Discovery (LLD) permite que Zabbix detecte automáticamente:

- Interfaces de red
- Sistemas de archivos
- Procesos
- Discos
- Contenedores
- Servicios
- Máquinas virtuales

Ejemplo:

Si un servidor añade una nueva interfaz de red, Zabbix la detecta automáticamente y crea:

- Métricas
- Triggers
- Gráficos

Sin intervención manual.

8.6. DESCUBRIMIENTO DE RED

Zabbix puede escanear una red completa para:

- Detectar nuevos dispositivos
- Identificar servicios activos
- Añadir hosts automáticamente
- Aplicar plantillas según reglas

Esto es especialmente útil en redes educativas o empresariales donde los dispositivos pueden cambiar con frecuencia.

8.7. AUTOMATIZACIÓN MEDIANTE API

La API de Zabbix permite automatizar tareas avanzadas como:

- Alta de nuevos hosts
- Aplicación de plantillas
- Creación de triggers
- Exportación de métricas
- Integración con ClickUp o GitHub

9. MÉTRICAS EN ZABBIX

Las métricas son el elemento central de cualquier sistema de monitorización. En Zabbix, una métrica se denomina **ítem** y representa un dato concreto que el sistema recoge de un host, un servicio o un dispositivo de red.

Estas métricas permiten evaluar el estado de la infraestructura, detectar problemas, generar alertas y visualizar información en tiempo real mediante gráficos y dashboards.

Zabbix ofrece una gran variedad de tipos de métricas, desde recursos básicos del sistema hasta datos avanzados obtenidos mediante protocolos como SNMP, IPMI, JMX o consultas HTTP.

9.1. TIPOS DE MÉTRICAS EN ZABBIX

Zabbix clasifica las métricas según la forma en la que se obtienen y el tipo de información que proporcionan.

Las métricas de sistema operativo son las más comunes y permiten monitorizar:

- CPU
 - Uso total
 - Carga media
 - Uso por núcleo
- Memoria RAM
 - Memoria usada
 - Memoria libre
 - Swap
- Disco
 - Espacio disponible
 - Lecturas/escrituras por segundo
 - Uso de inodos
- Red
 - Tráfico entrante y saliente
 - Errores
 - Paquetes descartados

- Procesos
 - Número de procesos activos
 - Servicios en ejecución
 - Procesos críticos

Estas métricas se obtienen mediante Zabbix Agent 2, instalado en los hosts monitorizados.

Las métricas de servicios y aplicaciones, en ellas Zabbix permite monitorizar servicios como:

- Apache / Nginx
- MySQL / MariaDB
- SSH
- DNS
- DHCP
- Samba
- Proxmox VE
- Docker

Ejemplos de métricas:

- Estado del servicio
- Tiempo de respuesta
- Número de conexiones
- Consultas por segundo

Las métricas SNMP, este es el protocolo estándar para monitorizar dispositivos de red:

- Switches
- Routers
- Firewalls
- Puntos de acceso
- Impresoras

Usos típicos:

- Estado de interfaces
- Tráfico por puerto

- Temperatura del dispositivo
- Uso de CPU del switch
- Estado de la fuente de alimentación

Las métricas ICMP (Ping) permiten comprobar:

- Disponibilidad del host
- Latencia
- Pérdida de paquetes

Son útiles para detectar caídas de red o problemas de conectividad.

Las métricas HTTP/HTTPS las usa Zabbix para realizar comprobaciones web para monitorizar:

- Estado de páginas web
- Tiempo de respuesta
- Códigos de error
- Certificados SSL

Las métricas personalizadas permiten a Zabbix crear métricas propias mediante:

- Scripts
- Comandos remotos
- Consultas API
- Archivos de log
- UserParameters

Esto permite adaptar la monitorización a cualquier necesidad.

9.2. FRECUENCIA DE ACTUALIZACIÓN (INTERVALOS)

Cada métrica tiene un intervalo de actualización configurable:

- 1 segundo → Métricas críticas
- 30–60 segundos → Métricas estándar
- 5 minutos → Métricas poco cambiantes
- 1 hora o más → Métricas de inventario

Un intervalo demasiado bajo puede sobrecargar el servidor o la base de datos.

9.3. TRIGGERS BASADOS EN MÉTRICAS

Los triggers son reglas que analizan las métricas y detectan problemas.

Ejemplos:

- CPU > 90% durante 5 minutos
- Memoria libre < 10%
- Latencia > 100 ms
- Servicio Apache caído
- Switch sin respuesta por SNMP

Los triggers permiten generar alertas y ejecutar acciones automáticas.

9.4. VISUALIZACIÓN DE MÉTRICAS

Zabbix ofrece varias formas de visualizar métricas:

- Gráficos

Representan la evolución de una métrica en el tiempo.

- Dashboards

Paneles personalizados con:

- Gráficos
- Mapas
- Widgets
- Listas de problemas

- Mapas de red

Muestran la topología de la infraestructura.

- Pantallas (screens)

Conjuntos de gráficos organizados.

- Vistas de problemas

Lista de alertas activas ordenadas por severidad.

9.5. IMPORTANCIA DE LAS MÉTRICAS EN LA MONITORIZACIÓN

Las métricas permiten:

- Detectar fallos antes de que afecten al usuario
- Identificar cuellos de botella
- Optimizar recursos
- Prevenir caídas del sistema
- Automatizar acciones correctivas
- Generar informes y estadísticas

Sin métricas, Zabbix no podría funcionar como sistema de monitorización.

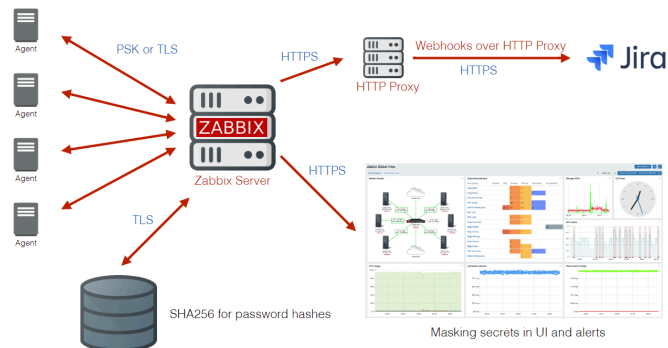
10. ESTADO DEL ARTE

La monitorización de sistemas ha evolucionado de forma significativa en los últimos años, pasando de simples comprobaciones de disponibilidad a complejos sistemas de observabilidad capaces de analizar miles de métricas en tiempo real. En este capítulo se analizan las principales tecnologías, tendencias y herramientas actuales del sector, con especial atención a Zabbix y a su posición dentro del ecosistema moderno de monitorización.

10.1. ZABBIX

Zabbix es una plataforma de código abierto que permite monitorizar infraestructuras completas desde un único punto de control. Su arquitectura distribuida, basada en agentes, proxies y un

servidor central, permite escalar desde pequeñas redes hasta entornos empresariales con millones de métricas.



Características principales:

- **Recopilación de datos en tiempo real** desde servidores, dispositivos de red, servicios cloud y contenedores.
- **Alertas personalizables** con condiciones complejas, escalado y múltiples canales (email, SMS, webhooks).
- **Visualización avanzada** mediante dashboards, mapas topológicos, gráficos y reportes.
- **Automatización de acciones** ante eventos: reinicio de servicios, ejecución de scripts, generación de tickets.
- **Compatibilidad multiplataforma:** Linux, Windows, macOS, BSD, Solaris, AIX, etc.
- **Integración con herramientas externas** como Grafana, Ansible, Docker, Kubernetes, AWS, Azure, ClickUp y GitHub.

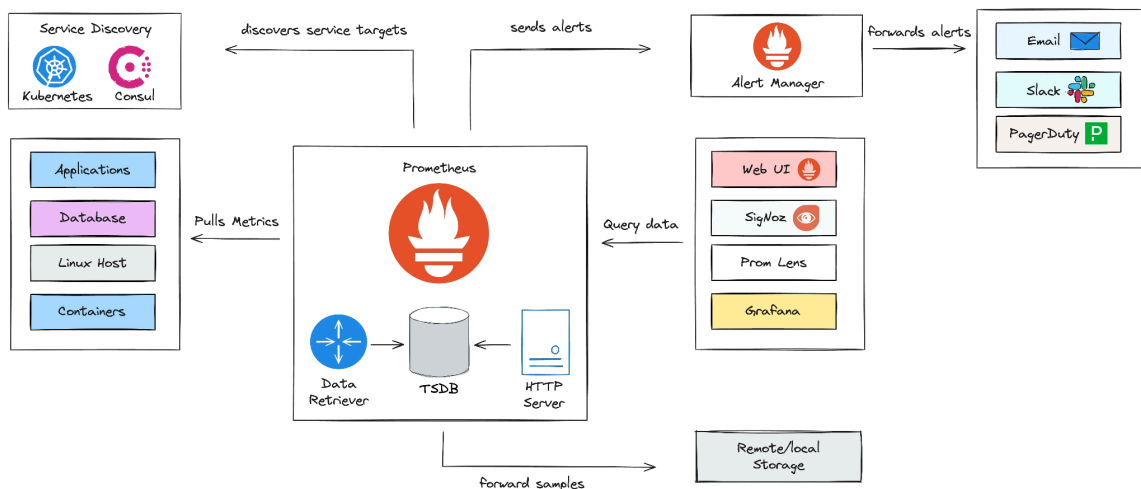
Zabbix destaca por su equilibrio entre potencia, flexibilidad y coste (gratuito).

10.2. PROMETHEUS

Prometheus es un sistema de monitorización de código abierto diseñado para recopilar métricas en formato de series temporales. Su arquitectura basada en exporters y un servidor central lo convierte en una herramienta ideal para entornos cloud-native y sistemas distribuidos como Kubernetes.

Características principales:

- Recopilación eficiente de métricas con baja latencia.
- Lenguaje de consultas PromQL para análisis avanzado.
- Integración nativa con Grafana para visualización.
- Alertmanager para gestión de notificaciones.
- Exporters disponibles para múltiples tecnologías.
- Escalabilidad horizontal mediante federación y almacenamiento externo.

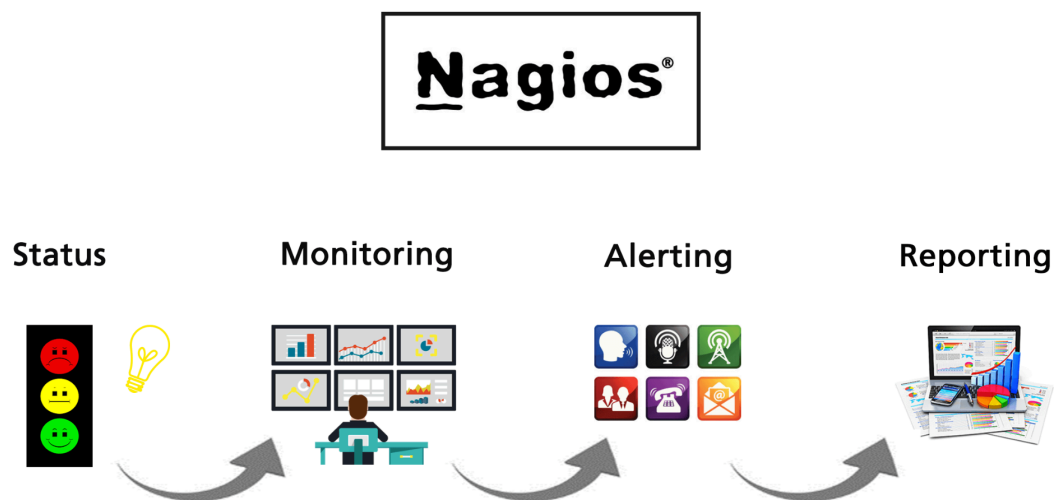


10.3. NAGIOS

Nagios es otro sistema de monitorización que permite supervisar servicios, recursos y dispositivos mediante un sistema basado en plugins. Su arquitectura modular facilita la personalización en entornos tradicionales.

Características principales:

- Supervisión de disponibilidad y rendimiento de servicios.
- Sistema de alertas básico con notificaciones por email.
- Gran cantidad de plugins disponibles para ampliar funcionalidades.
- Compatible con Linux, Windows y dispositivos de red.
- Interfaz web sencilla para visualización de estados.
- Integración con NRPE para ejecución remota de comandos.

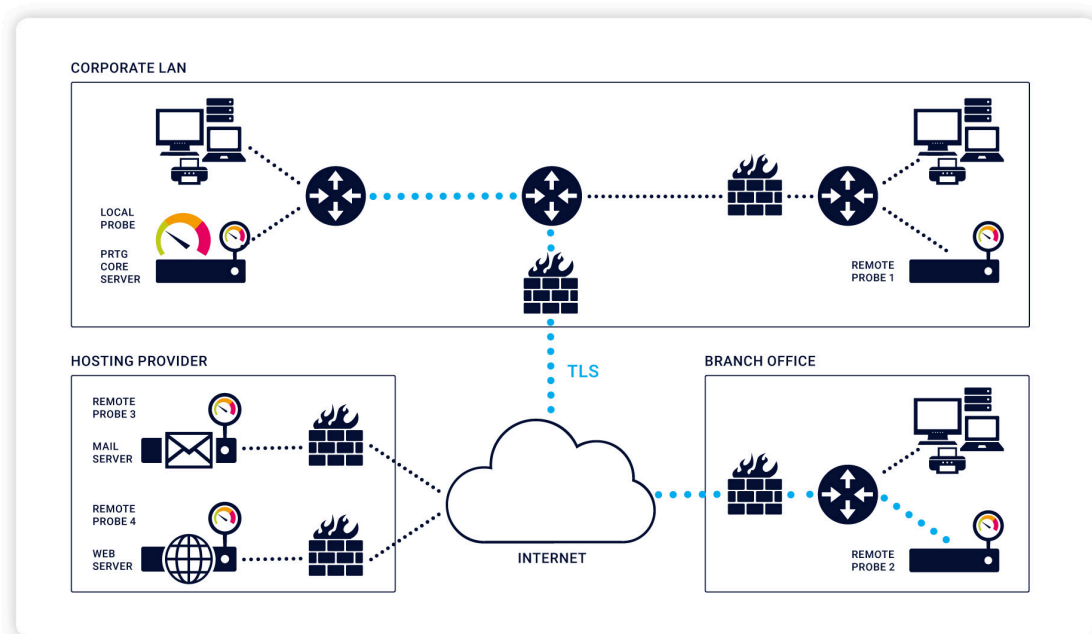


10.4. PRTG NETWORK MONITOR

PRTG es una solución comercial que permite monitorizar redes, servidores y servicios mediante sensores preconfigurados. Su arquitectura basada en sondas remotas y un servidor central facilita la implementación rápida en entornos medianos.

Características principales:

- Sensores listos para usar para múltiples tecnologías.
- Interfaz gráfica moderna y aplicación móvil.
- Alertas inteligentes con umbrales dinámicos.
- Informes automáticos y mapas de red interactivos.
- Alta disponibilidad mediante configuración en clúster.
- Integración con servicios cloud y virtualización.



11. ESTUDIO DE VIABILIDAD

El estudio de viabilidad analiza si el proyecto es factible desde los puntos de vista técnico, económico, operativo y temporal. Su objetivo es determinar si la implantación de un sistema de monitorización basado en Zabbix es adecuada para el entorno educativo del IES Guadalpeña y si puede desarrollarse con los recursos disponibles.

Este análisis permite justificar la realización del proyecto y asegurar que su ejecución es realista y sostenible.

11.1. VIABILIDAD TÉCNICA

La viabilidad técnica evalúa si el proyecto puede llevarse a cabo con los recursos tecnológicos disponibles.

- Compatibilidad del software:

Zabbix 7.0 LTS es totalmente compatible con:

- Debian 12 (sistema operativo elegido)
- MariaDB 10.11 (base de datos utilizada)
- Apache 2.4 y PHP 8.2 (frontend)
- Agentes para Linux, Windows y Proxmox
- Dispositivos de red mediante SNMP

No existe ninguna incompatibilidad técnica que impida su instalación.

- Recursos hardware disponibles:

El proyecto se ejecuta en una máquina virtual con:

- 3 vCPUs
- 10 GB de RAM
- 200 GB de almacenamiento
- VirtualBox como plataforma de virtualización

Aunque Zabbix recomienda 16 GB para instalaciones medianas, los 10 GB disponibles han demostrado ser suficientes para el entorno.

- Infraestructura del centro:

El Departamento de Informática dispone de:

- Servidores Proxmox
- Equipos Debian 12
- Dispositivos de red compatibles con SNMP
- Red estable para comunicación entre agentes y servidor

Todo ello permite desplegar un entorno de monitorización realista.

- Conclusión técnica

El proyecto es técnicamente viable, ya que todos los componentes necesarios están disponibles y son compatibles.

11.2. VIABILIDAD ECONÓMICA

Para evaluar el coste real de implantar un sistema de monitorización basado en Zabbix, se ha calculado el precio de todos los equipos y licencias necesarios para montar la infraestructura completa desde cero.

El objetivo es estimar cuánto costaría implementar este proyecto en un entorno educativo real.

11.2.1. **COSTE DEL HARDWARE**

Para desplegar la infraestructura completa del proyecto, se requiere la adquisición de varios equipos físicos que formarán parte del entorno monitorizado. A continuación se detallan todos los dispositivos necesarios, junto con sus especificaciones técnicas completas y su coste aproximado.

- Servidor Zabbix (Debian 12)

Equipo físico dedicado, montado en rack, con capacidad suficiente para alojar el servidor Zabbix, la base de datos MariaDB y el frontend web.

- CPU Intel Xeon E3 o AMD EPYC básico
- 32 GB de RAM ECC
- 2× SSD de 480 GB en RAID 1
- Placa base con IPMI y doble interfaz de red
- Fuente 80+ Gold
- Chasis 1U para rack
- Regleta enrackable profesional

Precio aproximado: 860 €

- Equipo Proxmox

Servidor físico para virtualización, con recursos suficientes para alojar máquinas virtuales de prueba y monitorización.

- CPU AMD Ryzen 7
- 32 GB de RAM
- SSD NVMe de 1 TB
- Torre ATX con buena ventilación

Precio aproximado: 490 €

- Clientes Monitorizados

Se incluyen cuatro equipos físicos que simulan distintos entornos de trabajo:

- Cliente Debian 13 → PC básico con i3, 8 GB RAM, SSD 256 GB
- Cliente MX Linux → PC similar, con Linux ligero
- Cliente Windows Server 2022 → PC avanzado con 16 GB RAM, SSD 512 GB
- Incluye licencia Windows Server Standard (800 €)
- Incluye 10 CALs (180 €)
- Cliente Windows 10 → PC básico con licencia Windows 10 Pro (145 €)

Precio total aproximado de los 4 equipos: 2.725 €

- Switch gestionable SNMP

Dispositivo de red necesario para interconectar todos los equipos y permitir la monitorización por SNMP.

Se ha tomado como referencia el modelo TP-Link Festa FS308G, con 8 puertos Gigabit y gestión básica.

Precio aproximado: 61 €

- Accesorios de red

Incluye cableado Ethernet, regleta enrackable, patch panel y elementos de montaje.

Precio aproximado: 100 €

11.2.2. **COSTE DEL SOFTWARE**

El proyecto requiere la instalación de diversos sistemas operativos y herramientas para que cada equipo cumpla su función dentro de la infraestructura monitorizada. Aunque gran parte del software utilizado es libre y no supone coste económico, algunos equipos requieren licencias propietarias que sí deben contemplarse en el presupuesto.

- Software libre (sin coste)
 - Debian 12 (servidor Zabbix)
 - Debian 13 (cliente Linux)
 - MX Linux (cliente Linux ligero)
 - Proxmox VE (hipervisor de virtualización)
 - MariaDB 10.11 (base de datos)
 - Apache 2.4 (servidor web)
 - PHP 8.2 (lenguaje para el frontend)
 - Zabbix Server 7.0 LTS
 - Zabbix Agent 2 (para todos los clientes)

Estos componentes permiten construir un entorno profesional sin necesidad de adquirir licencias adicionales.

Coste total software libre: 0 €

- Software propietario (con coste)

Algunos equipos requieren sistemas operativos de pago, especialmente aquellos que deben simular entornos empresariales reales.

- Windows Server 2022 (cliente + servidor DHCP)

Incluye:

- Licencia Windows Server 2022 Standard
- 10 licencias de acceso de cliente (CALs) para permitir conexiones de red

Coste aproximado:

- Windows Server 2022 Standard → 800 €
- 10 CALs → 180 €

Total Windows Server: 980 €

- Windows 10 Pro (cliente)

El equipo Windows 10 requiere una licencia profesional para permitir:

- Integración en redes corporativas
- Políticas de grupo
- Monitorización avanzada
- Compatibilidad con Zabbix Agent 2

Coste aproximado: 145 €

11.2.3. **COSTE DE MANO DE OBRA**

La implantación de un sistema de monitorización completo requiere diversas tareas técnicas que deben ser realizadas por personal cualificado. Estas tareas incluyen la instalación física de los equipos, la configuración de los sistemas operativos, la puesta en marcha de los servicios de red, la instalación de Zabbix y la integración de todos los clientes en el entorno monitorizado.

Para este cálculo se ha tomado como referencia un coste estándar de **25 € por hora**, habitual en servicios técnicos de nivel profesional.

- **Tareas incluidas en la instalación**

A continuación se detallan las tareas necesarias y el tiempo estimado para cada una de ellas:

- Montaje de los equipos

Incluye:

- Instalación del servidor Zabbix en el rack
- Montaje del equipo Proxmox
- Montaje de los cuatro equipos cliente
- Conexión al switch y cableado estructurado

Tiempo estimado: 4 horas

Coste: 100€

- Instalación del servidor Zabbix

Incluye

- Instalación del sistema operativo
- Configuración de red
- Instalación de MariaDB, Apache y PHP
- Instalación y configuración de Zabbix Server y Frontend

Tiempo estimado: 6 horas

Coste: 150 €

- Instalación del servidor Proxmox

Incluye:

- Instalación del hipervisor
- Configuración de almacenamiento
- Configuración de red
- Pruebas de virtualización

Tiempo estimado: 3 horas

Coste: 75 €

- Instalación de clientes Linux

Incluye:

- Instalación del sistema operativo
- Configuración de red
- Instalación de Zabbix Agent 2
- Pruebas de conectividad con el servidor

Tiempo estimado: 4 horas

Coste: 100 €

- Instalación de Windows Server 2022

Incluye:

- Instalación del sistema operativo
- Activación de licencia
- Instalación del rol DHCP
- Configuración del ámbito y opciones
- Instalación de Zabbix Agent 2
- Pruebas de servicio

Tiempo estimado: 6 horas

Coste: 150 €

- Instalación de Windows 10

Incluye:

- Instalación del sistema operativo
- Activación de licencia
- Configuración de red
- Instalación de Zabbix Agent 2

Tiempo estimado: 2 horas

Coste: 50 €

- Configuración de Zabbix y agentes

Incluye:

- Alta de hosts en Zabbix
- Aplicación de plantillas
- Configuración de métricas
- Pruebas de monitorización
- Ajustes de triggers y alertas

Tiempo estimado: 8 horas

Coste: 200 €

- Pruebas finales

Incluye:

- Comprobación de alertas
- Verificación de métricas
- Pruebas de red
- Validación del entorno completo

Tiempo estimado: 3 horas

Coste: 75 €

11.3. VIABILIDAD OPERATIVA

La viabilidad operativa analiza si el sistema puede ser utilizado, administrado y mantenido correctamente por los usuarios finales y por el personal técnico del centro. Este aspecto es fundamental para garantizar que la infraestructura de monitorización no solo pueda instalarse, sino que también pueda mantenerse en funcionamiento a largo plazo sin requerir recursos extraordinarios.

11.3.1. **CAPACIDAD DEL PERSONAL TÉCNICO**

El Departamento de Informática del centro dispone de conocimientos suficientes en:

- Administración de sistemas Linux
- Redes y servicios
- Virtualización con Proxmox
- Gestión de sistemas Windows Server
- Monitorización y mantenimiento de infraestructuras

11.3.2. **FACILIDAD DE USO DEL SISTEMA**

Zabbix ofrece:

- Interfaz web intuitiva
- Dashboards personalizables
- Alertas automáticas
- Plantillas preconfiguradas
- Documentación extensa

11.3.3. **MANTENIMIENTO DEL SISTEMA**

El mantenimiento requerido es mínimo y se basa en:

- Actualizaciones periódicas del sistema operativo
- Revisión de alertas y métricas
- Copias de seguridad de la base de datos
- Sustitución de hardware en caso de fallo
- Ajustes de configuración según necesidades del aula

11.3.4. **INTEGRACIÓN CON EL ENTORNO EDUCATIVO**

El sistema se adapta perfectamente al entorno del centro porque:

- Permite monitorizar equipos reales utilizados en clase
- Facilita prácticas de administración de sistemas
- Permite a los alumnos visualizar problemas reales
- Mejora la gestión de los recursos del aula
- Se integra con servicios como DHCP, Proxmox y Linux

Esto convierte el proyecto en una herramienta pedagógica útil y

sostenible.

11.4. VIABILIDAD TEMPORAL

La viabilidad temporal analiza si el proyecto puede desarrollarse dentro del periodo establecido para el módulo de Proyecto Integrado. Para ello se ha elaborado una planificación realista basada en las tareas necesarias, la complejidad técnica del sistema y el tiempo disponible durante el curso académico.

El objetivo es garantizar que todas las fases del proyecto —desde la instalación del hardware hasta la configuración final de Zabbix— puedan completarse sin retrasos y con margen suficiente para realizar pruebas y correcciones.

- Planificación estimada del proyecto

A continuación se presenta una estimación del tiempo necesario para completar cada fase del proyecto:

1. Preparación y análisis inicial

Incluye investigación, recopilación de información, diseño de la infraestructura y planificación.

Duración estimada: 2 semanas

2. Montaje físico de la infraestructura

Instalación del servidor Zabbix, equipo Proxmox, clientes y switch.

Duración estimada: 2 semanas

3. Instalación de sistemas operativos

Instalación de Debian 12, Debian 13, MX Linux, Windows Server 2022 y Windows 10.

Duración estimada: 1 semana y media

4. Instalación y configuración de servicios

Incluye:

- Instalación de Zabbix Server
- Configuración de MariaDB, Apache y PHP
- Instalación de Proxmox
- Configuración del servidor DHCP en Windows Server

Duración estimada: 1 semana

5. Instalación y configuración de agentes

Instalación de Zabbix Agent 2 en todos los clientes y configuración de la comunicación con el servidor.

Duración estimada: 1 semana

6. Integración y monitorización

Alta de hosts, aplicación de plantillas, configuración de métricas, triggers y alertas.

Duración estimada: 2 semana

7. Pruebas, validación y ajustes finales

Comprobación del correcto funcionamiento del sistema, resolución de incidencias y optimización.

Duración estimada: <1 semana

8. Redacción de la memoria y documentación final

Elaboración de la memoria completa, anexos, capturas y conclusiones de toda la instalación realizada, con documentación.

Duración estimada: 2 semanas

- Duración total estimada

Sumando todas las fases:

Duración total: 13 semanas aproximadamente

Este tiempo se ajusta perfectamente al calendario del módulo de Proyecto Integrado, permitiendo incluso cierto margen para imprevistos o ampliaciones.

Enero							Febrero							Marzo						
L	M	X	J	V	S	D	L	M	X	J	V	S	D	L	M	X	J	V	S	D
			1	2	3	4							1							1
5	6	7	8	9	10	11	2	3	4	5	6	7	8	2	3	4	5	6	7	8
12	13	14	15	16	17	18	9	10	11	12	13	14	15	9	10	11	12	13	14	15
19	20	21	22	23	24	25	16	17	18	19	20	21	22	16	17	18	19	20	21	22
26	27	28	29	30	31		23	24	25	26	27	28		23	24	25	26	27	28	29
														30	31					

Abril							Mayo							Junio						
L	M	X	J	V	S	D	L	M	X	J	V	S	D	L	M	X	J	V	S	D
			1	2	3	4					1	2	3	1	2	3	4	5	6	7
6	7	8	9	10	11	12	4	5	6	7	8	9	10	8	9	10	11	12	13	14
13	14	15	16	17	18	19	11	12	13	14	15	16	17	15	16	17	18	19	20	21
20	21	22	23	24	25	26	18	19	20	21	22	23	24	22	23	24	25	26	27	28
27	28	29	30				25	26	27	28	29	30	31	29	30					

12. ANÁLISIS DE REQUISITOS

El análisis de requisitos permite definir con precisión qué necesita el sistema para funcionar correctamente y qué condiciones deben cumplirse para garantizar una implantación exitosa.

En este proyecto, los requisitos se dividen en requisitos funcionales, requisitos no funcionales, requisitos de hardware, requisitos de software y requisitos de red.

Este punto establece las bases técnicas que guiarán la instalación, configuración y validación del sistema de monitorización basado en Zabbix.

12.1. REQUISITOS FUNCIONALES

Los requisitos funcionales describen las funciones que el sistema debe ser capaz de realizar.

- Monitorización de servidores y equipos cliente

El sistema debe monitorizar el servidor Zabbix, el servidor Proxmox y los cuatro equipos cliente.

Debe recoger métricas de CPU, RAM, disco, red y servicios.

- Monitorización de red mediante SNMP

El switch debe ser monitorizado mediante SNMP v2c o v3.

Debe mostrar tráfico por puerto, estado de interfaces y errores.

- Gestión de alertas

El sistema debe generar alertas cuando se superen umbrales definidos.

Las alertas deben clasificarse por severidad.

- Integración de agentes

Todos los equipos deben tener instalado Zabbix Agent 2.

Los agentes deben comunicarse correctamente con el servidor.

- Paneles de control

Deben crearse dashboards para visualizar métricas en tiempo real.

Deben incluir gráficos, tablas y vistas de problemas.

- Descubrimiento automático

El sistema debe detectar interfaces, sistemas de archivos y servicios mediante LLD.

- Registro

El sistema debe registrar eventos, accesos y cambios de configuración.

12.2. REQUISITOS NO FUNCIONALES

Los requisitos no funcionales definen las características de calidad del sistema.

- Rendimiento

El servidor debe ser capaz de procesar todas las métricas sin retrasos.

La base de datos debe responder en menos de 200 ms por consulta.

- Disponibilidad

El sistema debe estar disponible al menos el 95% del tiempo.

El servidor debe arrancar automáticamente tras un reinicio.

- Seguridad

Las comunicaciones deben estar cifradas mediante TLS cuando sea posible.

El acceso al frontend debe requerir autenticación.

Windows Server debe gestionar DHCP de forma segura.

- Escalabilidad

El sistema debe permitir añadir nuevos hosts sin reconfigurar toda la infraestructura.

- Usabilidad

La interfaz debe ser comprensible para profesores y alumnos.

Los dashboards deben ser claros y fáciles de interpretar.

12.3. REQUISITOS HARDWARE

Los requisitos de hardware se basan en la infraestructura definida en el capítulo 13.

- Servidor Zabbix
 - CPU Xeon
 - 32 GB RAM ECC
 - RAID 1 SSD
 - Doble interfaz de red
 - Chasis 1U + regleta enrackable
- Servidor Proxmox
 - CPU Ryzen 7
 - 32 GB RAM
 - SSD NVMe 1 TB
- Equipos cliente
 - Debian 13 → i3 + 8 GB RAM
 - MX Linux → i3 + 8 GB RAM
 - Windows Server 2022 → i5 + 16 GB RAM
 - Windows 10 → i3 + 8 GB RAM
- Switch SNMP
 - 8 puertos Gigabit
 - Gestión SNMP v1/v2c/v3

12.4. REQUISITOS SOFTWARE

- Sistemas operativos
 - Debian 12 (servidor Zabbix)
 - Debian 13 (cliente)
 - MX Linux (cliente)
 - Windows Server 2022 (DHCP + cliente)
 - Windows 10 Pro (cliente)
 - Proxmox VE (hipervisor)
- Software de monitorización
 - Zabbix Server 7.0 LTS
 - Zabbix Agent 2
 - MariaDB 10.11
 - Apache 2.4
 - PHP 8.2
- Servicios adicionales
 - DHCP en Windows Server
 - SNMP en switch
 - SSH en equipos Linux

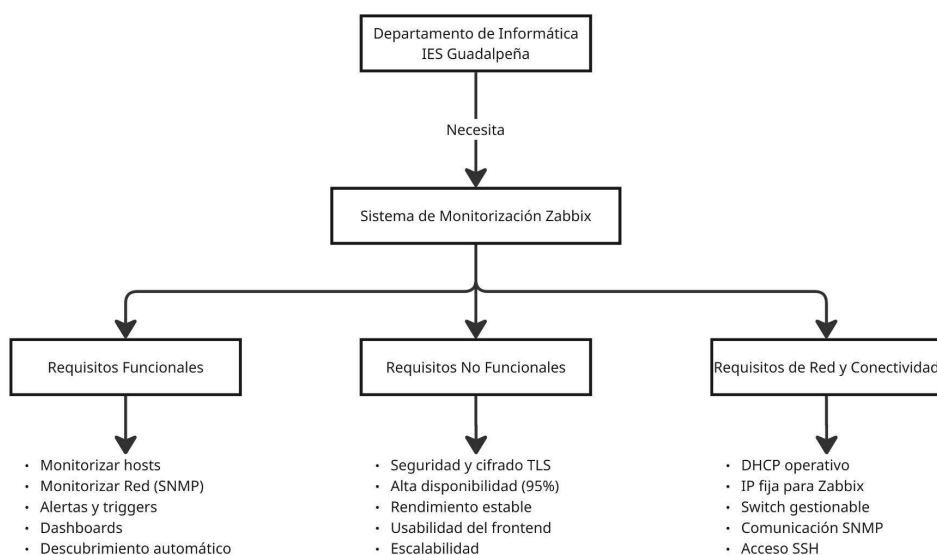
12.5. REQUISITOS DE RED

- Direccionamiento IP
 - El servidor DHCP debe asignar direcciones automáticamente.
 - El servidor Zabbix debe tener IP fija.
 - Proxmox debe tener IP fija.
- Puertos necesarios
 - 10051/TCP → Zabbix Server
 - 10050/TCP → Zabbix Agent
 - 161/UDP → SNMP
 - 80/443 → Frontend web
 - 22/TCP → SSH
 - 67/68 → DHCP
- Topología
 - Todos los equipos deben conectarse al switch gestionable.
 - El servidor Zabbix debe estar en la misma red que los clientes.

12.6. DIAGRAMAS VISUALES

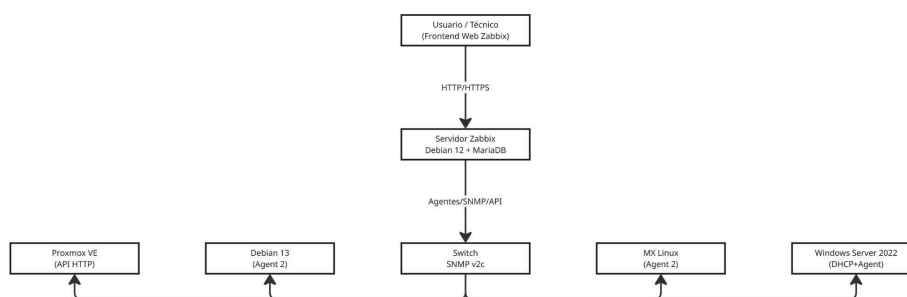
- Diagramas de requisitos

Aquí se resumen los requisitos principales del sistema desde el ojo del cliente:



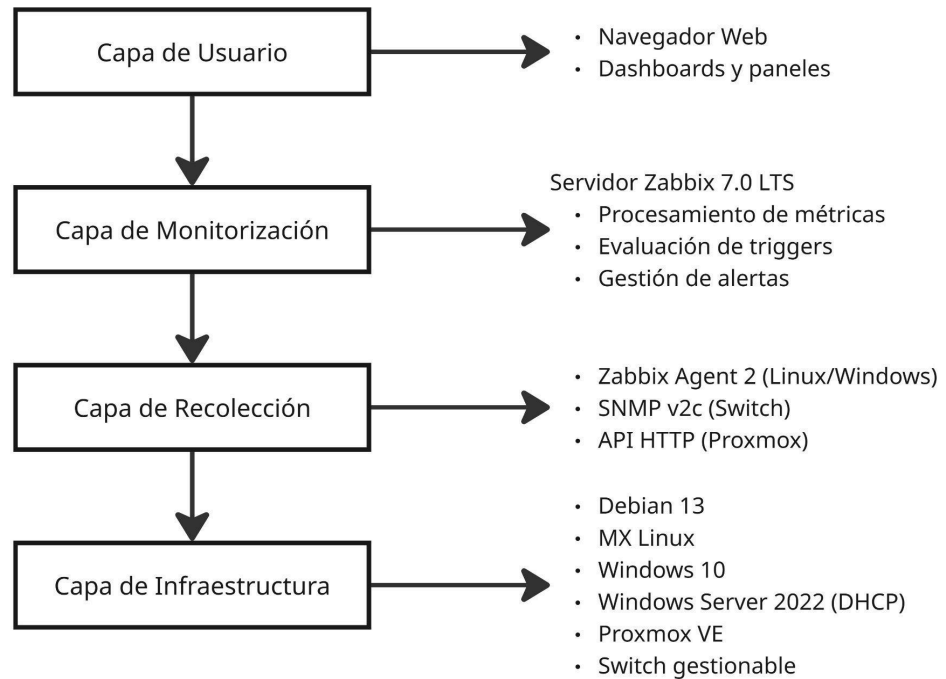
- Diagramas de relación entre componentes

Este muestra cómo interactúan los distintos componentes del prototipo dentro del entorno monitorizado:



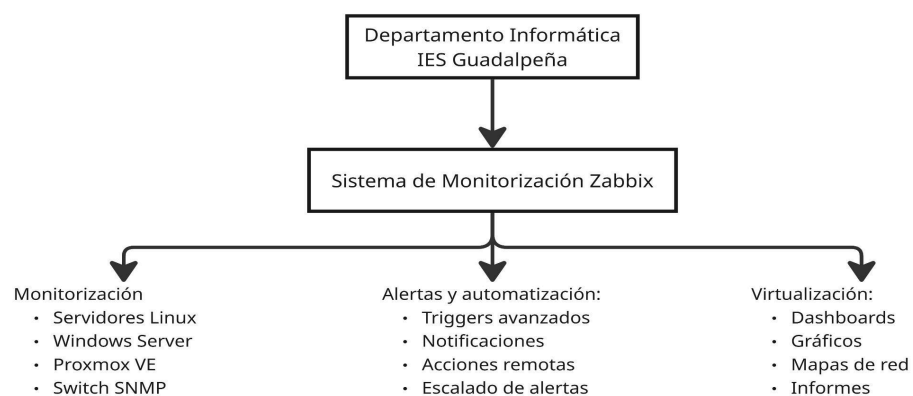
- Diagrama de arquitectura lógica del sistema

Este representa la estructura lógica del sistema de monitorización:



- Diagrama del requisitos del cliente

Este resume lo que el departamento de informática espera del sistema:



12.7. ANÁLISIS DE RIESGOS Y VULNERABILIDADES

- Riesgos técnicos

Los riesgos técnicos están relacionados con fallos en la infraestructura, incompatibilidades o problemas derivados del propio funcionamiento del sistema.

Riesgos identificados:

- Sobrecarga del servidor Zabbix

Un exceso de métricas o un mal dimensionamiento puede provocar lentitud o caídas.

- Fallo en la base de datos MariaDB

La corrupción de tablas o un mal rendimiento afecta directamente a Zabbix.

- Incompatibilidad entre versiones

Actualizaciones de Debian, PHP o Zabbix pueden generar errores.

- Fallo en los agentes Zabbix

Si un agente deja de enviar datos, se pierde visibilidad del host.

- Problemas de latencia o pérdida de paquetes

Afectan a la recepción de métricas en tiempo real.

- Dependencia del servicio DHCP

Si Windows Server 2022 falla, los clientes pueden perder conectividad.

- Riesgo de seguridad

Los riesgos de seguridad afectan a la confidencialidad, integridad y disponibilidad del sistema.

Riesgos identificados:

- Acceso no autorizado al frontend de Zabbix

Un atacante podría visualizar o manipular la infraestructura.

- Comunicaciones sin cifrar

El uso de agentes sin TLS puede permitir ataques de interceptación.

- Exposición del puerto SNMP

SNMP v2c utiliza comunidades en texto plano.

- Contraseñas débiles o reutilizadas

Riesgo de acceso indebido por fuerza bruta.

- Falta de actualizaciones de seguridad

Vulnerabilidades en Debian, Apache o MariaDB pueden ser explotadas.

- Escalada de privilegios

Usuarios con permisos excesivos pueden modificar configuraciones críticas.

- Riesgos operativos

Los riesgos operativos están relacionados con el uso diario del sistema y la interacción de los usuarios.

Riesgos identificados:

- Errores de configuración por parte del administrador

Un trigger mal configurado puede generar alertas falsas o ignorar fallos reales.

- Falta de copias de seguridad

La pérdida de la base de datos implica perder toda la configuración.

- Desconocimiento del personal

Usuarios sin formación pueden modificar parámetros críticos.

- Dependencia de un único administrador

Si solo una persona conoce el sistema, se genera un punto único de fallo.

- Cambios no documentados

Dificultan la resolución de incidencias.

- Riesgos de infraestructura

Estos riesgos afectan al hardware y a la red donde se despliega el sistema.

Riesgos identificados:

- Fallo del switch gestionable

Dejaría sin conectividad a todos los hosts monitorizados.

- Cortes eléctricos

Pueden provocar corrupción de la base de datos o pérdida de datos.

- Fallo del hardware de la máquina anfitriona

Afecta a todas las máquinas virtuales.

- Problemas en VirtualBox

Errores en adaptadores de red o snapshots pueden afectar al entorno.

- Saturación de la red del aula

Puede retrasar la entrega de métricas.

- Medidas a tomar

Para reducir los riesgos identificados, se aplican las siguientes medidas:

Medidas técnicas:

- Ajustar el número de métricas y frecuencia de actualización.
- Optimizar MariaDB (buffers, índices, logs).
- Monitorizar el propio servidor Zabbix.
- Configurar triggers de salud del sistema.

Medidas de seguridad:

- Activar TLS en agentes y servidor.
- Configurar firewall (solo puertos 80/443/10050/10051/161).
- Cambiar comunidad SNMP por una segura.
- Aplicar RBAC y contraseñas robustas.
- Actualizar Debian, Apache, PHP y Zabbix regularmente.

Medidas operativas:

- Realizar copias de seguridad automáticas de la base de datos.
- Documentar todos los cambios.
- Formar al personal del departamento.
- Crear un plan de recuperación ante fallos.

Medidas de infraestructura:

- Usar SAI para evitar cortes eléctricos.
- Supervisar el estado del switch mediante SNMP.
- Mantener snapshots estables de las máquinas virtuales.

13. DISEÑO DEL PROTOTIPO

Este apartado describe cómo se construye y organiza el prototipo de la solución de monitorización. Se detalla la arquitectura, el papel de cada equipo, la topología de red y el funcionamiento interno del sistema. El objetivo es mostrar de forma clara cómo se ha diseñado y ensamblado el prototipo que servirá como base para las pruebas y validación del proyecto.

13.1. VISIÓN GENERAL DEL PROTOTIPO

El prototipo se compone de una infraestructura real formada por:

- Un servidor Zabbix con Debian 12
- Un servidor Proxmox para virtualización
- Un switch gestionable
- Cuatro equipos cliente con distintos sistemas operativos
- Servicios de red esenciales (DHCP, SNMP, SSH, HTTP)

Todos los equipos están conectados físicamente al switch, formando una red local completamente funcional.

13.2. ROL DE CADA COMPONENTE DEL PROTOTIPO

13.2.1. SERVIDOR ZABBIX (DEBIAN 12)

Es el núcleo del prototipo.

Gestiona:

- Recogida de métricas
- Procesamiento de datos
- Generación de alertas
- Dashboards y visualización
- Base de datos MariaDB

13.2.2. **SERVIDOR PROXMOX**

Permite:

- Crear máquinas virtuales de prueba
- Simular entornos adicionales
- Ser monitorizado mediante agente y API

13.2.3. **SWITCH GESTIONABLE**

Actúa como centro de comunicaciones del prototipo:

- Conecta todos los equipos
- Proporciona métricas SNMP
- Permite ver tráfico por puerto
- Detecta errores de red

13.2.4. **CLIENTE DEIBAN 13**

Equipo Linux moderno para pruebas de monitorización

13.2.5. **CLIENTE MX LINUX**

Equipo ligero que permite comprobar la monitorización en sistema de bajo consumo

13.2.6. **CLIENTE WINDOWS SERVER 2022 (DHCP)**

Equipo crítico del prototipo:

- Proporciona direcciones IP a toda la red
- Se monitoriza mediante Zabbix Agent
- Permite validar métricas de servicios Windows

13.2.7. **CLIENTE WINDOWS 10**

Simula un puesto de trabajo estándar dentro del prototipo

13.3. TOPOLOGÍA DEL PROTOTIPO

La red del prototipo se organiza de forma sencilla:

- Todos los equipos conectados al switch
- Windows Server 2022 como servidor DHCP
- Zabbix y Proxmox con IP fija
- Comunicación mediante:
 - 10051/TCP → Zabbix Server
 - 10050/TCP → Zabbix Agent
 - 161/UDP → SNMP
 - 22/TCP → SSH
 - 80/443 → Frontend web

13.4. FLUJO DE FUNCIONAMIENTO

- ★ Los agentes envían métricas al servidor Zabbix
- ★ El switch aporta datos SNMP
- ★ Zabbix procesa y almacena la información
- ★ Los triggers detectan problemas
- ★ Se generan alertas
- ★ Los dashboards muestran el estado del prototipo

13.5. DISEÑO LÓGICO

El prototipo se basa en:

- Un servidor central
- Clientes distribuidos
- Monitorización híbrida (agente + SNMP)
- Servicios reales (DHCP, virtualización, Linux, Windows)

13.6. JUSTIFICACIÓN DEL PROTOTIPO

Este prototipo:

- Es simple de desplegar
- Es escalable
- Simula un entorno empresarial real
- Permite prácticas educativas
- Facilita la monitorización de distintos sistemas operativos

- Representa una infraestructura completa y funcional

14. DISEÑO DEL PROTOTIPO

El diseño del prototipo constituye la base técnica sobre la que se construirá el sistema de monitorización. En este punto se define la arquitectura de red, la topología del entorno, los mecanismos de seguridad, los componentes implicados y la forma en la que todos ellos se integran dentro del ecosistema de Zabbix.

14.1. DISEÑO DE LA ARQUITECTURA DE RED

La arquitectura del prototipo se basa en un modelo centralizado, donde el Servidor Zabbix actúa como núcleo del sistema. Todos los equipos monitorizados —servidores, máquinas virtuales, clientes Windows/Linux y dispositivos de red— se comunican con él mediante agentes, API o SNMP.

La arquitectura se divide en cuatro capas:

Capa 1: Infraestructura física y virtual

- Máquina anfitriona con VirtualBox
- Switch gestionable físico
- Red LAN del aula
- Adaptadores NAT y solo-anfitrión

Capa 2: Servidores principales

- Servidor Zabbix (Debian 12)
 - Base de datos MariaDB
 - Apache + PHP
 - Procesamiento de métricas
- Servidor Proxmox VE
 - Entorno de virtualización
 - Integración mediante API HTTP

Capa 3: Equipos monitorizados

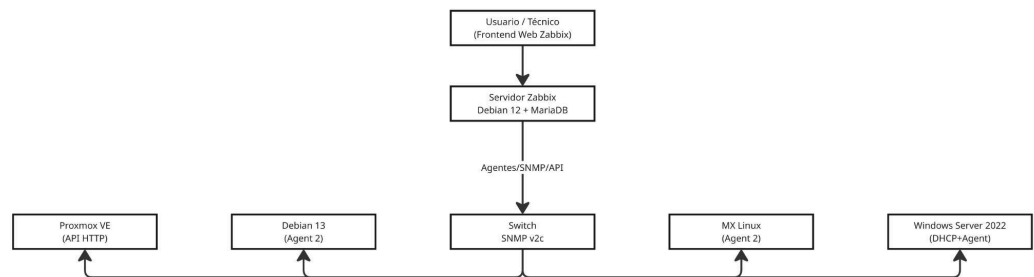
- Debian 13 (Agent 2)
- MX Linux (Agent 2)
- Windows Server 2022 (DHCP + Agent 2)
- Windows 10 (Agent 2)

Capa 4: Dispositivos de red

- Switch gestionable SNMP v2c

14.2. DISEÑO DE LA TOPOLOGÍA DE RED

La topología del prototipo se ha diseñado para simular un entorno real de un departamento de informática, donde todos los equipos se conectan a un switch centralizado.



14.3. DISEÑO DE LAS SOLUCIONES DE SEGURIDAD

La seguridad es un elemento crítico en cualquier sistema de monitorización. El prototipo incorpora medidas de protección en todas las capas:

Seguridad en comunicaciones

- Activación de TLS en Zabbix Server y Zabbix Agent 2
- Restricción de puertos mediante firewall
- SNMP configurado con comunidad segura

Seguridad del frontend

- Autenticación basada en roles (RBAC)
- Usuarios separados por permisos
- Políticas de contraseñas robustas

Seguridad del servidor

- Hardening de Debian 12
- Deshabilitación de servicios innecesarios
- Actualizaciones automáticas de seguridad
- Copias de seguridad programadas de MariaDB

Seguridad de red

- Segmentación lógica del tráfico
- Supervisión del switch mediante SNMP
- Control de acceso físico al equipo

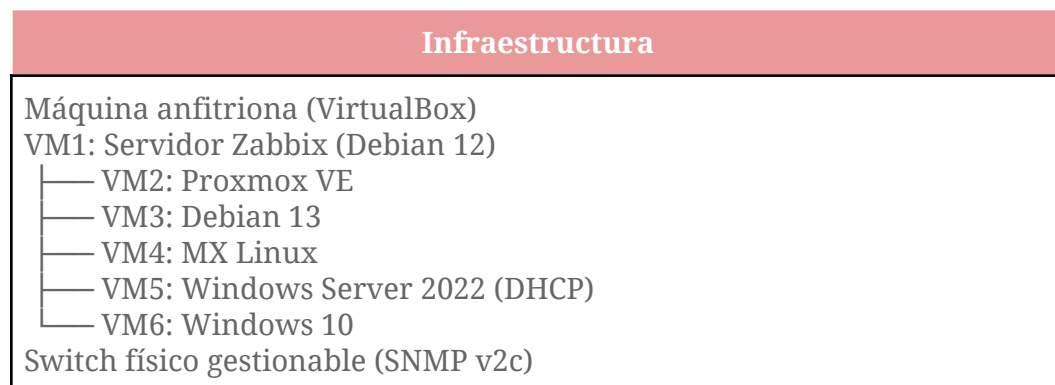
14.4. DIAGRAMA DE DESPLIEGUE

El despliegue del prototipo se realiza sobre VirtualBox, con las siguientes máquinas virtuales:

Máquina	Sistema	Función	Recursos
VM1	Debian 12	Servidor Zabbix	3 vCPU, 10Gb RAM, 200Gb SSD
VM2	Proxmox VE	Virtualización	4 vCPU, 8 Gb RAM
VM3	Debian 13	Cliente Linux	2 vCPU, 4Gb RAM
VM4	MX Linux	Cliente Linux	2 vCPU, 4Gb RAM
VM5	Windows Server	DHCP+Agente	4 vCPU, 8Gb RAM
VM6	Windows 10	Cliente Windows	2 vCPU, 4Gb RAM

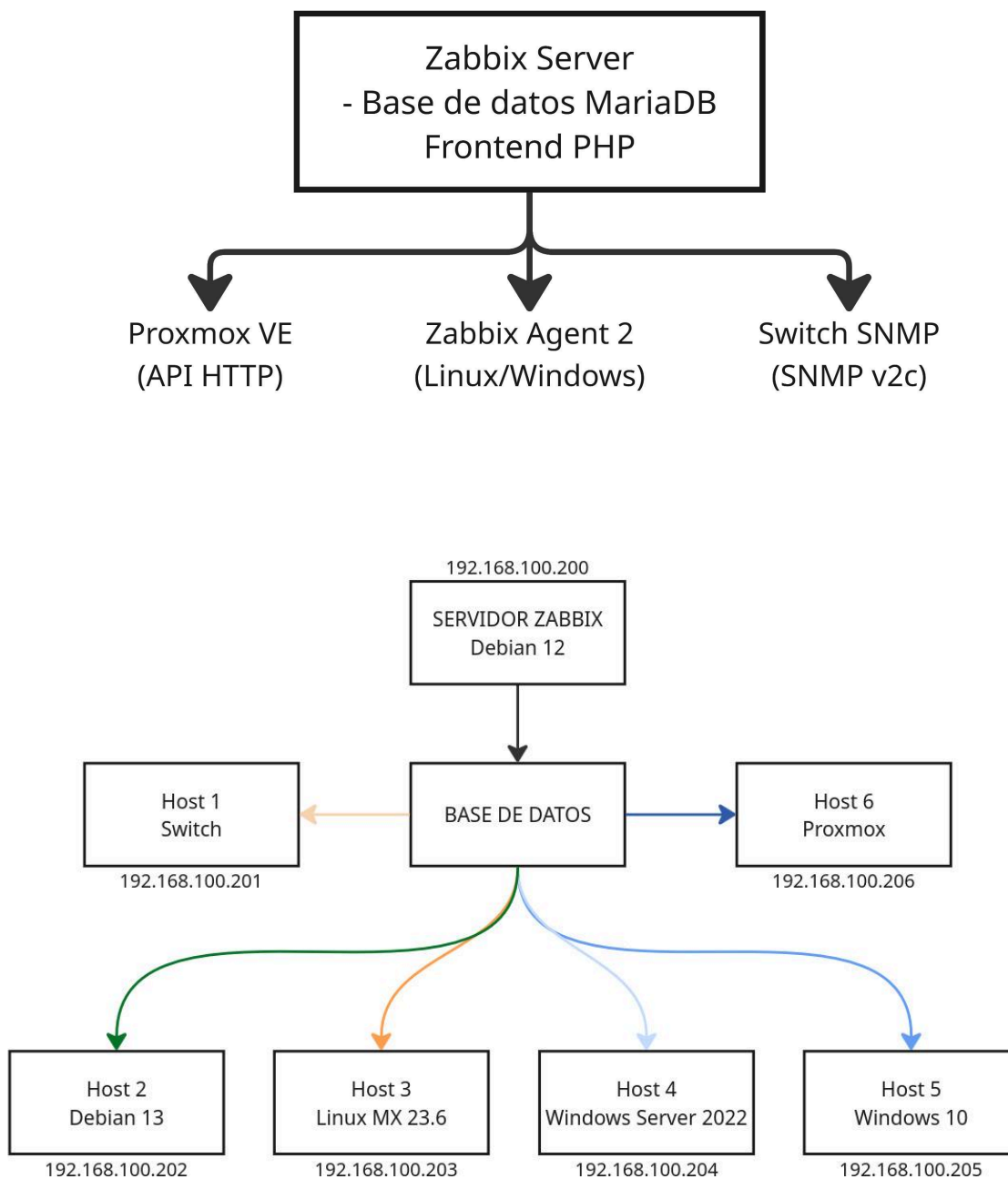
14.5. DIAGRAMA DE INFRAESTRUCTURA

Este diagrama representa la infraestructura completa del prototipo:



14.6. DIAGRAMA DE INTEGRACIÓN

Este muestra cómo se integran los distintos sistemas con zabbix:



15. IMPLEMENTACIÓN

En esta fase se muestra la manera en la que hemos montado, construido y puesto en marcha el sistema o en palabras más sencillas es la parte práctica del proyecto en la que hemos instalado, configurado y conectado los equipos y el servidor Zabbix con sus respectivas partes de seguridad y copias backups.

15.1. INSTALACIÓN Y CONFIGURACIÓN DE LOS DISPOSITIVOS DE RED

En este apartado nos centramos en instalar todo el software necesario tanto en el servidor como en los equipos que vamos a monitorizar con este, configurar sus respectivas direcciones IP, añadir los equipos al servidor...

15.1.1. SERVIDOR ZABBIX

En primer lugar, instalamos en una máquina virtual “Debian 12” el agente Zabbix (si esta cumple con los requisitos en cuanto a recursos hardware que requiere Zabbix para que funcione correctamente), esto lo hacemos en una máquina con S.O derivado de Linux ya que posee una terminal de comandos fácil de manejar y en la que se usan comandos conocidos y no tan complejos como en Windows. El paquete de instalación del agente Zabbix se encuentra disponible en su página web oficial y para esta máquina usaremos este paquete (Agente Zabbix) ya que estamos utilizando una máquina con sistema operativo.

Finalmente lo que queda en cuanto al servidor sería establecer una dirección IP fija o DHCP como es en este caso para esta máquina, instalar y configurar el motor de la base de datos, el cual en este caso utilizaremos el motor “mysql” y por último configurar el frontend (interfaz web de administración de Zabbix).

Sabiendo que además haciendo uso de la terminal de la máquina debian 12 podemos instalar más opciones como el comando ping o traceroute para que se pueda utilizar directamente desde el frontend, por ello es importante lo que se comentaba anteriormente de la importancia de disponer de una terminal de comandos sencilla para administrar el servidor.

15.1.2. AGENTE ZABBIX

El agente Zabbix al igual que en el servidor va a ser el paquete que vamos a utilizar para la monitorización de los distintos equipos que van a disponer de Sistemas Operativos ya sean Windows o linux

Para ello lo que haremos será instalar el paquete de instalación del agente Zabbix que se encuentra disponible en su página web oficial y para esta máquina usaremos este paquete (Agente Zabbix) ya que estamos utilizando una máquina con sistema operativo.

Después, lo que tendremos que hacer será poner el mismo nombre que tendrá el servidor, que tanto el equipo como el servidor estén en la misma red y desde el servidor añadir el equipo para poder monitorizarlo

Finalmente en nuestro servidor Zabbix tendremos que entrar en el apartado donde se encuentran los equipos, darle a añadir un equipo, rellenar todas las características y finalmente añadir el equipo como agente Zabbix y esperar a que este se conecte (el equipo deberá aparecer en verde)

15.1.3. PROTOCOLO SNMP

El protocolo **SNMP (Simple Network Management Protocol o Protocolo simple de manejo de redes)**, es aquel que se encarga de identificar a los equipos que lo contengan para que se puedan monitorizar y gestionar a través de la red.

Este protocolo es muy utilizado sobre todo en los dispositivos de redes como pueden ser por ejemplo los **routers, switches, antenas, repetidores, servidores**, entre otros.

Para poder monitorizar estos dispositivos con Zabbix hay que seguir los siguientes pasos:

- **Entrar en la interfaz web** (si el dispositivo dispone de ella) del dispositivo, previamente asegurándonos de que la máquina o el equipo con el que vayamos a hacer esto este en la misma red o al menos tenga una dirección IP del rango de

ese equipo si este no está configurado o posee otra dirección ajena a la red (posteriormente para monitorizar este dispositivo se debe encontrar en la misma red que el servidor)

- **Activar el protocolo SNMP.** Por ejemplo en el caso de este proyecto el SNMP de los dispositivos será “ProyectoZabbix”
- Finalmente en **nuestro servidor Zabbix** tendremos que entrar en el apartado donde se encuentran los equipos, darle a **añadir un equipo**, rellenar todas las características y finalmente en lugar de añadir el equipo como agente Zabbix donde se encontraba el apartado anterior, activar la casilla que pone SNMP con su respectiva versión (en este caso la v2), por último pondremos el nombre SNMP que se mencionó anteriormente y esperar a que este se conecte (el equipo deberá aparecer en verde)

15.2. CONFIGURACIÓN DE LAS SOLUCIONES DE SEGURIDAD

15.2.1. CONTROL DE ACCESO

En el frontend de Zabbix (a parte de los usuarios dedicados a la administración de este servicio dentro de la máquina en la que se encuentra instalado), existen una serie de roles o usuarios dedicados a garantizar el manejo más sencillo y seguro de este programa de monitorización, entre ellos encontramos los siguientes roles de usuario:

- **Usuario:** Solo tiene permisos de lectura pudiendo ver únicamente los paneles de los equipos monitorizados con su respectivo estado, alertas problemas...
- **Administrador:** Tiene permisos de lectura y escritura dentro del servidor pudiendo manejar todo tipo de plantillas y opciones pero no puede gestionar a los usuarios ni tocar los permisos.
- **Super Administrador:** Es el que tiene el control total dentro del servidor pudiendo hacer todo lo anterior como gestionar los usuarios y sus permisos pudiendo dar o denegar estos y borrar, crear, modificar usuarios dentro del servidor interno, entre otras cosas.

15.2.2. COMUNICACIÓN CIFRADA

El frontend de Zabbix (al gestionarse vía web desde cualquier navegador), utiliza el protocolo **HTTPS (Hypertext Transfer Protocol “Secure”)** el cual es aquel que hace que la página web de manejo del servidor sea más seguras y además las transferencias de datos entre el servidor y las máquina agentes y SNMP que se gestionan con este van cifrados para que nadie pueda interceptar el tráfico de estos.

15.2.3. PROTECCIÓN DE LA BASE DE DATOS

En el servidor Debian 12 que es el que estamos utilizando para manejar el servidor Zabbix podemos **administrar las bases de datos de forma segura** por ejemplo poniéndole una **contraseña segura** a las bases de datos y además con la opción de solo poder **gestionar las bases de datos de forma interna** lo que quiere decir que solo los usuarios superadministradores o root podrán tener acceso a ellas.

15.3. CONFIGURACIÓN DE LAS SOLUCIONES DE VIRTUALIZACIÓN

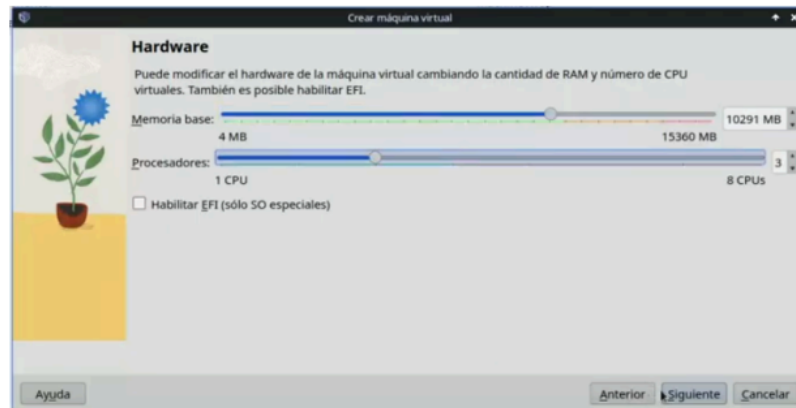
Para este proyecto, se ha optado por utilizar un entorno de virtualización, en este caso **VirtualBox** en el que se alojan tanto las máquinas clientes que están siendo monitorizadas con el servidor Zabbix y la máquina virtual Debian 12 del propio servidor.



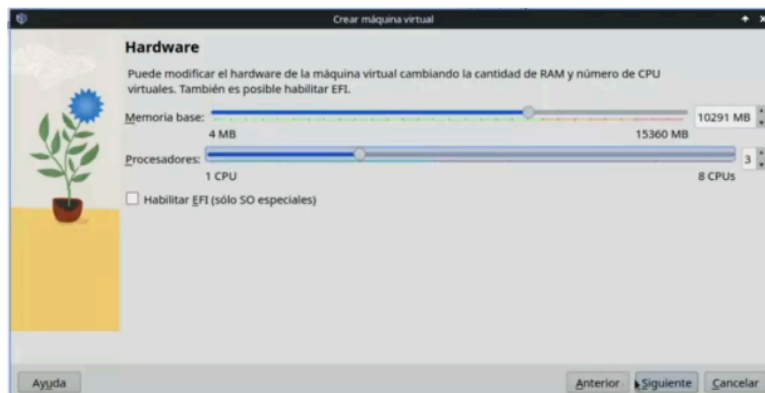
A cada máquina se le han reservado sus respectivos recursos pero haciendo hincapié en el servidor sobre todo ya que este es el que **más carga de trabajo** va a tener en todo momento debido a que llegamos a esta conclusión al hacer el análisis de métricas para todos los equipos que se iban a monitorizar y darnos cuenta de que iban a ser muchos ya que este proyecto está destinado al **departamento de informática del IES Guadalpeña**.

Así que sobre todo en este nos hemos parado más para gestionar correctamente sus recursos de manera que este **no tenga sobrecarga** en ningún momento debido a la falta de recursos los cuales están administrados de la siguiente manera (a continuación se muestran solo los requisitos principales ya sean de memoria, disco duro y adaptadores red):

- Recomendable **16GB de RAM** para que la máquina no se quede bloqueada por falta de memoria



- **200GB de almacenamiento de disco duro** reservado para el servidor ya que requiere muchas operaciones de lectura/escritura que pueden llenar el disco muy rápido



- 2 adaptadores disponibles, en este caso uno **adaptador puente** para que se conecte a la red del aula y otro **Red interna (ProyectoZabbix)** para integrar un servidor **DHCP** Windows que le reparta direcciones al servidor Zabbix (lo recomendable es que el propio servidor tenga su propia red pero en este proyecto se suma el segundo servidor Windows para integrar el servicio DHCP)

The image shows two screenshots of the Zabbix web interface for configuring network adapters. Both screenshots are titled 'Red' and show tabs for 'Adaptador 1', 'Adaptador 2', 'Adaptador 3', and 'Adaptador 4'.

Top Screenshot (Adaptador 1):

- ☒ Habilitar adaptador de red
- Conectado a: Adaptador puente
- Nombre: Realtek PCIe GbE Family Controller
- Tipo de adaptador: Intel PRO/1000 MT Desktop (82540EM)
- Modo promiscuo: Denegar
- Dirección MAC: 080027AF3DFF

Bottom Screenshot (Adaptador 2):

- ☒ Habilitar adaptador de red
- Conectado a: Red interna
- Nombre: ProyectoZabbix
- Tipo de adaptador: Intel PRO/1000 MT Desktop (82540EM)
- Modo promiscuo: Denegar
- Dirección MAC: 0800279B9A54

15.4. CONFIGURACIÓN DE LAS SOLUCIONES DE ALMACENAMIENTO EN RED

Para las soluciones de almacenamiento en red, como estamos utilizando una máquina virtual dentro del entorno de VBox, podemos utilizar el servicio SMB o Samba para transferir carpetas o almacenamiento en red, para ello podemos seguir el siguiente proceso:

1. Debemos descargar el paquete cifs-utils y samba en la máquina Debian 12 para que el sistema reconozca el protocolo SMB

```

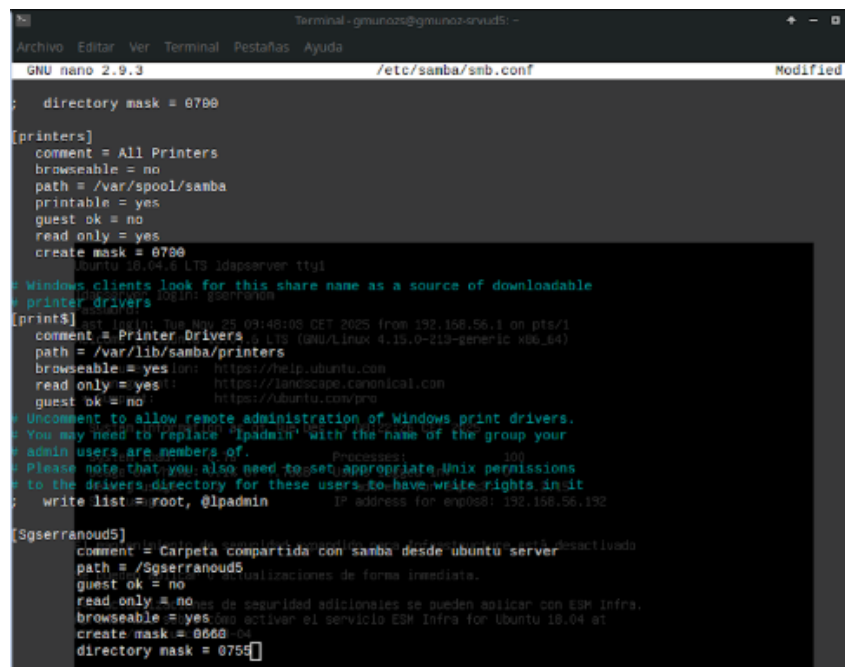
eugenio@eugenio-ervud8:~$ sudo apt install samba
Leyendo lista de paquetes... Hecho
Se instalarán los siguientes paquetes adicionales:
attr libverbs-providers libavahi-client3 libavahi-common-data libavahi-common3 libcephfs2 libcups2
libgpgme11 liblibverbs1 libjamson4 libldb1 libldb-route-3-200 libnss3 libpython-stallib
libpython2.7 libpython2.7-minimal libpython2.7-stallib librados2 libralloc2 libtdbi libtevent0
libtdclient0 python python-crypto python-dnspython python-ldb python-minimal python-samba
python-talloc python-tdb python2.7 python2.7-minimal samba-common samba-common-bin
samba-dsdb-modules samba-libc samba-vfs-modules tdb-tools
Paquetes sugeridos:
cups-common python-doc python-tk python-crypto-doc python-gpgme python2.7-doc binfmt-support bind9
bind9utils ctdb ldb-tools ntp | chrony samba-dsdb-modules winbind heimdal-clients

```


- Después debemos crear la carpeta compartida y darle sus respectivos permisos en la máquina Debian 12

```
gmunoz@gmunoz-srvud5:~$ sudo mkdir /Sgserranoud5
gmunoz@gmunoz-srvud5:~$ sudo chmod -R 777 /Sgserranoud5/
gmunoz@gmunoz-srvud5:~$ sudo chown :sambashare /Sgserranoud5/
gmunoz@gmunoz-srvud5:~$
```

- Después debemos añadirla editando el archivo `/etc/fstab` sacando una copia de seguridad previamente para poder recuperar el contenido en caso de que ocurra un fallo crítico ya que esta carpeta si tiene algo mal puede corromper el sistema



```
Terminal: gmunoz@gmunoz-srvud5: ~
GNU nano 2.9.3 /etc/samba/smb.conf Modified

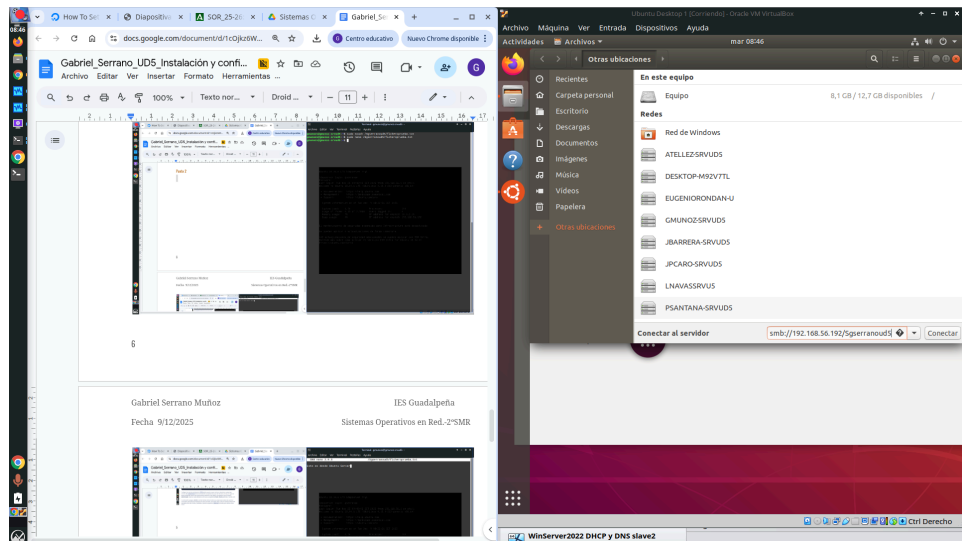
; directory mask = 0700

[printers]
comment = All Printers
browseable = no
path = /var/spool/samba
printable = yes
guest ok = no
read only = yes
create mask = 0700

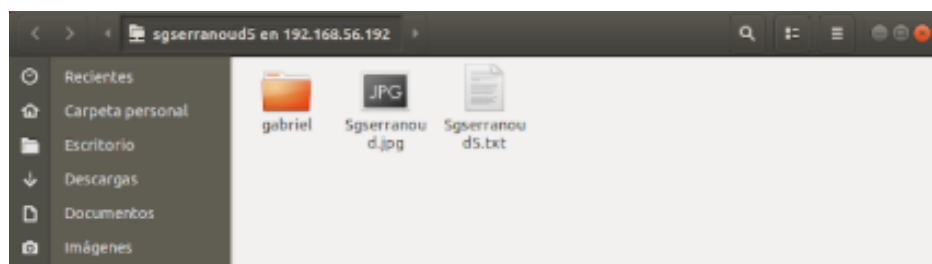
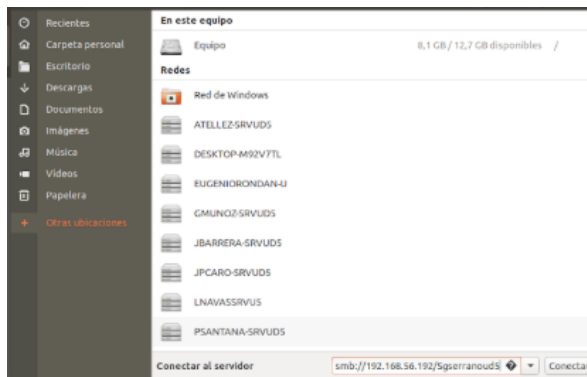
; Windows clients look for this share name as a source of downloadable
; printer drivers
[printers]
comment = Printer Drivers
path = /var/lib/samba/printers
browseable = yes
read only = yes
guest ok = no
write list = root, @lpadmin

[Sgserranoud5]
comment = Carpeta compartida con samba desde ubuntu server
path = /Sgserranoud5
guest ok = no
read only = no
browseable = yes
create mask = 0660
directory mask = 0755
```

4. Por último, debemos comprobar desde el explorador de archivos de una máquina ubuntu por ejemplo que esta carpeta se pasa correctamente. Para ello, debemos entrar en el apartado de otras ubicaciones y escribir en la línea de texto lo siguiente: smb//Ip del servidor/carpeta compartida



5. Finalmente, podemos crear un archivo con contenido e ir hacia la máquina cliente y si este aparece es que ha salido bien



15.5. CONFIGURACIÓN DE LAS COPIAS DE SEGURIDAD

En este apartado veremos cómo vamos a respaldar los archivos importantes de nuestro servidor con una **copia de seguridad backup** para tener un **archivo de respaldo** por si acaso el original se pierde, se elimina accidentalmente o se corrompe.

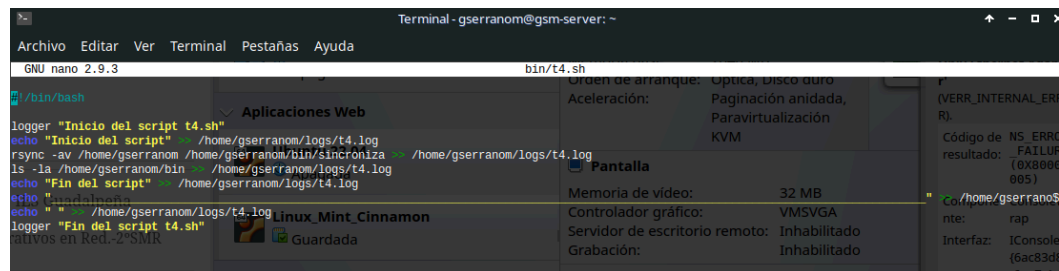
En primer lugar, conoceremos el servicio con el cual sirve realizar tareas programadas como copias de seguridad de archivos o directorios y esto lo hacemos con la herramienta **crontab**.

Utilizaremos esta herramienta como mencionamos anteriormente para **respaldar archivos importantes** como por ejemplo el **mysqldump.sql** que es el archivo que contiene toda la información de la base de datos del servidor junto a los directorios del servidor apache (server web) y el directorio **/etc/zabbix** el cual contiene los archivos de configuración del servidor más importantes del server Zabbix como el **zabbix_server.conf**.

Debemos conocer el funcionamiento de la herramienta crontab ya que este se divide en una serie de campos los cuales son: **Minutos-Horas-Día del mes-Mes-Día de la semana** y por último **la ruta del script** que se va a ejecutar en ese momento



Para realizar copias de seguridad backups con la herramienta crontab tendremos la necesidad de **crear scripts ejecutables** que contengan **funciones** por ejemplo la del comando **rsync** que nos permite realizar **copias de seguridad de directorios y archivos**. También debemos tener en cuenta la opción de **redirigir** todas las opciones a un archivo de log en el que se van a ir guardando todos los procesos que ocurren al ejecutarse el script



```

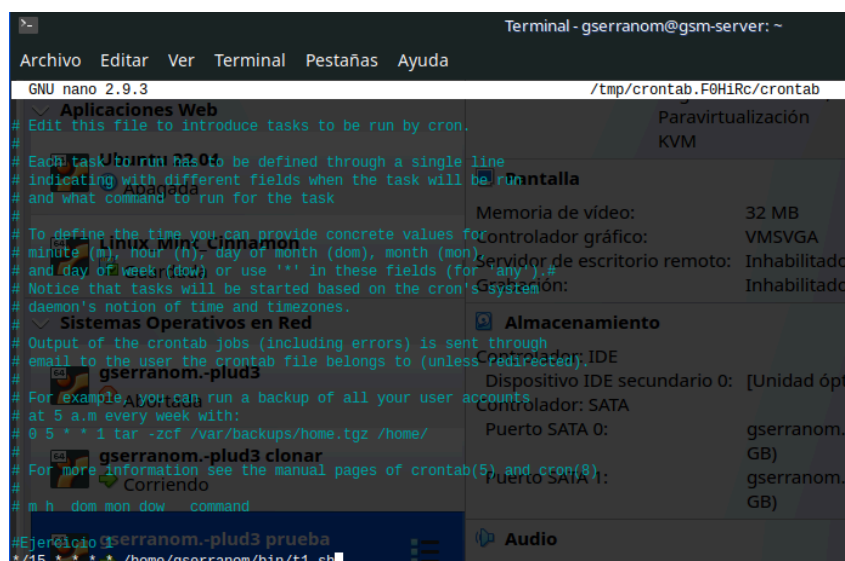
GNU nano 2.9.3 bin/t4.sh

/bin/bash
logger "Inicio del script t4.sh"
rsync -av /home/gserranom /home/gserranom/bin/sincroniza /home/gserranom/logs/t4.log
ls -la /home/gserranom/bin /home/gserranom/logs/t4.log
echo "Fin del script"
logger "Fin del script t4.sh"

```

Al crear el script es cuando entra en juego la herramienta crontab. Para ello tendremos que **programar la tarea** a la determinada fecha y hora que queremos que se ejecute este script con las opciones de backup para los archivos y directorios importantes de Zabbix.

Por ejemplo en la siguiente imagen se puede ver como se programa dicha tarea cada 15 minutos, como el primer apartado indica los minutos, estos se muestran con un **"*/15"** ya que **no es una hora completa**



```

GNU nano 2.9.3 /tmp/crontab.F0H1Rc/crontab

# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon)
# and day of week (dow) or use '*' in these fields (for "any").
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
#
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow command
*/15 * * * * /home/gserranom/bin/t4.sh

```

Una vez se ejecuta el script podemos comprobar que haciendo un **tail -f** al directorio **/var/log/syslog** se puede observar que las líneas que se escribían en el script con el comando **logger** que indican el **inicio y el fin del script** están apareciendo en tiempo real en este directorio a esa determinada fecha y hora en la que se supone que se debería de cumplir dicha función

```
gserranom@gsm-server:~/bin$ tail -f /var/log/syslog
Nov 4 10:36:02 gsm-server CRON[4145]: (CRON) info (No MTA installed, discarding output)
Nov 4 10:37:01 gsm-server CRON[4159]: (gserranom) CMD (/home/gserranom/bin/t1.sh)
Nov 4 10:37:01 gsm-server CRON[4160]: (gserranom) CMD (/home/gserranom/bin/t4.sh)
Nov 4 10:37:01 gsm-server CRON[4158]: (CRON) info (No MTA installed, discarding output)
Nov 4 10:37:01 gsm-server gserranom: Inicio del script t4.sh
Nov 4 10:37:01 gsm-server gserranom: Fin del script t4.sh
Nov 4 10:37:01 gsm-server CRON[4157]: (CRON) info (No MTA installed, discarding output)
Nov 4 10:37:20 gsm-server crontab[4169]: (gserranom) BEGIN EDIT (gserranom)
Nov 4 10:37:33 gsm-server crontab[4169]: (gserranom) REPLACE (gserranom)
Nov 4 10:37:33 gsm-server crontab[4169]: (gserranom) END EDIT (gserranom)
Nov 4 10:38:01 gsm-server cron[1104]: (gserranom) RELOAD (crontabs/gserranom)
Nov 4 10:38:01 gsm-server CRON[4185]: (gserranom) CMD (/home/gserranom/bin/t4.sh)
Nov 4 10:38:01 gsm-server CRON[4186]: (gserranom) CMD (/home/gserranom/bin/t1-1.sh)
Nov 4 10:38:01 gsm-server CRON[4184]: (CRON) info (No MTA installed, discarding output)
Nov 4 10:38:01 gsm-server gserranom: Inicio del script t4.sh
Nov 4 10:38:01 gsm-server gserranom: Fin del script t4.sh
Nov 4 10:38:01 gsm-server CRON[4183]: (CRON) info (No MTA installed, discarding output)
```

Finalmente si comprobamos en el **fichero de log** en el que se supone que se deben redirigir todos los procesos que se ejecutan con ese script deberán de aparecer todos al comprobar la salida de este utilizando el comando **“cat”**. Usando este comando podemos ver por ejemplo el **inicio, contenido del directorio con las copias de seguridad** que se han hecho con el script y el **fin del script**.

```
gserranom/sincroniza/home/gserranom/bin/home/gserranom/.local/
gserranom/sincroniza/home/gserranom/bin/home/gserranom/.local/share/
gserranom/sincroniza/home/gserranom/bin/home/gserranom/.local/share/nano/
gserranom/sincroniza/home/gserranom/bin/home/gserranom/bin/
gserranom/sincroniza/home/gserranom/bin/home/gserranom/bin/home.tar.gz
gserranom/sincroniza/home/gserranom/bin/home/gserranom/bin/t2.log
gserranom/sincroniza/home/gserranom/bin/home/gserranom/bin/t2.sh
gserranom/sincroniza/home/gserranom/bin/home/gserranom/bin/t3.sh
gserranom/sincroniza/home/gserranom/bin/home/gserranom/logs/
gserranom/sincroniza/home/gserranom/bin/home/gserranom/logs/t2.log
gserranom/sincroniza/home/gserranom/bin/home/gserranom/logs/t3.log
gserranom/sincroniza/home/gserranom/logs/
gserranom/sincroniza/home/gserranom/logs/t2.log
gserranom/sincroniza/home/gserranom/logs/t3.log
gserranom/sincroniza/home/gsm-plud3/
gserranom/sincroniza/home/gsm-plud3/.bash_logout
gserranom/sincroniza/home/gsm-plud3/.bashrc
gserranom/sincroniza/home/gsm-plud3/.profile
gserranom/sincroniza/home/gsm-plud3/.sudo_as_admin_successful
gserranom/sincroniza/home/gsm-plud3/.cache/
gserranom/sincroniza/home/gsm-plud3/.gnupg/
gserranom/sincroniza/home/lost+found/

sent 1,663,893,431 bytes received 11,326 bytes 195,753,500.82 bytes/sec
total size is 1,663,449,937 speedup is 1.00

drwxr-xr-x 4 gserranom gserranom 4096 nov 4 10:20
drwxr-xr-x 9 gserranom gserranom 4096 nov 4 10:10
drwxr-xr-x 3 gserranom gserranom 4096 nov 4 09:44
-rw-rw-r-- 1 gserranom gserranom 65837414 nov 4 09:54 home.tar.gz
drwxr-xr-x 3 gserranom gserranom 4096 nov 4 10:20 sincroniza
-rw-rw-r-- 1 gserranom gserranom 4096 nov 4 09:26 t2.log
-rwxr-xr-x 1 gserranom gserranom 429 nov 4 08:59 t2.sh
-rwxr-xr-x 1 gserranom gserranom 486 nov 4 09:45 t3.sh
-rwxr-xr-x 1 gserranom gserranom 541 nov 4 10:19 t4.sh
Fin del script
```

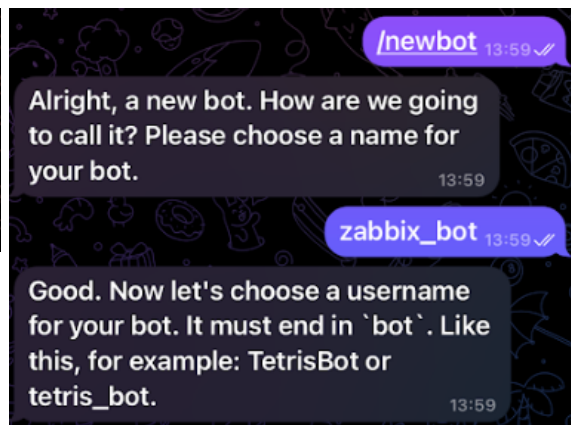
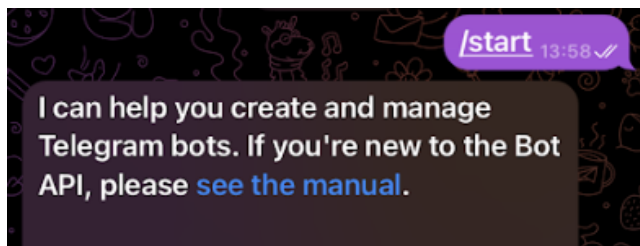
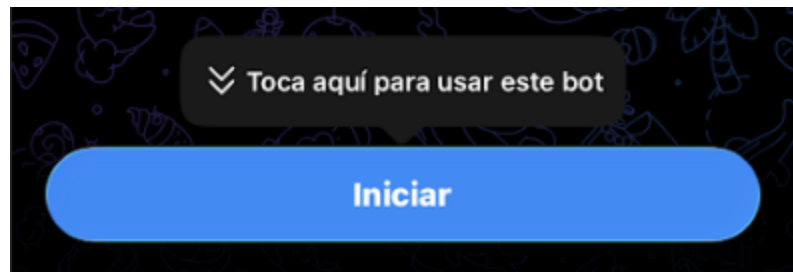
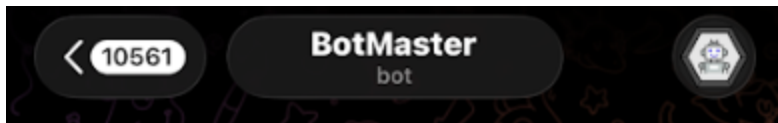
Orden de arranque:	Óptica, Disco duro
Aceleración:	Paginación anidada, Paravirtualización KVM
Pantalla	
Memoria de vídeo:	32 MB
Controlador gráfico:	VMSVGA
Servidor de escritorio remoto:	Inhabilitado
Grabación:	Inhabilitado
Almacenamiento	
Controlador:	IDE
Dispositivo IDE secundario 0:	[Unidad óptica]
Controlador:	SATA
Puerto SATA 0:	gserranom.-pl (GB)
Puerto SATA 1:	gserranom.-pl (GB)
Audio	
Inhabilitado	
Red	
Adaptador 1:	Intel PRO/1000 MT Desktop (NA
Adaptador 2:	Intel PRO/1000 MT Desktop (Ad «vboxnet0»)

15.6. CONFIGURACIÓN DE LOS UMBRALES DE AVISO

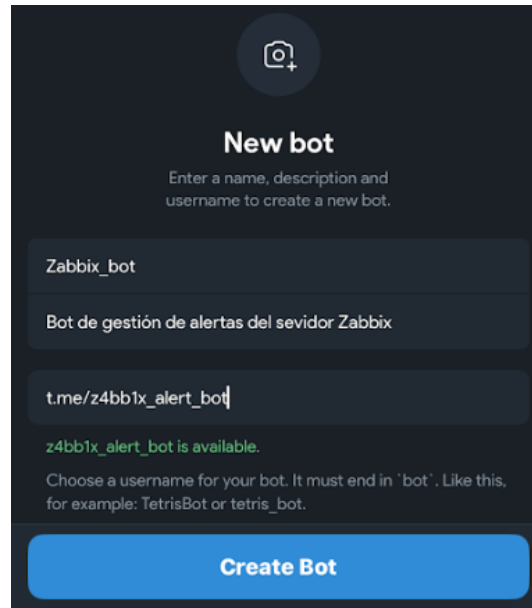
15.6.1. GRUPO DE TELEGRAM CON BOTFATHER

Para configurar un grupo de telegram en el que podamos recibir notificaciones de aviso para cuando los dispositivos que monitorizamos se apaguen, se enciendan, tengan un gran consumo de recursos... tenemos que usar a “BotFather”

Este bot se encarga de lo que venimos mencionando, para ello tenemos que buscar BotFather en el buscador de telegram, iniciar la conversación y crear el bot

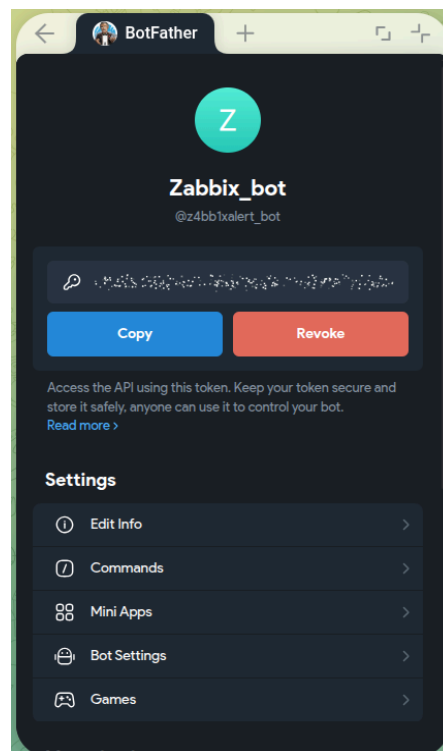


Una vez creamos el bot se nos abre un menú, en este debemos poner el nombre del bot con una descripción y el nombre de usuario con la extensión t.me (telegram)



The screenshot shows the 'New bot' interface in BotFather. At the top, there's a camera icon and the title 'New bot' with the instruction 'Enter a name, description and username to create a new bot.' Below this are three input fields: the first contains 'Zabbix_bot', the second contains 'Bot de gestión de alertas del servidor Zabbix', and the third contains 't.me/z4bb1x_alert_bot'. A green message below the third field states 'z4bb1x_alert_bot is available.' Below this is a note: 'Choose a username for your bot. It must end in "bot". Like this, for example: TetrisBot or tetris_bot.' At the bottom is a large blue button labeled 'Create Bot'.

Después de crearlo nos iremos a su chat y entraremos. Una vez entramos nos aparece un código el cual debemos pegar en el menú que nos aparecerá más abajo



BotFather

Sorry, this username is invalid.

14:20

Open

Ahora vamos a configurar el medio de telegram desde el frontend de Zabbix. Para ello nos iremos a “**Alertas→Tipos de medios→Telegram**”, pinchamos en telegram y se nos abrirá la configuración que será crucial para nuestro grupo, en este menú tenemos que cambiar lo siguiente (configuración necesaria, ya lo demás se puede cambiar si se requiere):

- **Tipo Webhook:** Significa que le indicamos al servidor que no va a enviar un correo ni SMS sino un mensaje vía HTTP hacia otra máquina (Telegram)
- **Alert Message:** En este apartado podemos indicar lo que queremos enviar en el mensaje, normalmente se deja igual
- **Api_chat_id:** Aquí te dice dónde mandar el mensaje, podemos poner el id del grupo o dejarlo tal y como está ya que si lo dejamos así, Zabbix solo mira el id que configuramos antes en los medios del usuario Administrador
- **Api Token:** En esta línea debemos pegar el conjunto de letras y números que nos dio Bot Father anteriormente

Tipo de medio

Tipo de medio Plantillas de mensaje 10 Opciones

* Nombre: Telegram

Tipo: Webhook

Parámetros	Nombre	Valor	Acción
	alert_message	{ALERT.MESSAGE}	Eliminar
	alert_subject	{ALERT.SUBJECT}	Eliminar
	api_chat_id	{ALERT.SENDTO}	Eliminar
	api_parse_mode	HTML	Eliminar
	api_token	8589965451:AAH-Ra8pOxFwGH_	Eliminar
	event_nseverity	{EVENT.NSEVERITY}	Eliminar
	event_severity	{EVENT.SEVERITY}	Eliminar
	event_source	1	Eliminar
	event_tags	{EVENT.TAGS.JSON}	Eliminar
	event_update_nseverity	{EVENT.UPDATE.NSEVERITY}	Eliminar
	event_update_severity	{EVENT.UPDATE.SEVERITY}	Eliminar
	event_update_status	{EVENT.UPDATE.STATUS}	Eliminar
	event_value	{EVENT.VALUE}	Eliminar

[Agregar](#)

* Comando: const CLogger = function(serviceName) {...

* Tiempo de espera: 30s

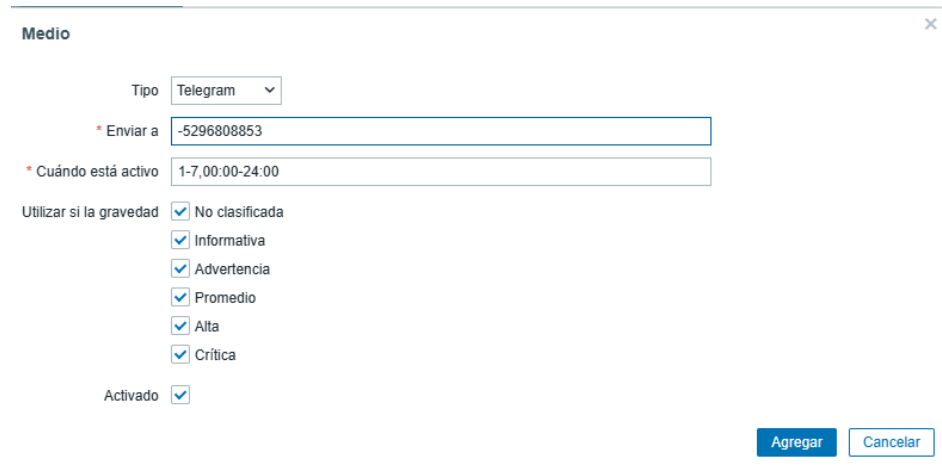
Etiquetas de proceso: ☒

Incluir entrada del menú de evento: ☐

* Nombre del menú de entrada:

[Modificar](#) [Clonar](#) [Eliminar](#) [Cancelar](#)

Después debemos añadir el id del grupo en el usuario administrador por ejemplo del zabbix para que se envíen los mensajes a ese grupo, para ello entramos en el apartado **“Usuario→Administrador→Medios→Introducir ID del grupo”**



Por último debemos activar la opción de que al ocurrir un fallo se reporten los problemas al administrador ya que si en caso de que algún equipo de error y esta opción no esté activada, el bot no sabrá que ese equipo ha fallado por lo tanto no mandará ningún mensaje



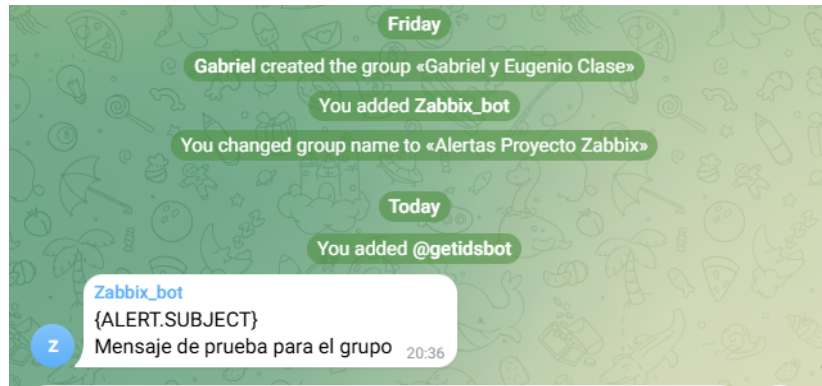
Después de configurar todo lo anterior, probaremos un mensaje de alerta para ver si todo está bien, para ello entramos en el final del apartado de telegram donde dice probar y pinchamos en probar ya que las características se llenan solas (a veces hay que poner el id del grupo)

The screenshot shows the Zabbix web interface with a modal dialog titled "Probar el tipo de medio 'Telegram'". The dialog contains the following fields:

- alert_message**: Mensaje de prueba para el grupo
- alert_subject**: {ALERT.SUBJECT}
- api_chat_id**: -5296808853
- api_parse_mode**: HTML
- api_token**: 8589965451:AAH-Ra8pOxFwGH_nN12tyky1bR9cEFmMm0
- event_nseverity**: {EVENT.NSEVERITY}
- event_severity**: {EVENT.SEVERITY}
- event_source**: 1
- event_tags**: {EVENT.TAGSJSON}
- event_update_nseverity**: {EVENT.UPDATE.NSEVERITY}
- event_update_severity**: {EVENT.UPDATE.SEVERITY}
- event_update_status**: {EVENT.UPDATE.STATUS}
- event_value**: {EVENT.VALUE}
- Respuesta**: false

Below the fields, it says "Tipo de respuesta: Cadena" and "Abrir registro". At the bottom right of the dialog are two buttons: "Probar" and "Cancelar".

Una vez le damos a probar, si lo hemos hecho todo bien el bot nos debería mandar un mensaje de prueba al grupo de telegram



Ahora vamos a realizar la prueba de fuego, vamos a configurar un equipo de prueba en el que le instalaremos el agente Zabbix y estará en la misma red que el propio servidor

Nuevo equipo

Equipo IPMI Etiquetas Macros Inventario Cifrado Asignación de valores

* Nombre de equipo: Portatil Prueba

Nombre visible: Portatil Prueba

Plantillas: Windows by Zabbix agent X (pulse aquí para buscar) [Seleccione]

* Grupos de equipos: Virtual machines X (pulse aquí para buscar) [Seleccione]

Interfaces	Tipo	Dirección IP	Nombre DNS	Conectado a	Puerto	Por defecto
Agente		192.168.1.55		IP DNS	10050	☒ Eliminar

[Agregar](#)

Descripción

Monitorizado por: Servidor Proxy Grupo de proxy

Activado ☒

[Agregar](#) [Cancelar](#)

Tendremos que esperar a que se ponga en verde y una vez que este esté en verde le provocaremos un fallo intencionado por ejemplo ponerle una ip incorrecta

Portatil Prueba 192.168.1.55:10050 ZBX class: os target: windows Ai

Equipo ?

Equipo IPMI Etiquetas Macros Inventario Cifrado Asignación de valores

* Nombre de equipo Portatil Prueba

Nombre visible Portatil Prueba

Plantillas Nombre Acción

Windows by Zabbix agent Desvincular Desvincular y eliminar

pulse aquí para buscar Seleccione

* Grupos de equipos Virtual machines X

pulse aquí para buscar Seleccione

Interfaces Tipo Dirección IP Nombre DNS Conectado a Puerto Por defecto

Agente 192.168.1.20 IP DNS 10050 Eliminar

Agregar

Descripción

Monitorizado por Servidor Proxy Grupo de proxy

Activado ☒

Modificar Clonar Eliminar Cancel

Una vez que el equipo se ponga en rojo vamos a volver a ponerlo bien tal y como estaba antes para comprobar que también nos da el aviso de que todo ha vuelto a la normalidad

Equipo ?

Equipo IPMI Etiquetas Macros Inventario Cifrado Asignación de valores

* Nombre de equipo Portatil Prueba

Nombre visible Portatil Prueba

Plantillas Nombre Acción

Windows by Zabbix agent Desvincular Desvincular y eliminar

pulse aquí para buscar Seleccione

* Grupos de equipos Virtual machines X

pulse aquí para buscar Seleccione

Interfaces Tipo Dirección IP Nombre DNS Conectado a Puerto Por defecto

Agente 192.168.1.55 IP DNS 10050 Eliminar

Agregar

Descripción

Monitorizado por Servidor Proxy Grupo de proxy

Activado ☒

Modificar Clonar Eliminar Cancel

Finalmente si todo ha salido bien, deberíamos de tener las notificaciones correspondientes de nuestro bot de telegram en las que indique que el equipo ha sufrido un fallo y después que este se recuperó ya que provocamos el fallo y posteriormente lo arreglamos

The screenshot displays the Zabbix web interface. On the left, a Telegram chat window shows two messages from 'Zabbix_bot' (ALERT.SUBJECT). The first message is a test message. The second message is an alert about a Windows agent being unavailable on a host named 'Portatil Prueba'. The alert details include the problem name, host, severity, and operational data. On the right, the Zabbix interface shows a sidebar with navigation options and a table of hosts.

Nombre	Interfaz	Disponibilidad	Etiquetas
Portatil Prueba	192.168.1.55:10050	ZBX	class:
Router Mikrotik	192.168.100.201:161	SNMP	class:
Switch A 1 ASIR	172.22.14.10:161	SNMP	class:
Switch B 1 ASIR	172.22.14.11:161	SNMP	class:
Switch_Proyecto_Zabbix	172.22.12.25:161	SNMP	class:
WIN-DHCP	192.168.100.204:10050	ZBX	class:
Zabbix server	127.0.0.1:10050	ZBX	class:

16. ADMINISTRACIÓN

16.1. GESTIÓN DE USUARIOS Y PERMISOS

Tal y como se mencionaba en la fase anterior, para poder asegurar el hecho de tener un servidor seguro, vamos a utilizar una estructura de control de acceso que se basa en roles de usuario.

En lo que se centra esta estructura es en que no se utilice la cuenta de “SuperAdmin” para tareas rutinarias de monitorización de dispositivos, sino que haya otra serie de usuarios encargados en hacer eso y que estos tengan una serie de permisos distintos

16.2. MONITOREO Y MANTENIMIENTO EN RED

Es importante controlar siempre la infraestructura de la red del servidor para poder evitar todo tipo de problemas que puedan ocurrir y así maximizar en medida de lo posible la seguridad de este, para ello hemos tenido en cuenta una serie de conceptos importantes

16.3. MONITOREO DE DISPONIBILIDAD

El primer concepto que hemos visto es el hecho de poder comprobar la disponibilidad de los dispositivos en todo momento haciendo uso del tradicional comando “ICMP/Ping”, pero en este caso directamente desde el frontend de Zabbix. Para ello debemos seguir los siguientes pasos:

1. Habilitar el comando desde el servidor Debian 12 (Comprobar la versión)

Lo primero que debemos saber es que sin habilitar el comando “ping” desde el propio servidor, nos saltará un error diciendo que el comando no está permitido para usar.

Para ello lo que debemos hacer es habilitar el comando desde el terminal modificando el archivo `/etc/zabbix/zabbix_agentd.conf`. Dentro de este archivo encontramos 2 opciones para habilitar estos comandos desde el frontend, tenemos tanto “Enable Remote Commands” para versiones antiguas y “AllowKey=system.run” para versiones más nuevas (desde la 5.0 en adelante)

```
/etc/zabbix/zabbix_agentd.conf 0 Li[62 0 62/557] *(1924/173235) 0095 0x023
# Mandatory: no
# Default:
# SourceIPs:

### Option: Allowkey
# Allow execution of items keys matching pattern.
# Multiple keys matching rules may be defined in combination with Denykey.
# Key pattern is wildcard expression, which support "*" character to match any number of any characters in certain position. It might be used in both key
# Parameters are processed one by one according their appearance order.
# If no Allowkey or Denykey rules defined, all keys are allowed.
# Mandatory: no

### Option: Denykey
# Deny execution of items keys matching pattern.
# Multiple keys matching rules may be defined in combination with Allowkey.
# Key pattern is wildcard expression, which support "*" character to match any number of any characters in certain position. It might be used in both key
# Parameters are processed one by one according their appearance order.
# If no Allowkey or Denykey rules defined, all keys are allowed.
# Unless another system.run[*] rule is specified Denykey=system.run[*] is added by default.
# Mandatory: no
# Default:
# Denykey=system.run[*]

### Option: EnableRemoteCommands - Deprecated, use Allowkey=system.run[*] or Denykey=system.run[*] instead
# Internal alias for Allowkey/Denykey parameters depending on value:
# 0 - Denykey=system.run[*]
# 1 - Allowkey=system.run[*]
#
```

Si usamos el comando `zabbix_server -V`, podemos ver la versión que tenemos instalada en el servidor, esto es muy importante para ver si tenemos que usar la opción “Enable Remote Commands” (vieja) o la opción “AllowKey=system.run” (nueva)

```
zabbix_server (Zabbix) 7.4.6
Revision 626d2b8a482 17 December 2025, compilation time: Dec 17 2025 14:02:54

Copyright (C) 2025 Zabbix SIA
License AGPLv3: GNU Affero General Public License version 3 <https://www.gnu.org/licenses/>.
This is free software: you are free to change and redistribute it according to
the license. There is NO WARRANTY, to the extent permitted by law.

This product includes software developed by the OpenSSL Project
for use in the OpenSSL Toolkit (http://www.openssl.org/).

Compiled with OpenSSL 3.0.13 30 Jan 2024
Running with OpenSSL 3.0.13 30 Jan 2024
```

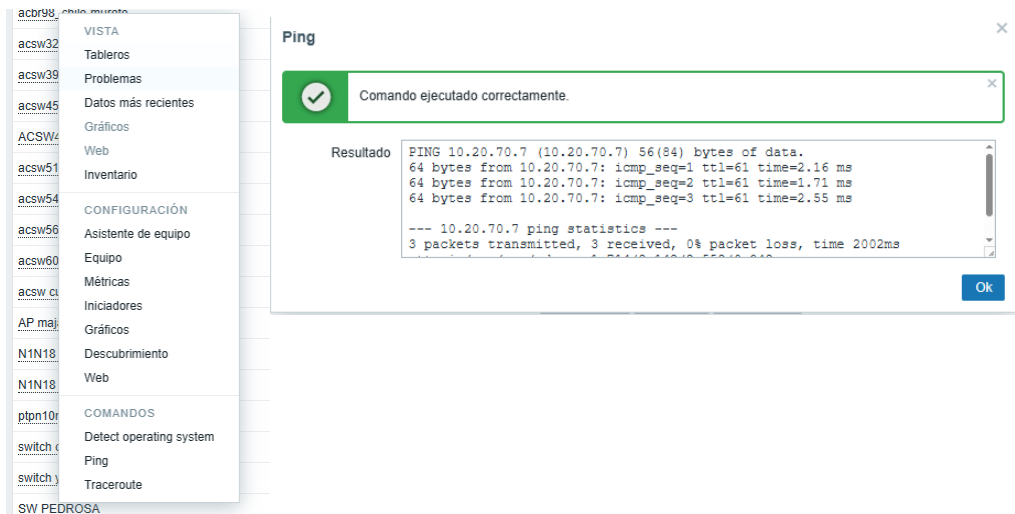
2. Activar la opción en el archivo de configuración

Una vez que sabemos qué opción debemos usar según la versión del agente y servidor Zabbix que tengamos instalada, procedemos a activarla.

En nuestro caso tenemos la versión 7.4 entonces usamos el AllowKey ya que la de Enable Remote Commands aparece como “deprecated” o en español “obsoleta”, para ello quitamos la “#” que aparece delante de la opción “AllowKey” y guardamos los cambios

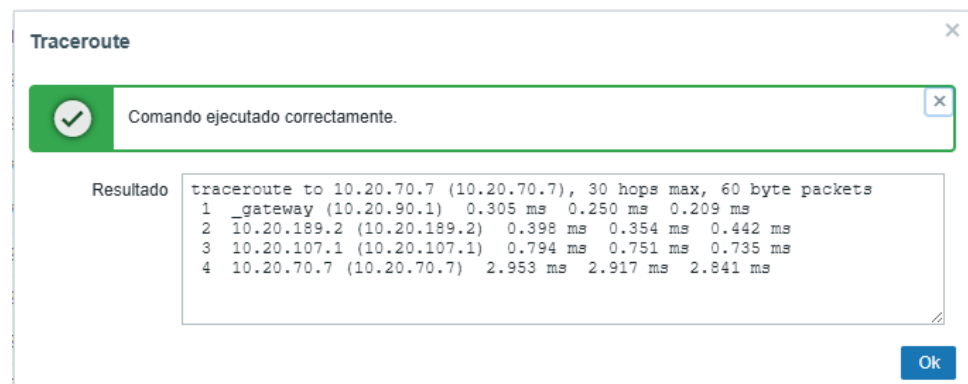
```
### Option: EnableRemoteCommands - Deprecated, use Allowkey=system.run[*] or Denykey=system.run[*] instead
# Internal alias for Allowkey/Denykey parameters depending on value:
# 0 - Denykey=system.run[*]
# 1 - Allowkey=system.run[*]
#
```

Finalmente, para comprobar si esto ha funcionado, debemos irnos al frontend, seleccionar un equipo cualquiera y en la parte de comandos pinchar en ping, si se ejecuta el comando ya sea que el equipo responda o no ya estará bien hecho

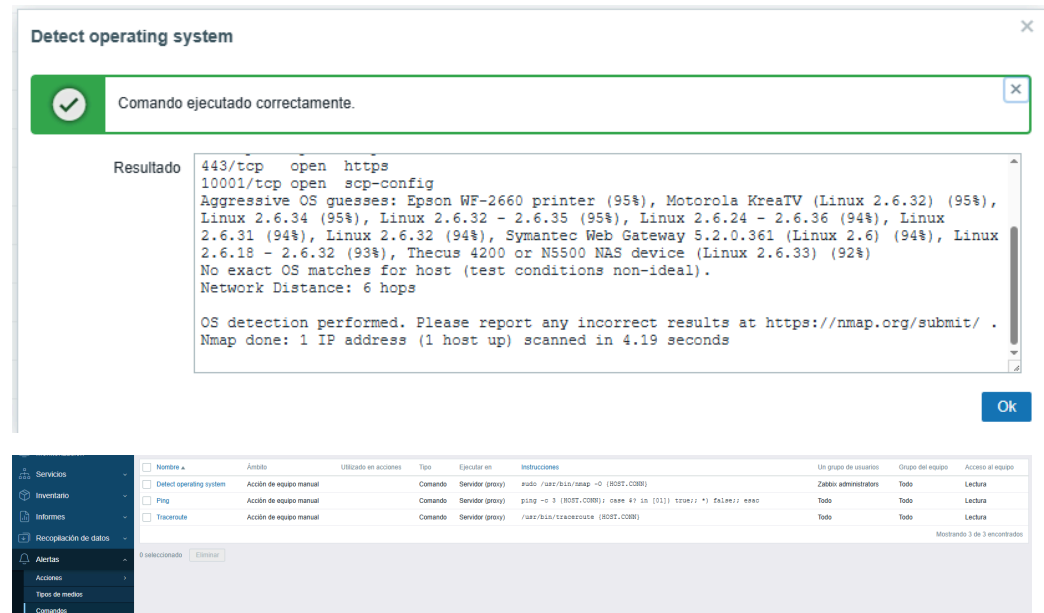


Además de este comando podemos usar otros más como por ejemplo nos podemos referir al traceroute o el comando para detectar el sistema operativo de cualquier dispositivo

Al usar el comando traceroute podemos ver cuando saltos se realizan desde la dirección de ese dispositivo hasta su gateway o puerta de enlace



En la siguiente imagen podemos ver cómo al hacer uso del comando “Detect operating system” o “Detectar sistema operativo”, se puede ver como dicho equipo al que hemos hecho la búsqueda aparece por ejemplo “Linux”



16.4. POLÍTICAS DE SEGURIDAD

A lo largo de este proyecto, ya sea en la fase de instalación, configuración o gestión de Zabbix, nos hemos esforzado en utilizar una serie de políticas de seguridad para poder proteger los datos y el sistema de monitorización para que este sea seguro y tenga los menores problemas posibles, para ello nos hemos centrado en los siguientes conceptos:

1. Configuración Segura del Servidor con el Agente Zabbix

Debido a que la versión del servidor Zabbix que estamos utilizando es la 7.4 (versión moderna), hemos seguido las recomendaciones que ponen en su página oficial en cuanto a los parámetros que se deben utilizar para poder hacer uso de los comandos remotos.

Como hemos comentado anteriormente, hemos descartado la opción de usar el parámetro “Enable Remote Commands” ya que este es el que se usaba en versiones antiguas (por debajo de la 5.0). Además de esto, este parámetro te permitía usar todos los comandos disponibles (1) o ninguno (0).

Al contrario de “Allowkey=system.run[*]”, este tiene dos opciones, ya sea “Allowkey” o “Denykey” las cuales te permite usar los comandos que nosotros queremos que se habiliten

Estos comandos permiten hacer lo que se llaman “listas blancas y negras” lo que lo explico a continuación con un ejemplo:

- Vamos a dejar pasar comandos que no hacen daño al sistema como el ping para comprobar la conexión entre el servidor y los equipos y df -h para comprobar el espacio en disco

```
AllowKey=system.run[ping -c 3 *]  
AllowKey=system.run[df -h]
```

- Después vamos a bloquear comandos que pueden ser críticos para el sistema como el comando rm con todos sus parámetros para borrar y reboot para reiniciar

```
DenyKey=system.run[rm *]  
DenyKey=system.run[reboot]
```

- Finalmente vamos a poner una opción en el servidor para que si alguien ejecuta cualquier otro comando que no esté arriba este se bloquee

```
AllowKey=system.run[ping -c 3 *]  
AllowKey=system.run[df -h]  
:  
DenyKey=system.run[rm *]  
DenyKey=system.run[reboot]  
:  
Denykey=system.run[*]
```

2. Uso de roles de usuario para gestionar el frontend

Como mencionamos en los apartados anteriores, hemos hecho uso de diferentes roles de usuario para no tener que depender de la cuenta de administrador para gestionar el servidor en todo

momento, así hacemos que haya que usar diferentes cuentas de usuario con contraseñas fuertes

3. Cifrado en el frontend del servidor Zabbix para que use el protocolo HTTPS

Para poder tener la mayor seguridad posible en nuestro servidor hemos configurado en la máquina Debian 12, el poder hacer uso del protocolo HTTPS (Hyper Text Transfer Protocol “Secure”) y así asegurar que las credenciales de las cuentas no viajen por internet y puedan robarla los atacantes.

Para poder usar el protocolo HTTP con el servidor Zabbix hemos seguido los siguientes pasos:

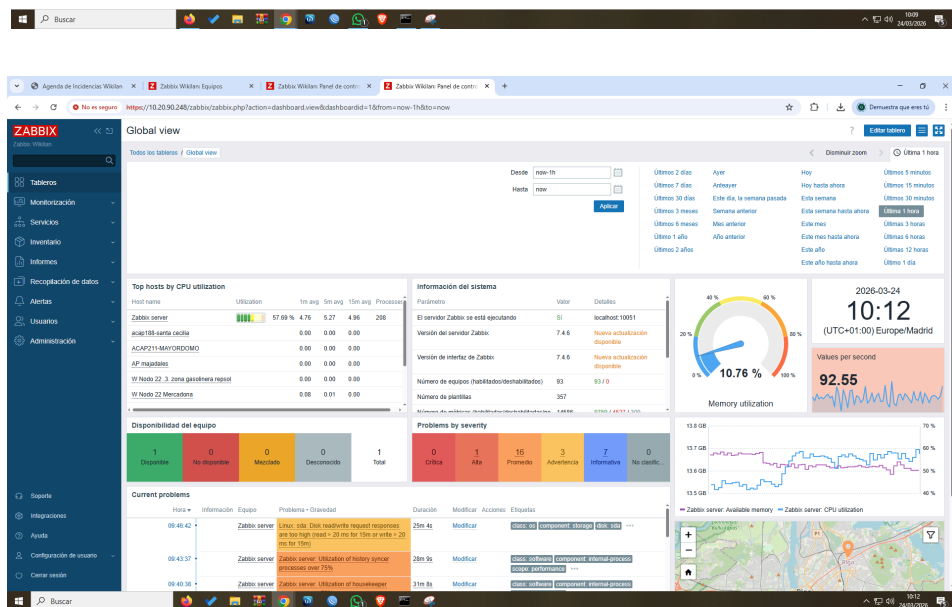
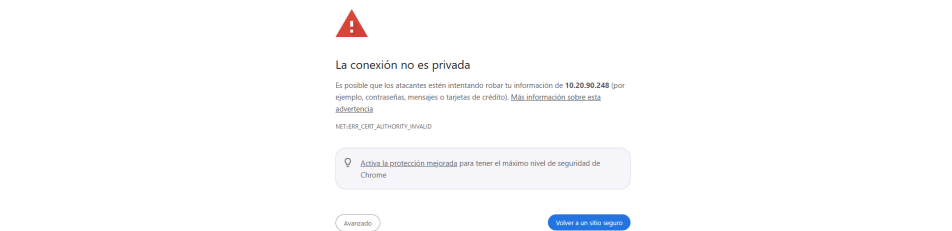
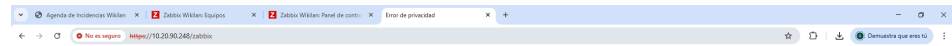
Primero debemos activar el módulo SSL en apache, activar la plantilla de sitio seguro que trae Debian por defecto y reiniciar el servicio Apache para poder aplicar los cambios

```
root@wikilan:/home/wikilan# a2enmod ssl
Considering dependency mime for ssl:
Module mime already enabled
Considering dependency socache_shmcb for ssl:
Module socache_shmcb already enabled
Module ssl already enabled
root@wikilan:/home/wikilan# a2ensite default-ssl
Enabling site default-ssl.
To activate the new configuration, you need to run:
  systemctl reload apache2
root@wikilan:/home/wikilan# systemctl restart apache2
root@wikilan:/home/wikilan# systemctl status apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
   Active: active (running) since Tue 2026-03-24 10:04:16 CET; 7s ago
     Docs: https://httpd.apache.org/docs/2.4/
   Process: 100928 ExecStart=/usr/sbin/apachectl start (code=exited, status=0/SUCCESS)
    Main PID: 100932 (apache2)
      Tasks: 7 (limit: 18642)
     Memory: 25.0M (peak: 25.2M)
        CPU: 449ms
   CGroup: /system.slice/apache2.service
           └─100932 /usr/sbin/apache2 -k start
             └─100934 /usr/sbin/apache2 -k start
               └─100935 /usr/sbin/apache2 -k start
                 └─100936 /usr/sbin/apache2 -k start
                   └─100937 /usr/sbin/apache2 -k start
                     └─100938 /usr/sbin/apache2 -k start
                       └─100943 /usr/sbin/apache2 -k start

mar 24 10:04:16 wikilan systemd[1]: Starting apache2.service - The Apache HTTP Server...
mar 24 10:04:16 wikilan apachectl[100931]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.
mar 24 10:04:16 wikilan systemd[1]: Started apache2.service - The Apache HTTP Server.
lines 1-21/21 (END)
```

Finalmente para comprobar lo anterior, ponemos “https/dirección o

carpeta del zabbix” y se verá cómo google te responde con un sitio no seguro, y al entrar en el frontend el “https” aparece tachado, pero esto es porque una empresa oficial no nos ha dado el certificado para tener una página segura, pero aun así hemos forzado al protocolo a actuar ya que este ya aparece



16.5. PLAN DE MANTENIMIENTO PREVENTIVO Y CORRECTIVO

16.5.1. MANTENIMIENTO PREVENTIVO

El mantenimiento preventivo se refiere a cómo vamos a **evitar** que el **servidor caiga** por falta de recursos o problemas (antes de que ocurra).

Actualizaciones

Las tareas principales que podemos realizar para llevar a cabo un buen mantenimiento preventivo son por ejemplo las **actualizaciones**, haciendo por ejemplo la **ejecución de scripts** que mencionamos antes con la herramienta **cron** para poder programar actualizaciones con “**apt update && upgrade && dist upgrade**” cada cierto tiempo

Por ejemplo, vamos a crear un script en el que haga todo lo anterior, para ello seguimos los siguientes pasos:

Primero crearemos dos carpetas, una para los scripts y otra para comprobar los ficheros de log

```
root@wikilan:/home/wikilan# ls
logs  mantenimiento  zabbix-release_latest_7.4+ubuntu24.04_all.deb
```

Lo primero que haremos será entrar en el editor nano y crear un script que tenga el inicio y fin del script para que este aparezca al comprobar los logs del sistema y redirigir esto y los comandos necesarios a un fichero de log para ver qué es lo que se ha hecho. En el script usamos comandos como el “logger” que lo que hace es **ejecutar el texto que pongamos entre comillas al comprobar los logs del sistema**.

Luego comandos como el “**echo**” que **imprimen texto en pantalla** que esto es muy útil antes de realizar cualquier proceso ya que este indica lo que se hace.

También los comandos que mencionamos anteriormente salvo que hacen uso del parámetro “-y” ya que todos ellos te preguntan que si quieres actualizar o no y como estas copias van a ser **programadas**, si queremos que se realicen **automáticamente** debe tener este

parámetro el cual contesta solo.

```
#!/bin/bash
logger "Inicio del script de actualizacion"
echo "Inicio del script" >> /home/wikilan/logs/actualizacion.txt
echo "-----" >> /home/wikilan/logs/actualizacion.txt
echo "Buscando actualizaciones para el sistema:" >> /home/wikilan/logs/actualizacion.txt
sudo apt update -y >> /home/wikilan/logs/actualizacion.txt
echo "-----" >> /home/wikilan/logs/actualizacion.txt
echo "Actualizando el sistema con los paquetes encontrados:" >> /home/wikilan/logs/actualizacion.txt
sudo apt upgrade -y >> /home/wikilan/logs/actualizacion.txt
echo "-----" >> /home/wikilan/logs/actualizacion.txt
echo "Comprobando la existencia de actualizaciones restantes:" >> /home/wikilan/logs/actualizacion.txt
sudo apt dist-upgrade -y >> /home/wikilan/logs/actualizacion.txt
echo "-----" >> /home/wikilan/logs/actualizacion.txt
echo "El sistema se ha actualizado por completo" >> /home/wikilan/logs/actualizacion.txt
echo "-----" >> /home/wikilan/logs/actualizacion.txt
echo "Fin del script" >> /home/wikilan/logs/actualizacion.txt
echo "-----" >> /home/wikilan/logs/actualizacion.txt
logger "Fin del script de actualizacion"
```

Finalmente le damos permisos al archivo para que se pueda ejecutar, leer y modificar

```
root@wikilan:/home/wikilan/mantenimiento# chmod 755 actualizacion.sh
root@wikilan:/home/wikilan/mantenimiento# ls
actualizacion.sh
```

Posteriormente, al crear el script ya podemos programar la tarea con cron, haciendo uso de su herramienta **crontab -e**. Esta tarea la programaremos para que se ejecute los **domingos a las 3 de la mañana** y también pondremos otra línea para que **se ejecute cada minuto para poder comprobar si el script está bien y el proceso que sigue**.

En las líneas ponemos **0 (minuto) 3 (horas) * (cualquier mes y día del mes) 0 (domingos)**

```
GNU nano 7.2 /tmp/crontab.b2W0Dp/crontab *
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow command
0 3 * * 0 /home/wikilan/mantenimiento/actualizacion.sh
*/1 * * * * /home/wikilan/mantenimiento/actualizacion.sh
```

Como podemos ver si hacemos un cat al fichero .txt que se deberá de haber generado al ejecutarse el script, es que ya sale que se están realizando las actualizaciones

```
root@wikilan:/home/wikilan# cat logs/actualizacion.txt _
Waiting for cache lock: Could not get lock /var/lib/dpkg/lock-frontent. It is held by process 105669 (apt)...
Waiting for cache lock: Could not get lock /var/lib/dpkg/lock-frontent. It is held by process 105669 (apt)...
Waiting for cache lock: Could not get lock /var/lib/dpkg/lock-frontent. It is held by process 105669 (apt)...
Waiting for cache lock: Could not get lock /var/lib/dpkg/lock-frontent. It is held by process 105669 (apt)...Inicio del script

Buscando actualizaciones para el sistema:
Obj:1 http://es.archive.ubuntu.com/ubuntu noble InRelease
Obj:2 http://es.archive.ubuntu.com/ubuntu noble-updates InRelease
Obj:3 http://es.archive.ubuntu.com/ubuntu noble-backports InRelease
Obj:4 http://security.ubuntu.com/ubuntu noble-security InRelease

Waiting for cache lock: Could not get lock /var/lib/dpkg/lock-frontent. It is held by process 105669 (apt)...
Waiting for cache lock: Could not get lock /var/lib/dpkg/lock-frontent. It is held by process 105669 (apt)...Preparando para desempaquetar ...
...6-ubuntu0.24.04.7_amd64.deb ...

Waiting for cache lock: Could not get lock /var/lib/dpkg/lock-frontent. It is held by process 105669 (apt)...Obj:5 https://repo.zabbix.com/zab
untu noble InRelease
Desempaquetando php8.3-... (8.3.6-ubuntu0.24.04.7) sobre (8.3.6-ubuntu0.24.04.6) ...
Obj:6 https://repo.zabbix.com/zabbix-tools/debian-ubuntu noble InRelease
Obj:7 https://repo.zabbix.com/zabbix/7.4/stable/ubuntu noble InRelease [4.681 B]

Waiting for cache lock: Could not get lock /var/lib/dpkg/lock-frontent. It is held by process 105669 (apt)...
Waiting for cache lock: Could not get lock /var/lib/dpkg/lock-frontent. It is held by process 105669 (apt)...root@wikilan:/home/wikilan#
```

Por último lo que debemos hacer, es quitar la línea de ejecución de 1 minuto que pusimos para comprobar que el script funcionaba correctamente

```
GNU nano 7.2 /tmp/crontab.WU141Y/crontab *
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
# m h dom mon dow command
0 3 * * 0 /home/wikilan/mantenimiento/actualizacion.sh
```

Revisión de logs del servidor

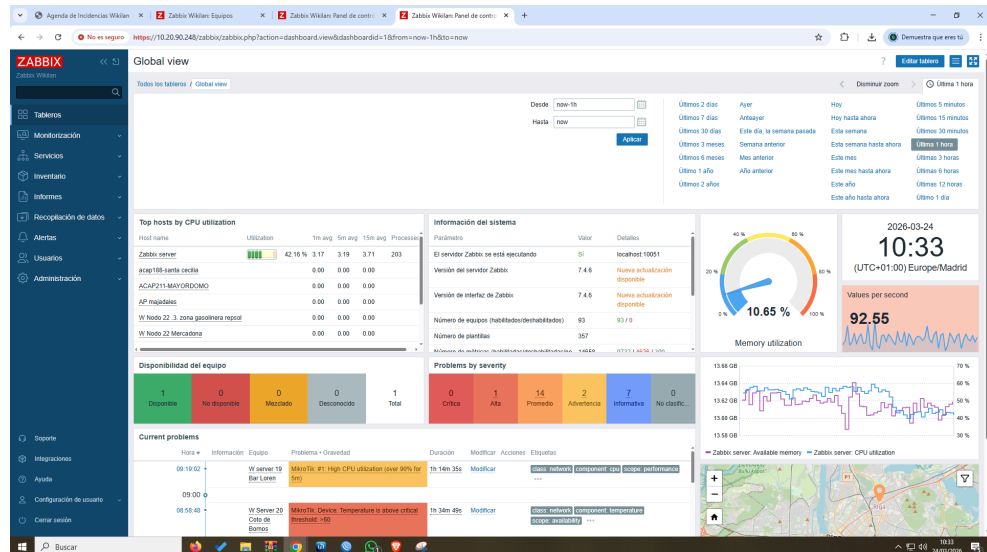
Otra de las tareas que podemos tener en cuenta es por ejemplo la **revisión diaria de los logs del servidor**, en ellos se pueden ver las **advertencias, problemas o información** diaria de este y así podemos tenerlo controlado en todo momento

Con el comando **tail -f** hacia la carpeta **/var/log/zabbix/zabbix_server.log**, podemos los logs del servidor en **tiempo real**.

```
root@wikilian:/home/wikilian# tail -f /var/log/zabbix/zabbix_server.log
04398:20260324:102844.468 resuming SNMP agent checks on host "W Server 17 Cerdilandia": connection restored
04398:20260324:102856.388 resuming SNMP agent checks on host "W Server 5 Guareña": connection restored
04398:20260324:102900.347 resuming SNMP agent checks on host "W Server 3 Cemerterio": connection restored
04398:20260324:102906.180 SNMP agent item "net.if.walk" on host "W Server 17 Cerdilandia" failed: first network error, wait for 15 seconds
04398:20260324:102906.180 SNMP agent item "net.if.walk" on host "W Server 26 Madronal" failed: first network error, wait for 15 seconds
04398:20260324:102910.083 SNMP agent item "net.if.walk" on host "W Server 5 Guareña" failed: first network error, wait for 15 seconds
04398:20260324:102922.347 SNMP agent item "net.if.walk" on host "W Server 3 Cemerterio" failed: first network error, wait for 15 seconds
04398:20260324:102925.332 resuming SNMP agent checks on host "W Server 17 Cerdilandia": connection restored
04398:20260324:102927.372 resuming SNMP agent checks on host "W Server 26 Madronal": connection restored
04398:20260324:102930.705 resuming SNMP agent checks on host "W Server 5 Guareña": connection restored
04398:20260324:102937.568 resuming SNMP agent checks on host "W Server 3 Cemerterio": connection restored
04398:20260324:102939.348 SNMP agent item "net.if.walk" on host "W Server 26 Madronal" failed: first network error, wait for 15 seconds
```

Automonitorización

Por último una de las tareas que considero de las más importantes es por ejemplo revisar la **automonitorización** que posee el propio servidor en el **frontend**, en ellas se pueden ver por ejemplo la carga de recursos (**CPU, RAM, almacenamiento...**), dando mensajes de alertas si por ejemplo el disco está apunto de estar lleno.



16.5.2. MANTENIMIENTO CORRECTIVO

En este apartado se explica cómo vamos a reaccionar en caso de que puedan ocurrir accidentes dentro del servidor, esto lo vamos a hacer con dos ejemplos:

Ejemplo 1

Lo primero que vamos a hacer es provocar un desastre que pueda ser crítico para el servidor, por ejemplo en el archivo `zabbix_agentd.conf`, vamos a escribir una línea que corrompa el archivo. Después vamos a reiniciar el servicio para que el servidor falle al leer este archivo

```
GNU nano 7.2 /etc/zabbix.
GabrielEstuvoAqui=FalloActuado1234567

# This is a configuration file for Zabbix agent daemon (Unix)
# To get more information about Zabbix, visit https://www.zabbix.com

##### GENERAL PARAMETERS #####
```

```
root@wikilan:/home/wikilan# systemctl restart zabbix-agent
Job for zabbix-agent.service failed because the control process exit
See "systemctl status zabbix-agent.service" and "journalctl -xeu zab
```

A la hora de actuar ante este problema, lo primero que haremos será comprobar el estado del servicio y ver si este se encuentra activo o no, en nuestro caso aparecerá inactivo

```
root@wikilan:/home/wikilan# systemctl status zabbix-agent
• zabbix-agent.service - Zabbix Agent
  Loaded: loaded (/usr/lib/systemd/system/zabbix-agent.service; enabled; preset: enabled)
  Active: activating (auto-restart) (Result: exit-code) since Tue 2026-03-24 11:18:03 CET; 3s ago
  Process: 132371 ExecStart=/usr/sbin/zabbix_agentd -c $CONFFILE (code=exited, status=1/FAILURE)
  CPU: 16ms
```

Si le hacemos un tail a las últimas 10 líneas del fichero de log del servidor podemos ver el error, deducimos que es del agente de Zabbix, por lo tanto está en el fichero que tocamos antes

```
root@wikilan:/home/wikilan# tail -n 10 /var/log/zabbix/zabbix_agentd.log
124196:20260324:111017.250 agent #7 started [listener #6]
124195:20260324:111017.251 agent #6 started [listener #5]
124198:20260324:111017.252 agent #9 started [listener #8]
124200:20260324:111017.253 agent #11 started [listener #10]
124197:20260324:111017.253 agent #8 started [listener #7]
124201:20260324:111017.255 agent #12 started [active checks #1]
124199:20260324:111017.256 agent #10 started [listener #9]
124201:20260324:111145.285 Active check configuration update started to fail
124189:20260324:111752.847 Got signal [signal:15(SIGTERM),sender_pid:132365,sender_uid:110,reason:0]. Exiting ...
124189:20260324:111752.852 Zabbix Agent stopped. Zabbix 7.4.8 (revision 6b0f6b25da0).
```

Después entramos en el fichero, borramos la línea incorrecta de este, reiniciamos el servicio y comprobamos el estado para ver si este ha vuelto a estar bien

```
GNU nano 7.2 /etc/zabbix/agent.conf
# This is a configuration file for Zabbix agent daemon (Unix)
# To get more information about Zabbix, visit https://www.zabbix.com

##### GENERAL PARAMETERS #####

### Option: PidFile
#     Name of PID file.
#
# Mandatory: no

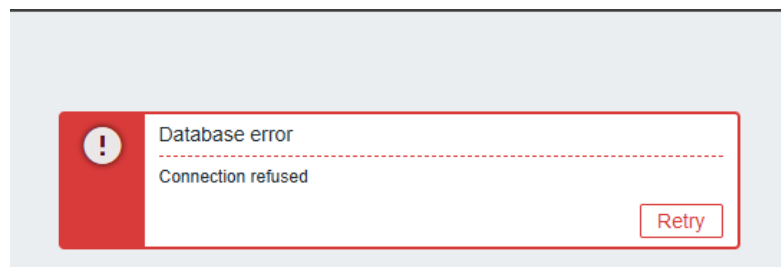
root@wikilan:/home/wikilan# systemctl restart zabbix-agent
root@wikilan:/home/wikilan# systemctl status zabbix-agent
• zabbix-agent.service - Zabbix Agent
   Loaded: loaded (/usr/lib/systemd/system/zabbix-agent.service; enabled; preset: enabled)
   Active: active (running) since Tue 2026-03-24 11:24:02 CET; 9s ago
     Process: 132484 ExecStart=/usr/sbin/zabbix_agentd -c $CONFFILE (code=exited, status=0/SUCCESS)
    Main PID: 132487 (zabbix_agentd)
      Tasks: 13 (limit: 18642)
     Memory: 8.6M (peak: 9.6M)
        CPU: 96ms
   CGroup: /system.slice/zabbix-agent.service
           └─132487 /usr/sbin/zabbix_agentd -c /etc/zabbix/zabbix_agentd.conf
             └─132488 "/usr/sbin/zabbix_agentd: collector [idle 1 sec]"
               └─132489 "/usr/sbin/zabbix_agentd: listener #1 [waiting for connection]"
                 └─132490 "/usr/sbin/zabbix_agentd: listener #2 [waiting for connection]"
                   └─132491 "/usr/sbin/zabbix_agentd: listener #3 [waiting for connection]"
                     └─132492 "/usr/sbin/zabbix_agentd: listener #4 [waiting for connection]"
                       └─132493 "/usr/sbin/zabbix_agentd: listener #5 [waiting for connection]"
                         └─132494 "/usr/sbin/zabbix_agentd: listener #6 [waiting for connection]"
                           └─132495 "/usr/sbin/zabbix_agentd: listener #7 [waiting for connection]"
                             └─132496 "/usr/sbin/zabbix_agentd: listener #8 [waiting for connection]"
                               └─132497 "/usr/sbin/zabbix_agentd: listener #9 [waiting for connection]"
                                 └─132498 "/usr/sbin/zabbix_agentd: listener #10 [waiting for connection]"
                                   └─132499 "/usr/sbin/zabbix_agentd: active checks #1 [idle 1 sec]"

mar 24 11:24:02 wikilan systemd[1]: Starting zabbix-agent.service - Zabbix Agent...
mar 24 11:24:02 wikilan systemd[1]: Started zabbix-agent.service - Zabbix Agent.
```

Ejemplo 2

En este caso, ocurrió un ejemplo real ya que de un momento a otro la base de datos dejó de responder así que para ello había que plantear una serie de soluciones

Lo primero es que podemos deducir que el error es de la base de datos ya que al entrar o manejar el frontend, este deja de responder y muestra un mensaje de error diciendo “Database error o error en la base de datos”



Una vez sabemos cuál es el motivo del error y este persiste por más veces que se intente recargar la página. Vamos a intentar comprobar el estado del servicio de la base de datos e intentamos recargarlo, en este caso el servidor se queda estancado intentando reiniciar el servicio ya que este no responde

```
root@wikilan:/home/wikilan/backups# systemctl status mysql
* mysql.service - MySQL Community Server
   Loaded: loaded (/usr/lib/systemd/system/mysql.service; enabled; preset: enabled)
   Active: inactive (dead) since Tue 2026-03-24 11:13:31 CET; 23h ago
   Duration: 23h 59min 26.585s
   Process: 948 ExecStart=/usr/sbin/mysqld (code=exited, status=0/SUCCESS)
   Main PID: 948 (code=exited, status=0/SUCCESS)
   Status: "Server shutdown complete"
   CPU: 2h 56min 55.606s

mar 23 11:11:48 wikilan systemd[1]: Starting mysql.service - MySQL Community Server...
mar 23 11:12:13 wikilan systemd[1]: Started mysql.service - MySQL Community Server.
mar 24 11:11:40 wikilan systemd[1]: Stopping mysql.service - MySQL Community Server...
mar 24 11:13:31 wikilan systemd[1]: mysql.service: Deactivated successfully.
mar 24 11:13:31 wikilan systemd[1]: Stopped mysql.service - MySQL Community Server.
mar 24 11:13:31 wikilan systemd[1]: mysql.service: Consumed 2h 56min 55.606s CPU time, 4.5G memory peak, 0B memory swap peak.
root@wikilan:/home/wikilan/backups# systemctl restart mysql
```

Después de comprobar el estado del servicio, vamos a comprobar la carga de recursos tanto de la memoria RAM como el espacio del disco duro, finalmente también podemos comprobar los procesos del motor de la base de datos para ver desde cuando está apagada

```
root@wikilan:/home/wikilan/backups# df -h
Filesystem      Size  Used Avail Use% Mounted on
tmpfs           1.6G  1.1M  1.6G   1% /run
/dev/mapper/ubuntu--vg-ubuntu--lv 72G   25G   45G  36% /
tmpfs           7.7G   0  7.7G   0% /dev/shm
tmpfs           5.0M   0  5.0M   0% /run/lock
/dev/sda2       2.0G  198M  1.6G  11% /boot
tmpfs           1.6G   12K  1.6G   1% /run/user/1000
root@wikilan:/home/wikilan/backups# free -h
              total        used        free      shared  buff/cache   available
Mem:          15Gi         847Mi       7.9Gi         51Mi        6.9Gi        14Gi
Swap:         3.8Gi           0B        3.8Gi

root@wikilan:/home/wikilan/backups# sudo journalctl -u mysql -n 20 --no-pager
mar 23 11:08:22 wikilan systemd[1]: Stopping mysql.service - MySQL Community Server...
mar 23 11:11:17 wikilan systemd[1]: mysql.service: Deactivated successfully.
mar 23 11:11:17 wikilan systemd[1]: Stopped mysql.service - MySQL Community Server.
mar 23 11:11:17 wikilan systemd[1]: mysql.service: Consumed 16h 26min 29.714s CPU time, 6.7G memory peak, 0B memory swap peak.
-- Boot 30208f951fad4699b7bf87ec78dce877 --
mar 23 11:11:48 wikilan systemd[1]: Starting mysql.service - MySQL Community Server...
mar 23 11:12:13 wikilan systemd[1]: Started mysql.service - MySQL Community Server.
mar 24 11:11:40 wikilan systemd[1]: Stopping mysql.service - MySQL Community Server...
mar 24 11:13:31 wikilan systemd[1]: mysql.service: Deactivated successfully.
mar 24 11:13:31 wikilan systemd[1]: Stopped mysql.service - MySQL Community Server.
mar 24 11:13:31 wikilan systemd[1]: mysql.service: Consumed 2h 56min 55.606s CPU time, 4.5G memory peak, 0B memory swap peak.
```

Una vez hemos comprobado todo lo anterior, en este caso no había ningún problema con los recursos del equipo así que vamos a proceder a reiniciar la máquina por completo para que vuelva a arrancar todos los servicios desde 0

```
root@wikilan:~# reboot_
```

En nuestro caso, podemos ver que si comprobamos el servicio ya aparece activo y en el estado dice que el servidor está operativo

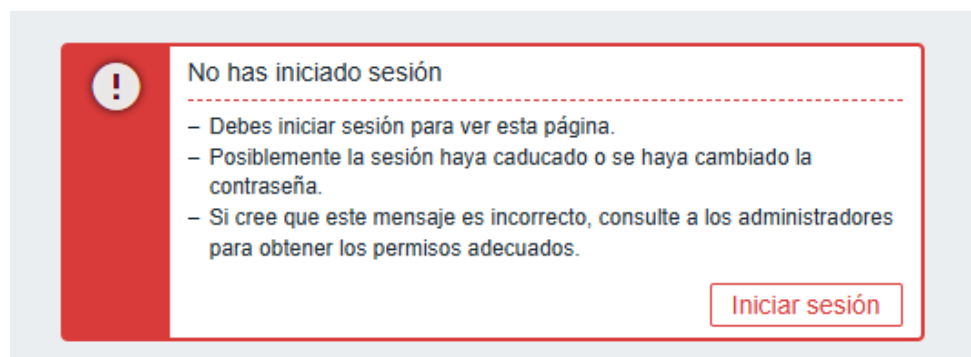
```
root@wikilan:~# systemctl status mysql
• mysql.service - MySQL Community Server
   Loaded: loaded (/usr/lib/systemd/system/mysql.service; enabled; preset: enabled)
   Active: active (running) since Wed 2026-03-25 11:18:47 CET; 6s ago
     Process: 718 ExecStartPre=/usr/share/mysql/mysql-systemd-start pre (code=exited, status=0/SUCCESS)
    Main PID: 920 (mysqld)
      Status: "Server is operational"
        Tasks: 39 (limit: 18640)
      Memory: 541.2M (peak: 541.4M)
         CPU: 5.061s
      CGroup: /system.slice/mysql.service
              └─920 /usr/sbin/mysqld

mar 25 11:18:14 wikilan systemd[1]: Starting mysql.service - MySQL Community Server...
mar 25 11:18:47 wikilan systemd[1]: Started mysql.service - MySQL Community Server.
```

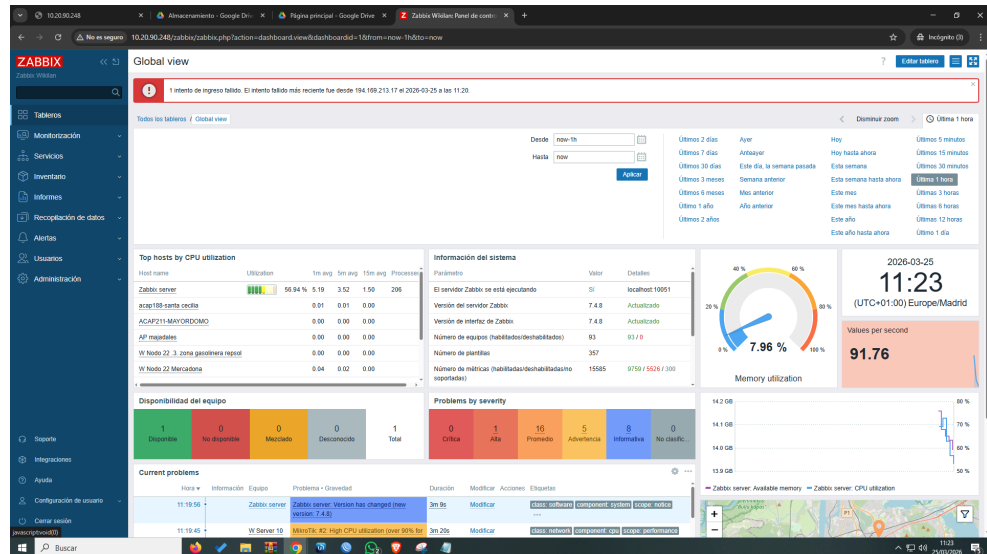
Finalmente, como tuvimos que reiniciar la máquina, el frontend de Zabbix se queda pillado y a veces introduciendo las credenciales correctamente no deja entrar en su frontend, así que para ello lo intentaremos de nuevo en una página de incógnito y veremos cómo ya si es posible acceder a este



The image shows the Zabbix login interface. At the top, there is a red box with the word "ZABBIX" in white. Below this, there is a form with the following fields: "Usuario" (User) with the value "Admin", "Contraseña" (Password) with masked characters ".....", and a checkbox labeled "Recordarme durante 30 días" (Remember me for 30 days) which is checked. A blue button labeled "Acceder" (Login) is at the bottom of the form. Below the form, there are links for "Ayuda" (Help) and "Soporte" (Support).



The image shows a red error message box with a white exclamation mark icon. The text inside the box reads: "No has iniciado sesión" (You have not logged in). Below this, there are three bullet points: "– Debes iniciar sesión para ver esta página." (You must log in to see this page.), "– Posiblemente la sesión haya caducado o se haya cambiado la contraseña." (Possibly the session has expired or the password has been changed.), and "– Si cree que este mensaje es incorrecto, consulte a los administradores para obtener los permisos adecuados." (If you believe this message is incorrect, consult the administrators to obtain the appropriate permissions.). At the bottom right of the box, there is a red button labeled "Iniciar sesión" (Log in).



16.6. IMPLEMENTACIÓN DE BACKUPS Y RECUPERACIÓN ANTE DESASTRES

En nuestro proyecto lo más importante no es el programa sino **los datos** que este posee en su interior, para ello necesitamos distintos planes para realizar la **recuperación y creación de copias de seguridad** para poder **actuar ante desastres**, esto lo podemos hacer de la misma forma como explicamos en el apartado anterior con las **tareas programadas**.

Para ello usaremos los scripts ejecutables que hemos estado mencionando todo este tiempo. Lo primero que haremos será crear una variable de **fecha** para poder poner la fecha en la que se **ejecuta** el script y en el nombre de la copia para ver **cuando se hace**. Luego usar el **inicio y fin** del script en el **logger** y en el **echo**. Usar el comando **tar**, **mysqldump** para hacer copias de seguridad tanto de los datos como de la base de datos y **ls** para comprobar el contenido y finalmente redirigirlo al fichero de log

```
#!/bin/bash
FECHA=$(date +%F)

logger "Inicio del script de backup"
echo "Inicio del script $FECHA" >> /home/wikilan/logs/backups.txt
echo "Realizando la copia de configuracion del servidor" >> /home/wikilan/logs/backups.txt
tar -czf /home/wikilan/backups/zabbix_config_$FECHA.tar.gz /etc/zabbix/ >> /home/wikilan/logs/backups.txt
echo "Realizando la copia a la base de datos" >> /home/wikilan/logs/backups.txt
mysqldump zabbix > /home/wikilan/backups/zabbix_db_$FECHA.sql >> /home/wikilan/logs/backups.txt
echo "Comprobando el contenido del directorio backups" >> /home/wikilan/logs/backups.txt
ls -la /home/wikilan/backups >> /home/wikilan/logs/backups.txt
echo "Fin del script $FECHA" >> /home/wikilan/logs/backups.txt
logger "Fin del script de backup"
```

Después de crearlo, hacemos lo de siempre, le damos permisos de ejecución al script para que al actuar la tarea programada pueda ejecutarse.

```
root@wikilan:/home/wikilan/backups# chmod 755 backups.sh
root@wikilan:/home/wikilan/backups# ls
backups.sh
```

Luego metemos la tarea programada en el crontab, en este caso programamos la tarea de copias de backups todos los días a las 4 de la mañana y también ponemos una cada minuto para comprobar que el script hace todo lo que debe.

```
GNU nano 7.2 /tmp/crontab.JX756H/crontab *
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow   command
0 3 * * 0 /home/wikilan/mantenimiento/actualizacion.sh
0 4 * * * /home/wikilan/backups/backups.sh
*/1 * * * * /home/wikilan/backups/backups.sh
```

Luego **esperamos un minuto** y vemos como en el `/var/log/syslog` aparece lo que escribimos con el comando **logger**.

```
root@wikilan:/home/wikilan/backups# tail -f /var/log/syslog
2026-03-24T11:50:00.519241+01:00 wikilan systemd[1]: sysstat-collect.service: Deactivated successfully.
2026-03-24T11:50:00.519661+01:00 wikilan systemd[1]: Finished sysstat-collect.service - system activity accounting tool.
2026-03-24T11:52:56.647813+01:00 wikilan crontab[132613]: (root) BEGIN EDIT (root)
2026-03-24T11:53:28.861306+01:00 wikilan crontab[132613]: (root) REPLACE (root)
2026-03-24T11:53:28.861675+01:00 wikilan crontab[132613]: (root) END EDIT (root)
2026-03-24T11:53:37.704794+01:00 wikilan crontab[132633]: (root) BEGIN EDIT (root)
2026-03-24T11:54:01.136772+01:00 wikilan cron[704]: (root) RELOAD (crontabs/root)
2026-03-24T11:55:01.144318+01:00 wikilan CRON[132645]: (root) CMD (command -v debian-sa1 > /dev/null && debian-sa1 1 1)
2026-03-24T11:56:03.123126+01:00 wikilan crontab[132633]: (root) REPLACE (root)
2026-03-24T11:56:03.123645+01:00 wikilan crontab[132633]: (root) END EDIT (root)
2026-03-24T11:57:01.150838+01:00 wikilan cron[704]: (root) RELOAD (crontabs/root)
2026-03-24T11:57:01.159469+01:00 wikilan CRON[132658]: (root) CMD (/home/wikilan/backups/backups.sh)
2026-03-24T11:57:01.171761+01:00 wikilan root: Inicio del script de backup
2026-03-24T11:57:01.593738+01:00 wikilan root: Fin del script de backup
2026-03-24T11:57:01.594555+01:00 wikilan CRON[132657]: (CRON) info (No MTA installed, discarding output)
```

Si el paso anterior salió bien, ahora comprobamos que en el fichero de log ocurre todo lo que habíamos puesto en el script, tanto los avisos de que iniciaba y terminaba el script, se hacían las diferentes copias y

posteriormente se comprobaba el contenido para ver si aparecían las copias ya hechas

```

root@wikilan:/home/wikilan/backups# cat /home/wikilan/logs/backups.txt
Inicio del script 2026-03-24

-----
Realizando la copia de configuracion del servidor
-----
Realizando la copia a la base de datos
-----
Comprobando el contenido del directorio backups
total 60
drwxr-xr-x 2 root    root      4096 mar 24 11:57 .
drwxr-x--- 9 wikilan wikilan  4096 mar 24 11:33 ..
-rwxr-xr-x 1 root    root      1253 mar 24 11:52 backups.sh
-rw-r--r-- 1 root    root     47087 mar 24 11:57 zabbix_config_2026-03-24.tar.gz
-rw-r--r-- 1 root    root         0 mar 24 11:57 zabbix_db_2026-03-24.sql
-----
Fin del script 2026-03-24

-----
Inicio del script 2026-03-24

-----
Realizando la copia de configuracion del servidor
-----
Realizando la copia a la base de datos
-----
Comprobando el contenido del directorio backups
total 60
drwxr-xr-x 2 root    root      4096 mar 24 11:57 .
drwxr-x--- 9 wikilan wikilan  4096 mar 24 11:33 ..
-rwxr-xr-x 1 root    root      1253 mar 24 11:52 backups.sh
-rw-r--r-- 1 root    root     47087 mar 24 11:58 zabbix_config_2026-03-24.tar.gz
-rw-r--r-- 1 root    root         0 mar 24 11:58 zabbix_db_2026-03-24.sql
-----
Fin del script 2026-03-24

```

Por último, cuando ya hemos visto que todo ha salido bien y el script actúa correctamente, borramos la línea de la tarea programada de cada minuto que pusimos para comprobar si el script funcionaba.

```

0 3 * * 0 /home/wikilan/mantenimiento/actualizacion.sh
0 4 * * * /home/wikilan/backups/backups.sh

```

Finalmente, he llegado a la conclusión de que lo mejor era ponerlo en dos horas diferentes para que cuando llegue el domingo y actúen las dos tareas **no haya sobrecarga** del equipo al intentar hacer las dos tareas

16.7. PLAN DE CONTINGENCIA

16.7.1. IDENTIFICACIÓN Y CLASIFICACIÓN DEL PROBLEMA

Lo primero de todo es que el administrador de sistemas ante cualquier fallos del servidor debe **clasificar el incidente** según los distintos niveles de alerta:

- Leve
- Grave
- Crítico

16.7.2. ACTUACIÓN ANTE ERROR

Error Leve

A la hora de referirnos a los errores leves podemos decir que son por ejemplo los **fallos de los distintos servicios** que hay dentro del servidor ya sea del propio servicio de Zabbix, error del motor de la base de datos como vimos previamente o del propio servidor web apache.

Para actuar ante estos errores siempre tendremos que comprobar el estado de los servicios, intentar reiniciarlos y comprobar los logs del servidor para poder identificar los errores

Error Grave

Si el servicio no arranca de ninguna forma posible como pasaba en el ejemplo anterior de cuando falló la base de datos, tendremos que **restaurar la última copia de seguridad** de la base de datos para poder **recuperar el estado** de cuando esta funcionaba correctamente.

Error Crítico

En cuanto a los errores críticos podemos referirnos a errores en el hardware más allá de quedarnos sin recursos disponibles, para ello debemos recuperar la máquina con **otra de reserva** o teniendo disponible un **snapshot** de cuando la máquina funcionaba correctamente y posteriormente hacer la **restauración completa** del servidor haciendo uso de los **backups disponibles**.

16.7.3. TIEMPOS DE RECUPERACIÓN

En los tiempos de recuperación nos referimos a los **tiempos que se tarda en recuperar los datos** con los backups disponibles y en los **puntos de recuperación** en los que podemos encontrarlos. En el caso de nuestro proyecto tenemos la recuperación garantizada en un periodo de 24 horas al haber programado los backups todos los días a las 4:00 AM.



16.8. SOPORTE A LA APLICACIÓN

Para garantizar un seguimiento de las averías y no depender de tener que avisar a técnicos de soporte de forma manual, podemos integrar nuestro servidor Zabbix con un sistema de tickets llamado HelpDesk.

16.8.1. HELPDESK

Este sistema de tickets como comentamos anteriormente sirve para que los técnicos puedan organizarse para no tengan todos la misma incidencia a la que acudir, es decir HelpDesk asigna un ticket a un técnico para una incidencia específica, esto lo explicamos a continuación con el proceso que sigue este sistema

Detección y aviso ante el error

Lo primero que ocurre es que Zabbix al detectar una incidencia utiliza sus umbrales de alerta, por ejemplo el envío de correos para

comunicarse con el sistema de tickets

Generación del ticket

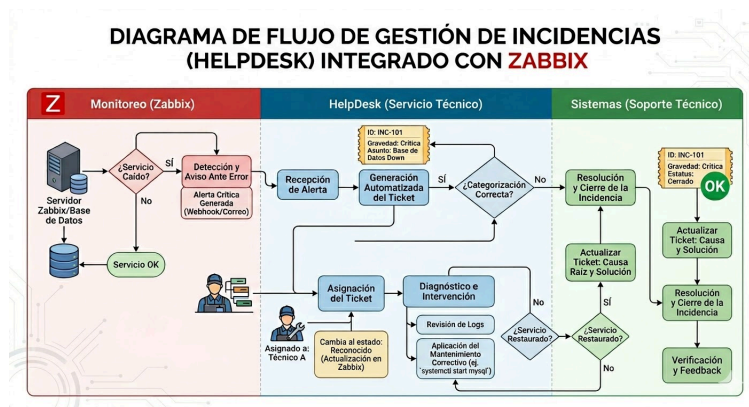
Cuando Zabbix envía el correo, HelpDesk recibe la alerta, por lo tanto este genera un ticket en el que se encuentra la fecha del problema, la gravedad y el dispositivo al que le ha ocurrido la incidencia

Asignación del ticket

Cuando el ticket ya está preparado, se le asigna a un técnico que esté disponible y a su vez Zabbix muestra en el estado del dispositivo afectado que está siendo evaluado, así si otro administrador ve que ese dispositivo tiene una incidencia puede saber que ya está siendo atendido

Cierre de la incidencia

Finalmente, si el técnico consigue resolver la incidencia, Zabbix detecta que se ha recuperado el estado correcto del equipo y se actualiza el ticket indicando que el problema está resuelto y ya lo único que queda es que el técnico documente la solución que se ha aplicado antes de cerrar el ticket



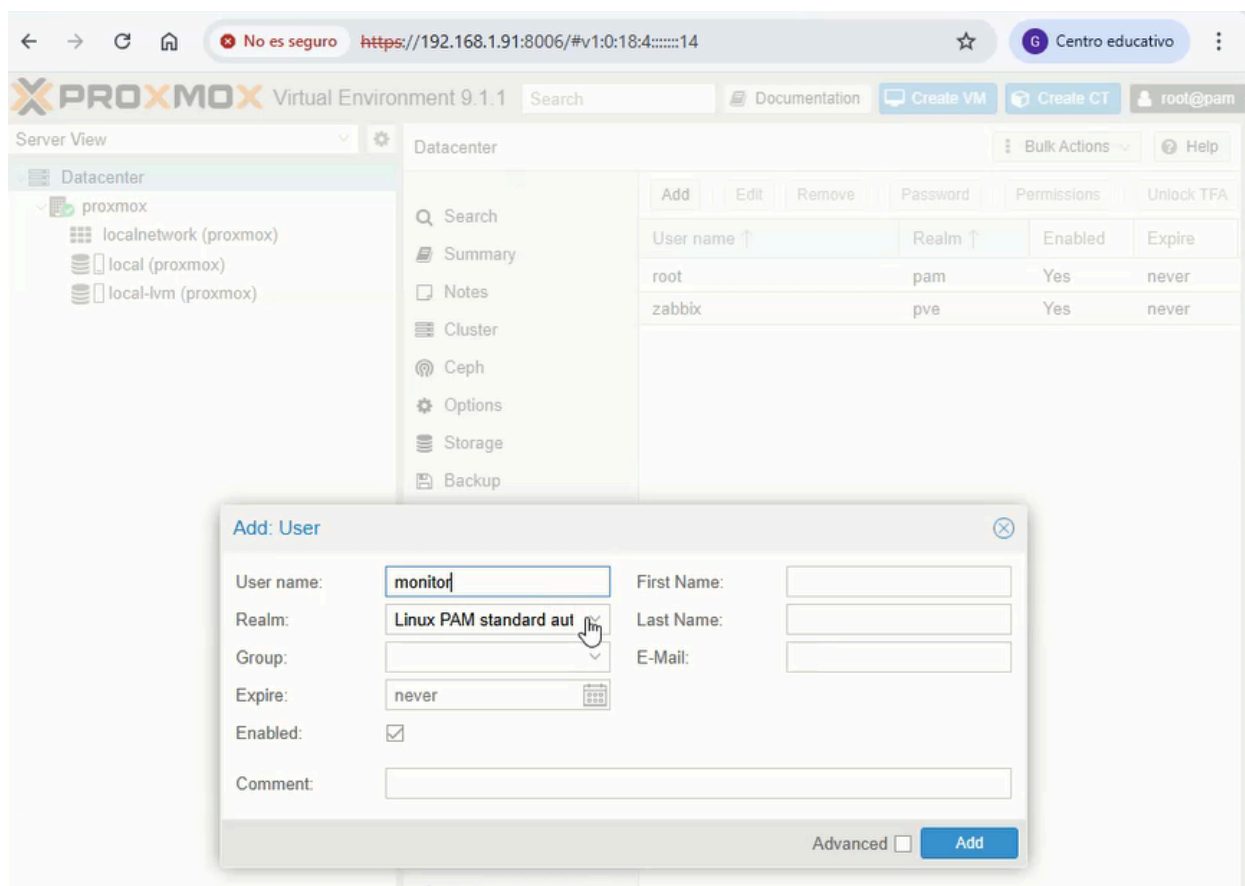
17. MONITORIZAR UN SERVIDOR PROXMOX EN ZABBIX

En este apartado se muestra cómo se integra un servidor Proxmox el cuál para este proyecto es un dispositivo o servicio crítico ya que con él vamos a poder crear distintas

máquinas virtuales en el servidor principal.

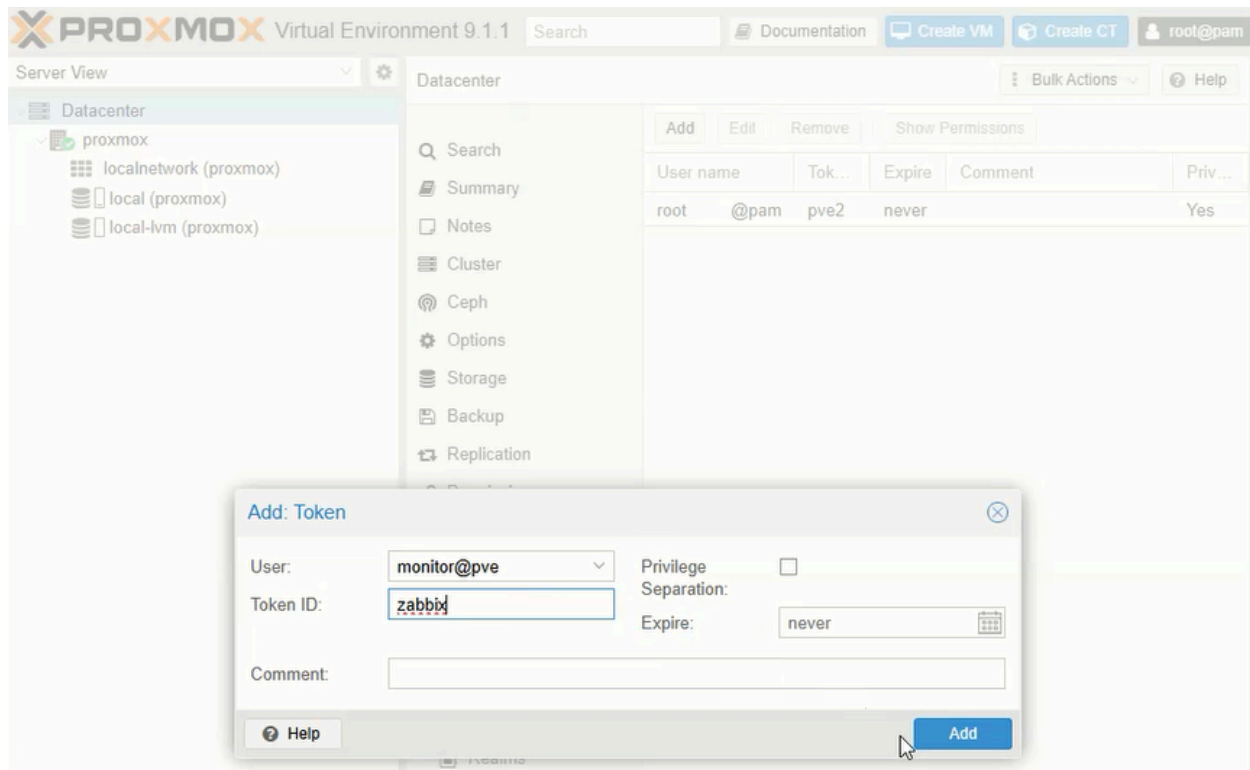
En primer lugar, una vez que tengamos el servidor Proxmox listo para usar, tendremos que entrar en su interfaz web, entrar en “DataCenter” → “Users” y crear el usuario monitor que será el que se va a integrar mediante las API entre Proxmox y Zabbix.

Esto lo hacemos para poder mantener un control de acceso estricto, ya que los procesos de monitorización de Zabbix se ejecutan usando esta cuenta que acabamos de crear en lugar de utilizar las demás del servidor mediante el Agente como hacemos con los demás dispositivos

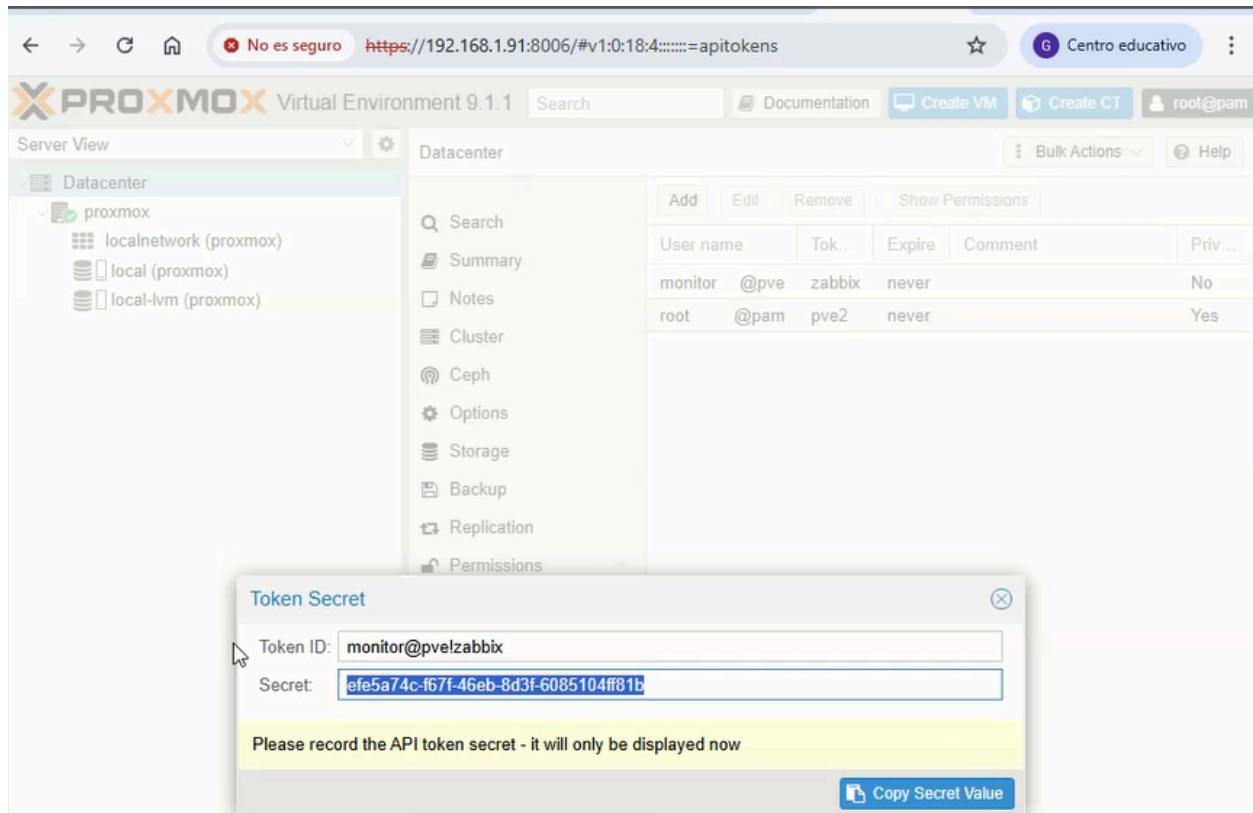


Una vez hemos creado el usuario con el que vamos a integrar Proxmox, hay que crear el Token de API para poder vincularlo.

Para ello entramos en el menú de Permisos, después buscamos la opción de API Tokens y le damos a añadir, después tendremos que rellenar los distintos campos como el usuario que creamos antes y un Token fácil como “zabbix” para acordarnos siempre de cómo se llama, finalmente tendremos que desmarcar la casilla de Separación de privilegios para que el Token tenga los permisos que nosotros pongamos después



Una vez tengamos creado el Token, hay que confirmar la creación de este, para ello se genera una contraseña secreta que la tendremos que guardar en un archivo de texto para tenerla siempre ya que este valor no se volverá a mostrar jamás a no ser de que tengamos que crear un Token pero que este sea totalmente distinto a este

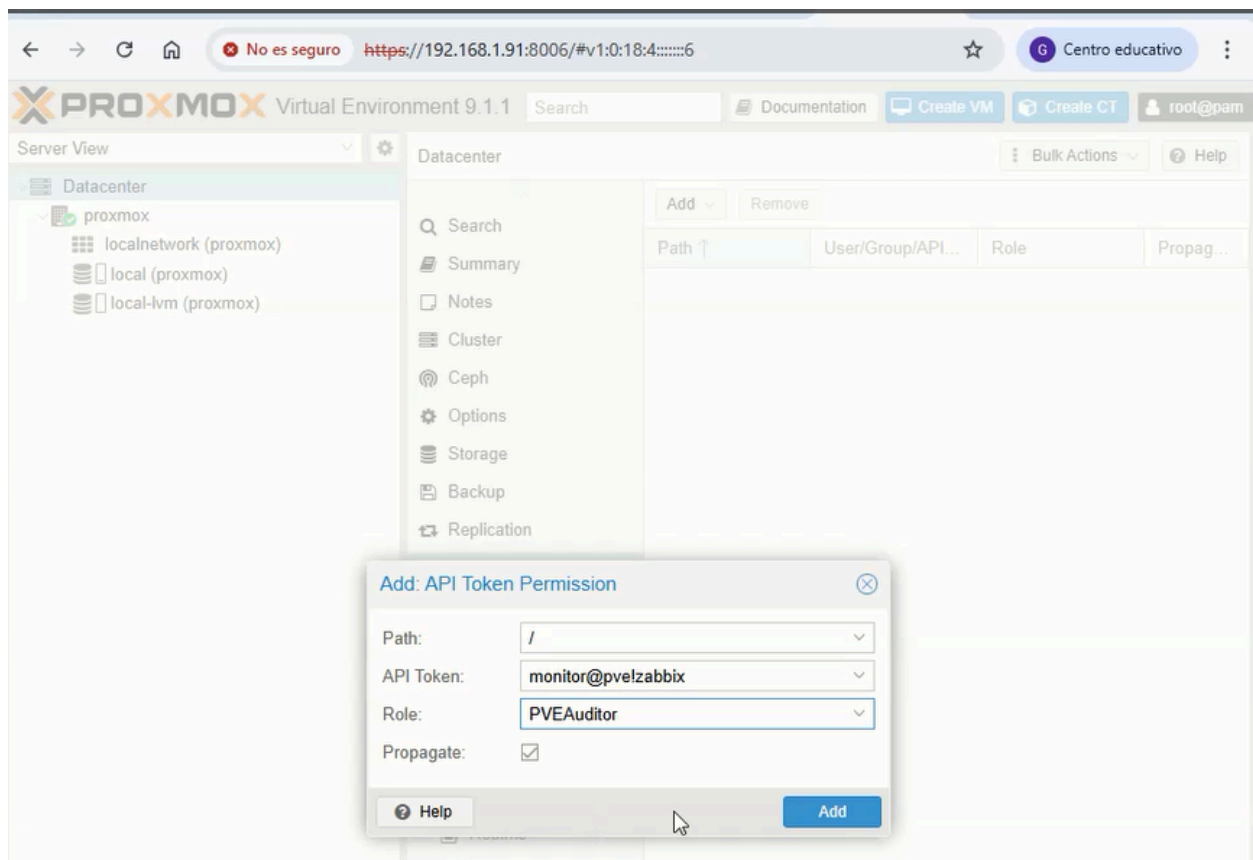


Ya que tenemos el Token creado con la clave secreta, hay que ponerle los permisos que el API Token va a tener

Para ello hay que entrar de nuevo en el apartado “Permisos” que se encuentra en “DataCenter”, hacer clic en añadir y seleccionar la opción API Token Permission

Después de esto hay que rellenar todos los campos con la siguiente información:

- **Path:** “/” es la raíz para que tenga alcance sobre todo el clúster
- **API Token:** En este campo tendremos que poner el nombre de usuario, equipo y nombre de la API que creamos antes el cual era el nombre fácil que hay que recordar siempre
- **Role:** Hay que poner el “PVEAuditor” que se encargará de establecer un control de acceso estricto en el que el agente sólo podrá hacer operaciones de lectura
- **Propagate:** Campo que debe estar marcado para que se establezcan todos los permisos creados

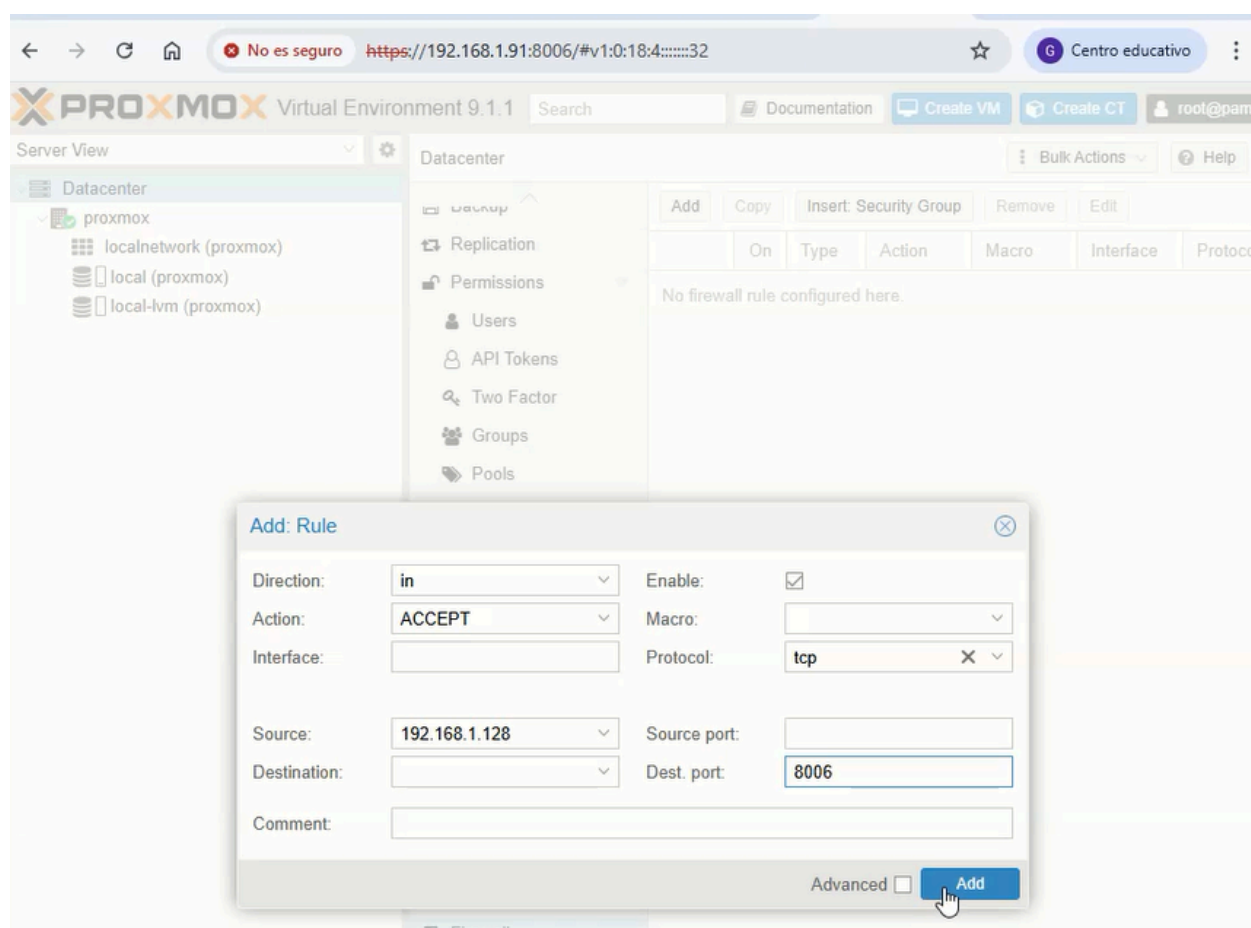


Ahora viene un paso importante para que se pueda establecer la conexión entre ambos servidores y este paso consiste en configurar el Firewall de Proxmox.

Para ello entramos en “Datacenter” → “Firewall” y pinchamos en añadir para crear la nueva regla de firewall

Dentro se abrirá un menú en el que deberán de aparecer todos los campos necesarios para poder crear dicha regla:

- **Direction:** IN, para dar a entender al Proxmox que el tráfico es entrante (De Zabbix a Proxmox)
- **Action:** Hay que poner “ACCEPT” para poder permitir la conexión entre ambos servidores
- **Protocol:** El protocolo que se sigue es el tradicional TCP
- **Source:** Hay que poner la dirección IP del servidor Zabbix
- **Dest.Port:** El puerto de gestión de Proxmox que por defecto es el 8006



Una vez hemos terminado toda la configuración desde el Proxmox, hay que configurar también desde Zabbix los distintos apartados para lograr la integración entre los dos servidores.

En Zabbix lo que tendremos que configurar son las distintas macros de Proxmox. Para ello entramos en el equipo de Proxmox y al tener puesta la plantilla suya, aparecen todas las macros de este dispositivo

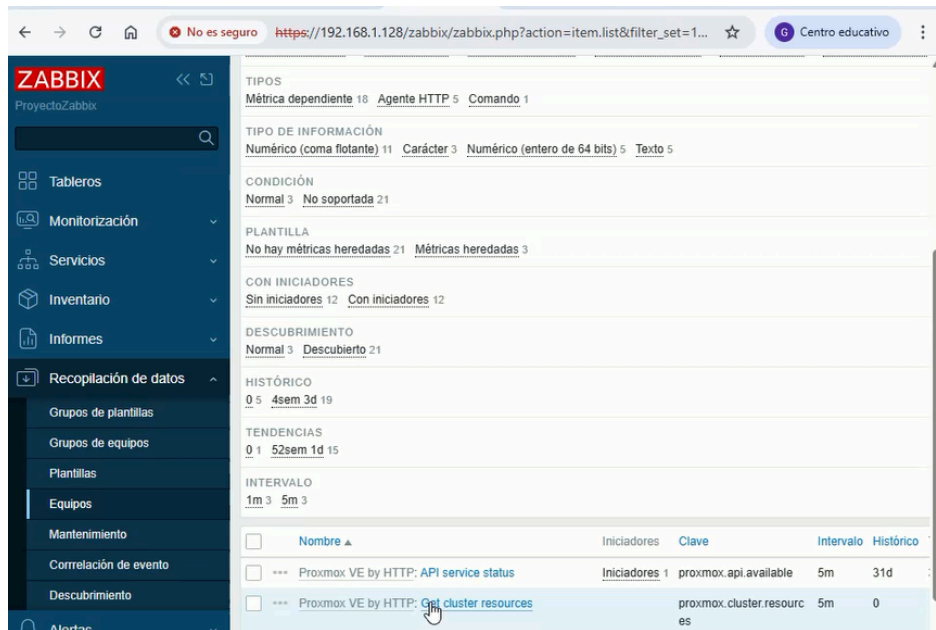
Si entramos en el apartado de Macros podemos ver todas las que tenemos que configurar como son por ejemplo:

- **{PVE.TOKEN.ID}**: Hay que introducir el ID completo (**monitor@pve!zabbix**)
- **{PVE.TOKEN.SECRET}**: Pegar el código secreto que se guardó anteriormente
- **{PVE.URL.HOST}**: Hay que escribir la dirección IP del servidor Proxmox
- **{PVE.URL.PORT}**: Se pone el puerto de Proxmox el cual es el 8006

The screenshot shows the Zabbix web interface in a browser. The address bar indicates the URL: `https://192.168.1.128/zabbix/zabbix.php?action=host.edit&hostid=10...`. The page title is "Equipos". The main content area is titled "Equipo" and shows a list of macros for a host. The macros are organized into a table with columns for the macro name, its value, and actions. The macros are:

Macro	Valor	Acción
{PVE.STORAGE.PUSE.MAX.WARN}	80	Cambio = Proxmox VE by HTTP: "80"
Aviso de peligro por poco espacio		
{PVE.SWAP.PUSE.MAX.CRIT}	90	Cambio = Proxmox VE by HTTP: "90"
Aviso crítico por sobrecarga del swap		
{PVE.SWAP.PUSE.MAX.WARN}	80	Cambio = Proxmox VE by HTTP: "80"
Aviso de peligro por sobrecarga del swap		
{PVE.TOKEN.ID}	monitor@pve!zabbix	Eliminar = Proxmox VE by HTTP: "USER@REALMITOKENID"
descripción		
{PVE.TOKEN.SECRET}	valor	Eliminar = Proxmox VE by HTTP: "xxxxxxxx-xxxx-xxxx-xxx..."
descripción		
{PVE.URL.HOST}	192.168.1.91	Eliminar = Proxmox VE by HTTP: "<SET PVE HOST>"
descripción		
{PVE.URL.PORT}	8006	Eliminar = Proxmox VE by HTTP: "8006"
descripción		
{PVE.VM.CPU.PUSE.MAX.CRIT}	90	Cambio = Proxmox VE by HTTP: "90"
descripción		
{PVE.VM.CPU.PUSE.MAX.WARN}	80	Cambio = Proxmox VE by HTTP: "80"
descripción		
{PVE.VM.MEMORY.PUSE.MAX.CRIT}	90	Cambio = Proxmox VE by HTTP: "90"
descripción		

Finalmente, para comprobar si está todo integrado correctamente, hay que ir al apartado de recopilación de datos → equipos y buscar una opción que se llama “Get cluster resources” para poder probar el intercambio entre los dos servidores



Después se abrirá el menú de métricas, en el tendremos que pinchar en probar

Lo último que queda por hacer es pinchar en “obtener el valor y probar”, cuando pinchemos aquí esperamos unos segundos y vemos que en la sección “Resultado” aparece una línea que dice “Resultado convertido a Texto {“data””, si aparece esto es porque Proxmox está correctamente integrado con Zabbix y todo lo anterior ha salido bien

ZABBIX TIPOS Métrica dependiente 18 Agente HTTP 5 Comando 1

Mé Probar métrica

Obtener valor del equipo ☒

Dirección del equipo Puerto

Prueba con **Servidor** Proxy

Valor Hora

☐ No soportada Error

Valor anterior Hora anterior

Secuencia de final de línea **LF** CRLF

Macros

{PVE.TOKEN.ID}	=	monitor@pvelzabbix
{PVE.TOKEN.SECRET}	=	efe5a74c-f67f-46eb-8d3f-6085104ff81b
{PVE.URL.HOST}	=	192.168.1.91
{PVE.URL.PORT}	=	8006

Pasos de preprocesamiento

Nombre	Resultado
1: Comprobar si hay valores incorrectos	{"data":{"cgroup-mode":"2","status":"online","type":"node","id":"node/proxmox","node":"..."}}
Resultado	Resultado convertido a Texto {"data":{"cgroup-mode":"2","status":"online","type":"node","id":"node/proxmox","node":"..."}}

Obtener valor y probar Cancelar

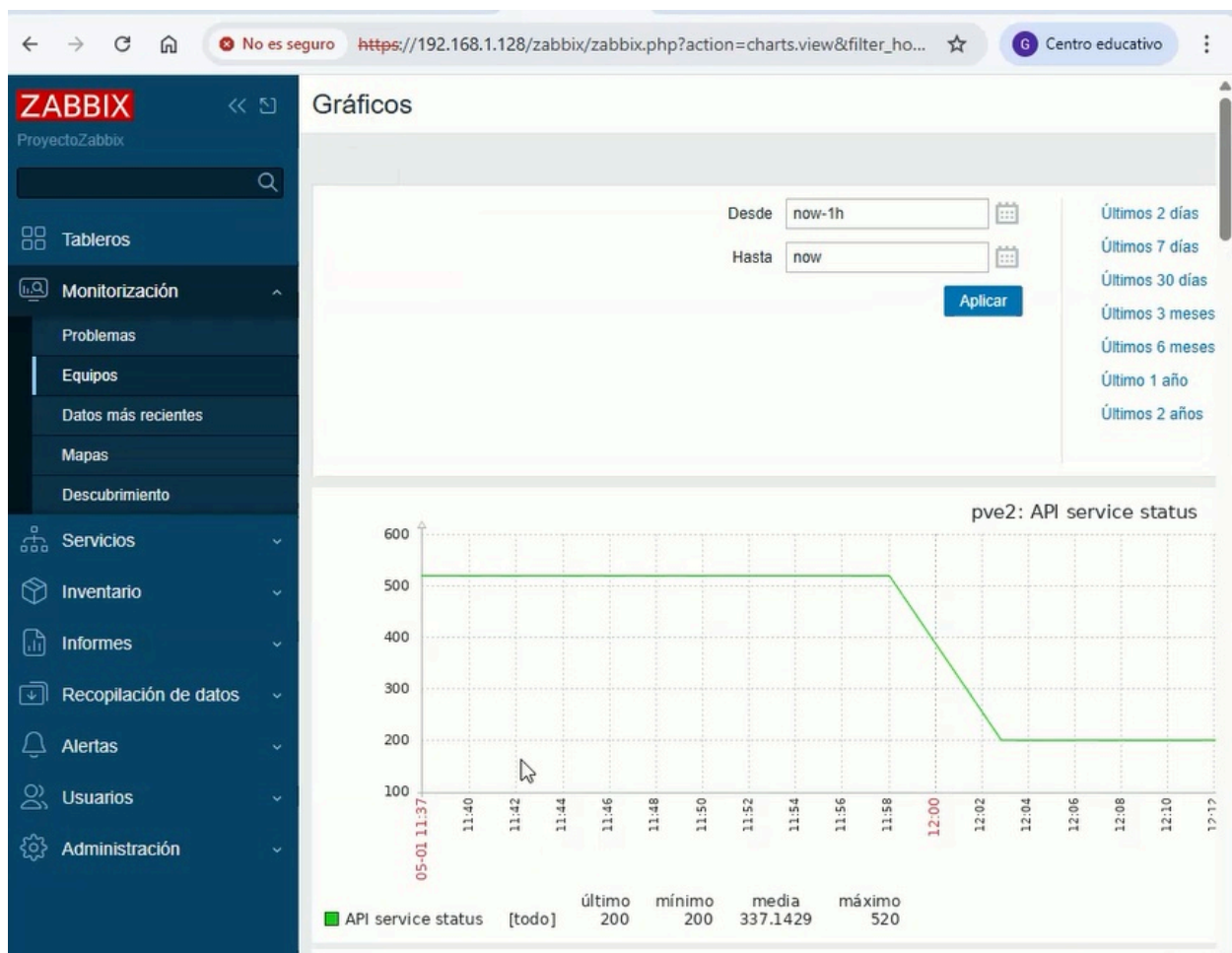
Authorization => PVEAPIToken={PVE.TOKEN.ID}={SP} Eliminar

Agregar

Finalmente ya solo queda ver si los gráficos de datos de Proxmox ya están funcionando correctamente

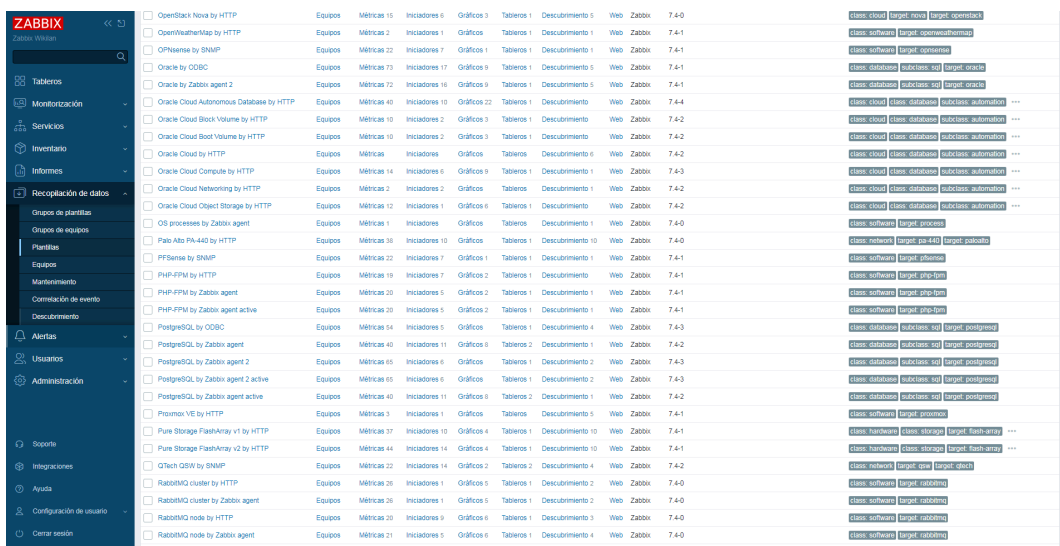
Para ello, hay que entrar en monitorización, equipos y pinchar en sus gráficos y una vez que pasen unos minutos y tras recargar el frontend un par de veces ya deben de aparecer los distintos gráficos que tiene proxmox como son algunos de los siguientes casos:

- **Nodo Principal de Proxmox / Contenedores LXC / Máquinas Virtuales Completas**
 - CPU
 - Memoria RAM
 - SWAP
 - Espacio en el Disco duro



17.1. Configuración de los umbrales de aviso (Proxmox)

Ya que vamos a monitorizar equipos importantes como puede ser por ejemplo el servidor Proxmox, necesitamos que nos brinde distintos avisos cuando ocurren situaciones en el servidor, para gestionar estos avisos tenemos que buscar la plantilla propia del Proxmox que se encuentra en recopilación de datos y entrar en ella



Una vez dentro podemos ver los macros que están activos en este momento los cuales son los siguientes:

Plantilla			
Plantilla Etiquetas Macros 14 Asignación de valores 3			
Macros de plantillas Etiquetas heredadas y de plantillas			
Macro	Valor	Descripción	
(\$PVE.CPU.PUSE.MAX.WARN)	90	Maximum used CPU in percentage.	Eliminar
(\$PVE.LXC.CPU.PUSE.MAX.WARN)	90	Maximum used CPU in percentage.	Eliminar
(\$PVE.LXC.DISK.PUSE.MAX.WARN)	90	Maximum used disk in percentage.	Eliminar
(\$PVE.LXC.MEMORY.PUSE.MAX.WARN)	90	Maximum used memory in percentage.	Eliminar
(\$PVE.MEMORY.PUSE.MAX.WARN)	90	Maximum used memory in percentage.	Eliminar
(\$PVE.ROOT.PUSE.MAX.WARN)	90	Maximum used root space in percentage.	Eliminar
(\$PVE.STORAGE.PUSE.MAX.WARN)	90	Maximum used storage space in percentage.	Eliminar
(\$PVE.SWAP.PUSE.MAX.WARN)	90	Maximum used swap space in percentage.	Eliminar
(\$PVE.TOKEN.ID)	USER@REALMITOKEND	API tokens allow stateless access to most parts of the REST API by another system, software or API client.	Eliminar
(\$PVE.TOKEN.SECRET)	xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx	Secret key.	Eliminar
(\$PVE.URL.HOST)	<SET PVE HOST>	The hostname or IP address of the Proxmox VE API host.	Eliminar
(\$PVE.URL.PORT)	8006	The API uses the HTTPS protocol and the server listens to port 8006 by default.	Eliminar
(\$PVE.VM.CPU.PUSE.MAX.WARN)	90	Maximum used CPU in percentage.	Eliminar
(\$PVE.VM.MEMORY.PUSE.MAX.WARN)	90	Maximum used memory in percentage.	Eliminar

En primer lugar, los avisos que vamos a empezar a configurar son los del propio Proxmox, es decir de la máquina que contiene el servicio Proxmox instalado ya que es el más crítico de todos al contener todas las máquinas. Los umbrales que vamos a configurar para la máquina son: Avisos críticos y warnings para CPU, RAM y disco (ROOT)

Para ello tendremos que usar los umbrales que empiezan por “\$PVE CPU/RAM/ROOT.PUSE.MAX” y al final de cada comando añadir Warn o Crit para peligro o crítico, después añadiendo el porcentaje para indicar cuando es un aviso u otro y finalmente una descripción para ver de qué se trata el problema

{PVE.CPU.PUSE.MAX.WARN}	80	T	Aviso de peligro o <u>w</u> arning por sobrecarga de <u>CPU</u>	Eliminar
{PVE.URL.PORT}	8006	T	The API uses the HTTPS protocol and the server listens to port 8006 by default.	Eliminar
{PVE.CPU.PUSE.MAX.CRIT}	90	T	Aviso crítico por sobrecarga de <u>CPU</u>	Eliminar
{PVE.MEMORY.PUSE.MAX.WARN}	80	T	Aviso de peligro o <u>w</u> arning por sobrecarga de <u>RAM</u>	Eliminar
{PVE.MEMORY.PUSE.MAX.CRIT}	90	T	Aviso crítico por sobrecarga de <u>RAM</u>	Eliminar
{PVE.ROOT.PUSE.MAX.WARN}	80	T	Aviso de peligro o <u>w</u> arning por sobrecarga de disco	Eliminar
{PVE.ROOT.PUSE.MAX.CRIT}	90	T	Aviso crítico por sobrecarga de Disco	Eliminar

Después, otros de los avisos que viene bien tener en Zabbix es el almacenamiento y el Swap o memoria de intercambio que hay dentro de la máquina del Proxmox, para gestionar esto tendremos que usar la sintaxis anterior pero poniendo Swap o Storage “\$PVE SWAP/STORAGE.PUSE.MAX”

{PVE.STORAGE.PUSE.MAX.CRIT}	90	T	Aviso crítico por poco espacio	Eliminar
{PVE.STORAGE.PUSE.MAX.WARN}	80	T	Aviso de peligro por poco espacio	Eliminar
{PVE.SWAP.PUSE.MAX.CRIT}	90	T	Aviso crítico por <u>s</u> obrecarga del <u>swap</u>	Eliminar
{PVE.SWAP.PUSE.MAX.WARN}	80	T	Aviso de peligro por sobrecarga del <u>swap</u>	Eliminar

Los siguientes avisos que vamos a configurar son los avisos LXC o contenedores, estos sirven para monitorizar las distintas máquinas ligeras y rápidas que solo sirven para instalar Linux y así montar distintos servidores, para ello usamos la misma sintaxis que en los avisos anteriores pero añadiendo LXC. En estos contenedores nos limitaremos a poner los umbrales de aviso de warnings y críticos para CPU, RAM y disco (DISK)

{PVE.LXC.CPU.PUSE.MAX.CRIT}	90	T	Aviso crítico por sobrecarga de CPU (contenedores)	Eliminar
{PVE.LXC.CPU.PUSE.MAX.WARN}	80	T	Aviso de peligro o warning por sobrecarga de CPU (Contenedores)	Eliminar
{PVE.LXC.DISK.PUSE.MAX.CRIT}	90	T	Aviso crítico por sobrecarga de Disco (Contenedores)	Eliminar
{PVE.LXC.DISK.PUSE.MAX.WARN}	80	T	Aviso de peligro o warning por sobrecarga de disco (contenedores)	Eliminar
{PVE.LXC.MEMORY.PUSE.MAX.CRIT}	90	T	Aviso crítico por sobrecarga de RAM (contenedores)	Eliminar
{PVE.LXC.MEMORY.PUSE.MAX.WARN}	80	T	Aviso de peligro por sobrecarga de RAM (contenedores)	Eliminar

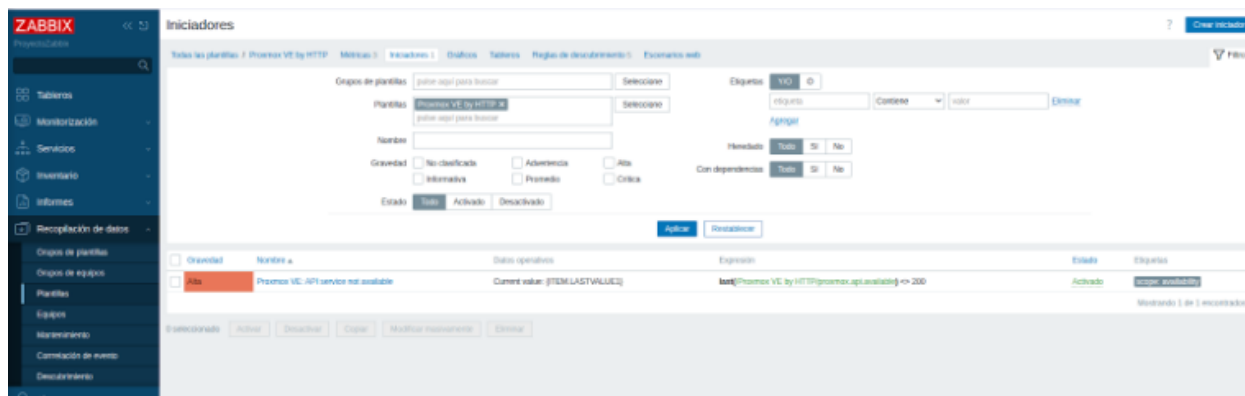
Finalmente los últimos avisos que vamos a configurar son los de las máquinas virtuales completas que vamos a gestionar con Proxmox. La diferencia entre estas y los contenedores es que las VM son máquinas con sistemas operativos completas en cambio de las LXC que eran ligeras y que sólo podían gestionar un Linux no muy pesado

{PVE.VM.CPU.PUSE.MAX.CRIT}	90	T	descripción	Eliminar
{PVE.VM.CPU.PUSE.MAX.WARN}	80	T	descripción	Eliminar
{PVE.VM.MEMORY.PUSE.MAX.CRIT}	90	T	descripción	Eliminar
{PVE.VM.MEMORY.PUSE.MAX.WARN}	80	T	descripción	Eliminar

17.1.1. Activación de triggers de aviso

Una vez se termina de configurar los avisos, en Zabbix hay que “dar de alta” al aviso crítico ya que este servicio de monitorización solo cuenta con el aviso de warning en el sistema, así que hay que crear el aviso crítico.

Para ello lo primero que hay que hacer es entrar en el apartado de recopilación de datos, entrar en plantillas y buscar la plantilla del Proxmox en este caso es “Proxmox by HTTP” y entrar dentro de esta



Una vez dentro, podemos ver una serie de apartados que están dentro del de Proxmox. En primer lugar vamos a empezar configurando la plantilla de la máquina en la que está instalado el servicio Proxmox, para ello tenemos que buscar la que dice “Node discovery” y entrar en “Prototipos del iniciador”

Una vez entramos en prototipos del iniciador podemos ver lo que se comentaba anteriormente sobre los avisos que aparecen en Zabbix (en este caso y en los demás solo aparece el warning por lo tanto debemos crearlo).

Para ver todas las características y funcionalidades que tiene cada apartado podemos entrar en alguno de ellos para comprobarlo como puede ser el más común, el aviso por sobrecarga en el procesador o CPU

☐	Advertencia	Proxmox VE: Node [{#NODE.NAME}] high CPU usage	Current use: (ITEM.LASTVALUE1)	$\min(\text{Proxmox VE by HTTP} \text{proxmox.node.cpu}[\{#NODE.NAME\}].5m) > (\$PVE.CPU.PUSE.MAX.WARN.*\{#NODE.NAME\})$	Si	Si	scope: performance
☐	Advertencia	Proxmox VE: Node [{#NODE.NAME}] high memory usage	Current use: (ITEM.LASTVALUE1) of (ITEM.LASTVALUE2)	$\min(\text{Proxmox VE by HTTP} \text{proxmox.node.memused}[\{#NODE.NAME\}].5m) / \text{last}(\text{Proxmox VE by HTTP} \text{proxmox.node.memtotal}[\{#NODE.NAME\}]) * 100 > (\$PVE.MEMORY.PUSE.MAX.WARN.*\{#NODE.NAME\})$	Si	Si	scope: performance
☐	Advertencia	Proxmox VE: Node [{#NODE.NAME}] high root filesystem space usage	Current use: (ITEM.LASTVALUE1) of (ITEM.LASTVALUE2)	$\min(\text{Proxmox VE by HTTP} \text{proxmox.node.rootused}[\{#NODE.NAME\}].5m) / \text{last}(\text{Proxmox VE by HTTP} \text{proxmox.node.roottotal}[\{#NODE.NAME\}]) * 100 > (\$PVE.ROOT.PUSE.MAX.WARN.*\{#NODE.NAME\})$	Si	Si	scope: capacity
☐	Advertencia	Proxmox VE: Node [{#NODE.NAME}] high swap space usage	Current use: (ITEM.LASTVALUE1) of (ITEM.LASTVALUE2)	$\min(\text{Proxmox VE by HTTP} \text{proxmox.node.swapused}[\{#NODE.NAME\}].5m) / \text{last}(\text{Proxmox VE by HTTP} \text{proxmox.node.swaptotal}[\{#NODE.NAME\}]) * 100 > (\$PVE.SWAP.PUSE.MAX.WARN.*\{#NODE.NAME\}) \text{ and } \text{last}(\text{Proxmox VE by HTTP} \text{proxmox.node.swaptotal}[\{#NODE.NAME\}]) > 0$	Si	Si	scope: capacity

Como podemos ver si entramos en cualquier apartado son varios conceptos siendo los más importantes el nombre del evento, donde podemos ver los comandos que introducimos anteriormente para indicar que el aviso es de CPU, RAM, disco y que sean warnings o críticos, la expresión, la gravedad que podemos indicar si el mensaje es grave, crítico, informativo, entre otros.

Prototipo de iniciador

Prototipo de iniciador

Etiquetas 1

Dependencias

* Nombre

Proxmox VE: Node [{{#NODE.NAME}}] high CPU usage

Nombre del evento

Proxmox VE: Node [{{#NODE.NAME}}] high CPU usage (over {{#PVE.CPU.PUSE.MAX.WARN}}% use)

Datos operativos

Current use: {{ITEM.LASTVALUE1}}

Gravedad

No clasificada

Informativa

Advertencia

Promedio

Alta

Crítica

* Expresión

min(/Proxmox VE by HTTP/proxmox.node.cpu[{{#NODE.NAME}}], 5m) > {{#PVE.CPU.PUSE.MAX.WARN}}%{{#NODE.NAME}}

Agregar

[Constructor de expresiones](#)

Generación del evento OK

Expresión

Expresión de recuperación

Ninguno

Modo de generador de eventos de PROBLEMA

Único

Múltiple

Cierre del evento OK

Todos los problemas

Todos los problemas si los valores coinciden

Permitir cierre manual

☐

Nombre del menú de entrada

URL del iniciador

URL del menú de entrada

Descripción

CPU usage.

Crear habilitado

☒

Descubrir

☒

Modificar

Clonar

Eliminar

Cancelar

Ahora una vez que sabemos lo de los avisos, vamos a crear los avisos críticos, para ello pinchamos en crear prototipo del iniciador y se abrirá el menú de antes. Ahora tendremos que poner todos los valores para el uso de CPU, para ello usamos los que ya están creados con el aviso del warning pero cambiando la directiva “WARN” por “CRIT” de critical en todos los apartados y finalmente cambiaremos la gravedad por Crítica

Ahora creamos el de la RAM, tendremos que poner todos los valores para el uso de RAM, para ello usamos los que ya están creados con el aviso de warning pero cambiando la directiva “WARN” por “CRIT” de critical en todos los apartados y finalmente cambiaremos la gravedad por Crítica

Después creamos el del espacio en el disco, tendremos que poner todos los valores para el uso de disco, para ello usamos los que ya están creados con el aviso de warning pero cambiando la directiva “WARN” por “CRIT” de critical en todos los apartados y finalmente cambiaremos la gravedad por Crítica

Nuevo prototipo de iniciador

Prototipo de iniciador Etiquetas Dependencias

* Nombre: Proxmox VE: Node [[{NODE.NAME}]] critical high root filesystem space usage (over {SPVE.ROOT.PUSE.MAX.CRIT}*{[NODE.NAME]})% use

Nombre del evento: Proxmox VE: Node [[{NODE.NAME}]] critical high root filesystem space usage (over {SPVE.ROOT.PUSE.MAX.CRIT}*{[NODE.NAME]})% use

Datos operativos: Current use: (ITEM.LASTVALUE1) of (ITEM.LASTVALUE2)

Gravedad: No clasificada Informativa Advertencia Promedio Alta **Crítica**

* Expresión: `min(/Proxmox VE by HTTP/proxmox.node.rootused[{NODE.NAME}], 50) / last(/Proxmox VE by HTTP/proxmox.node.rootused[{NODE.NAME}]) * 100 > {SPVE.ROOT.PUSE.MAX.CRIT}*{[NODE.NAME]}`

Constructor de expresiones

Generación del evento OK: Expresión Expresión de recuperación Ninguno

Modo de generador de eventos de PROBLEMA: Único Múltiple

Cierre del evento OK: Todos los problemas Todos los problemas si los valores coinciden

Permitir cierre manual: ☐

Nombre del menú de entrada: URL del iniciador

URL del menú de entrada:

Descripción:

Crear habilitado ☒

Descubrir ☒

Agregar Cancelar

Finalmente creamos el del uso del swap, tendremos que poner todos los valores para el uso de swap, para ello usamos los que ya están creados con el aviso de warning pero cambiando la directiva “WARN” por “CRIT” de critical en todos los apartados y finalmente cambiaremos la gravedad por Crítica

Nuevo prototipo de iniciador

Prototipo de iniciador Etiquetas Dependencias

* Nombre: Proxmox VE: Node [[{NODE.NAME}]] high swap space usage CRITICAL

Nombre del evento: Proxmox VE: Node [[{NODE.NAME}]] high swap space usage (over {SPVE.SWAP.PUSE.MAX.CRIT}*{[NODE.NAME]})% use

Datos operativos: Current use: (ITEM.LASTVALUE1) of (ITEM.LASTVALUE2)

Gravedad: No clasificada Informativa Advertencia Promedio Alta **Crítica**

* Expresión: `min(/Proxmox VE by HTTP/proxmox.node.swapused[{NODE.NAME}], 50) / last(/Proxmox VE by HTTP/proxmox.node.swapused[{NODE.NAME}]) * 100 > {SPVE.SWAP.PUSE.MAX.CRIT}*{[NODE.NAME]}`

Constructor de expresiones

Generación del evento OK: Expresión Expresión de recuperación Ninguno

Modo de generador de eventos de PROBLEMA: Único Múltiple

Cierre del evento OK: Todos los problemas Todos los problemas si los valores coinciden

Permitir cierre manual: ☐

Nombre del menú de entrada: URL del iniciador

URL del menú de entrada:

Descripción:

Crear habilitado ☒

Descubrir ☒

Modificar Clonar Eliminar Cancelar

De esta forma tienen que quedar los umbrales de aviso tanto de CPU, RAM, disco y SWAP con sus avisos de warning y críticos

Gravedad	Nombre	Datos operativos	Expresión	Crear habilitado	Descubrir	Etiquetas
Informativa	Proxmox VE: Node [{#NODE.NAME}]: Kernel version has changed	Current value: {ITEM.LASTVALUE1}	<code>last(Proxmox VE by HTTP/proxmox.node.kernelversion[{#NODE.NAME}].#1)<last(Proxmox VE by HTTP/proxmox.node.kernelversion[{#NODE.NAME}].#2) and length(last(Proxmox VE by HTTP/proxmox.node.kernelversion[{#NODE.NAME}]))>0</code>	Si	Si	scope: notice
Informativa	Proxmox VE: Node [{#NODE.NAME}]: PVE manager has changed	Current value: {ITEM.LASTVALUE1}	<code>last(Proxmox VE by HTTP/proxmox.node.pveversion[{#NODE.NAME}].#1)<last(Proxmox VE by HTTP/proxmox.node.pveversion[{#NODE.NAME}].#2) and length(last(Proxmox VE by HTTP/proxmox.node.pveversion[{#NODE.NAME}]))>0</code>	Si	Si	scope: notice
Informativa	Proxmox VE: Node [{#NODE.NAME}] has been restarted Depende de: Proxmox VE by HTTP: Proxmox VE: Node [{#NODE.NAME}] offline		<code>last(Proxmox VE by HTTP/proxmox.node.uptime[{#NODE.NAME}])<10m</code>	Si	Si	scope: notice
Critica	Proxmox VE: Node [{#NODE.NAME}]: high CPU usage CRITICAL	Current use: {ITEM.LASTVALUE1}	<code>min(Proxmox VE by HTTP/proxmox.node.cpu[{#NODE.NAME}].5m) > {PVE.CPU.PUSE.MAX.CRIT:"[{#NODE.NAME}]"}</code>	Si	Si	
Advertencia	Proxmox VE: Node [{#NODE.NAME}]: high CPU usage WARNING	Current use: {ITEM.LASTVALUE1}	<code>min(Proxmox VE by HTTP/proxmox.node.cpu[{#NODE.NAME}].5m) > {PVE.CPU.PUSE.MAX.WARN:"[{#NODE.NAME}]"}</code>	Si	Si	scope: performance
Advertencia	Proxmox VE: Node [{#NODE.NAME}]: high memory usage	Current use: {ITEM.LASTVALUE1} of {ITEM.LASTVALUE2}	<code>min(Proxmox VE by HTTP/proxmox.node.memused[{#NODE.NAME}].5m) / last(Proxmox VE by HTTP/proxmox.node.memtotal[{#NODE.NAME}]) * 100 > {PVE.MEMORY.PUSE.MAX.WARN:"[{#NODE.NAME}]"}</code>	Si	Si	scope: performance
Critica	Proxmox VE: Node [{#NODE.NAME}]: high MEMORY usage CRITICAL	Current use: {ITEM.LASTVALUE1}	<code>min(Proxmox VE by HTTP/proxmox.node.memused[{#NODE.NAME}].5m) / last(Proxmox VE by HTTP/proxmox.node.memtotal[{#NODE.NAME}]) * 100 > {PVE.MEMORY.PUSE.MAX.CRIT:"[{#NODE.NAME}]"}</code>	Si	Si	
Advertencia	Proxmox VE: Node [{#NODE.NAME}]: high root filesystem space usage	Current use: {ITEM.LASTVALUE1} of {ITEM.LASTVALUE2}	<code>min(Proxmox VE by HTTP/proxmox.node.rootused[{#NODE.NAME}].5m) / last(Proxmox VE by HTTP/proxmox.node.roottotal[{#NODE.NAME}]) * 100 > {PVE.ROOT.PUSE.MAX.WARN:"[{#NODE.NAME}]"}</code>	Si	Si	scope: capacity
Critica	Proxmox VE: Node [{#NODE.NAME}]: high root filesystem space usage CRITICAL	Current use: {ITEM.LASTVALUE1} of {ITEM.LASTVALUE2}	<code>min(Proxmox VE by HTTP/proxmox.node.rootused[{#NODE.NAME}].5m) / last(Proxmox VE by HTTP/proxmox.node.roottotal[{#NODE.NAME}]) * 100 > {PVE.ROOT.PUSE.MAX.CRIT:"[{#NODE.NAME}]"}</code>	Si	Si	
Advertencia	Proxmox VE: Node [{#NODE.NAME}]: high swap space usage	Current use: {ITEM.LASTVALUE1} of {ITEM.LASTVALUE2}	<code>min(Proxmox VE by HTTP/proxmox.node.swapused[{#NODE.NAME}].5m) / last(Proxmox VE by HTTP/proxmox.node.swaptotal[{#NODE.NAME}]) * 100 > {PVE.SWAP.PUSE.MAX.WARN:"[{#NODE.NAME}]"}</code> and <code>last(Proxmox VE by HTTP/proxmox.node.swaptotal[{#NODE.NAME}]) > 0</code>	Si	Si	scope: capacity
Critica	Proxmox VE: Node [{#NODE.NAME}]: high swap space usage CRITICAL	Current use: {ITEM.LASTVALUE1} of {ITEM.LASTVALUE2}	<code>min(Proxmox VE by HTTP/proxmox.node.swapused[{#NODE.NAME}].5m) / last(Proxmox VE by HTTP/proxmox.node.swaptotal[{#NODE.NAME}]) * 100 > {PVE.SWAP.PUSE.MAX.CRIT:"[{#NODE.NAME}]"}</code> and <code>last(Proxmox VE by HTTP/proxmox.node.swaptotal[{#NODE.NAME}]) > 0</code>	Si	Si	
Alta	Proxmox VE: Node [{#NODE.NAME}]: offline	Current value: {ITEM.LASTVALUE}	<code>last(Proxmox VE by HTTP/proxmox.node.online[{#NODE.NAME}]) < 1</code>	Si	Si	scope: availability

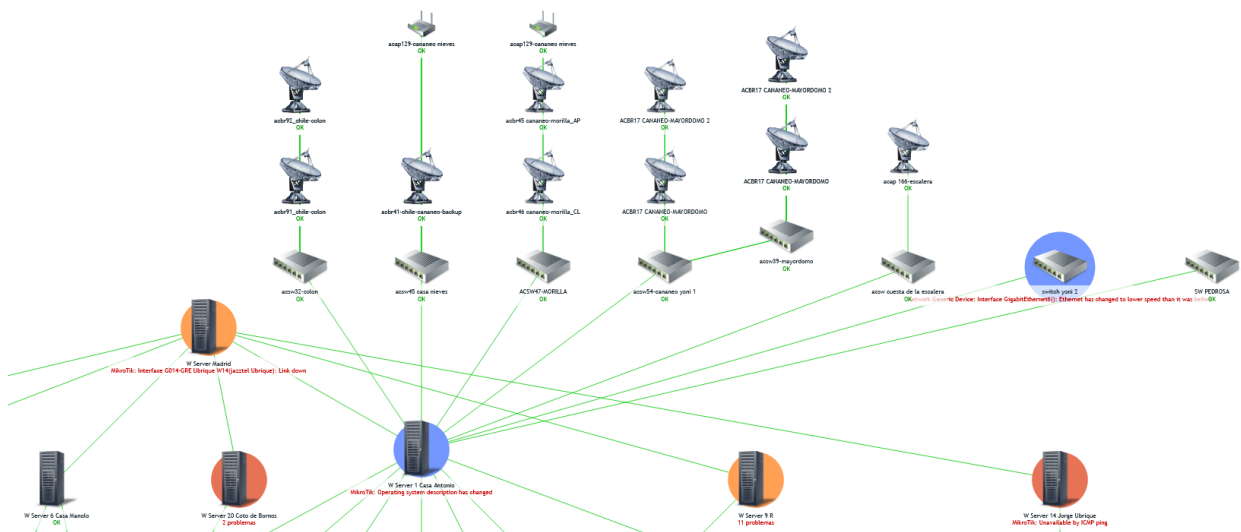
Mostrando 12 de 12 encontrados

17.2. Uso de mapas para aumentar la facilidad de gestión de los equipos

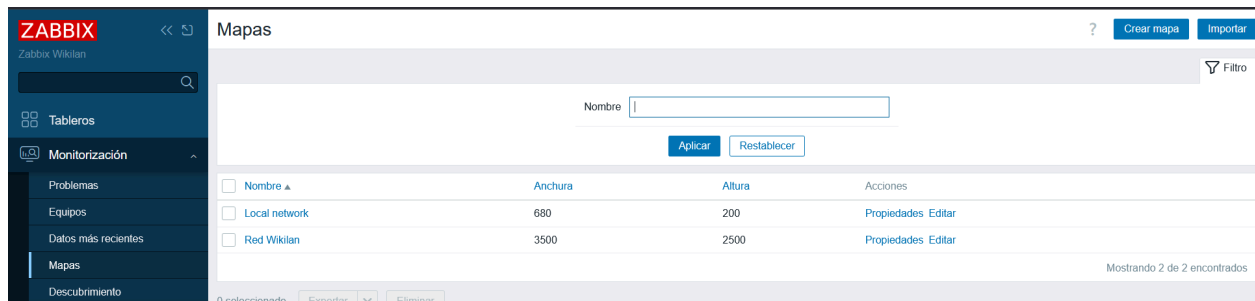
Con Zabbix existen varias formas de gestionar y monitorizar los equipos, la más conocida, fácil y más utilizada por todos los usuarios por norma general es la predeterminada que es la que se encuentra cuando nos vamos a monitorización y entramos en los equipos

Equipos									
Nombre	Interfaz	Disponibilidad	Etiquetas		Estado	Datos más recientes	Problemas	Gráficos	Tableros
acap129-cananeo nieves	10.20.32.7.161	SNMP	class: network	target: aios	target: ubiquiti	Activado	Datos más recientes 22	Problemas	Gráficos 2
acap130-cananeo	10.20.32.8.161	SNMP	class: network	target: aios	target: ubiquiti	Activado	Datos más recientes 22	Problemas	Gráficos 2
acap131-pedrosa	10.20.70.5.161	SNMP	class: network	target: aios	target: ubiquiti	Activado	Datos más recientes 22	Problemas	Gráficos 2
acap132-pedrosa	10.20.70.6.161	SNMP	class: network	target: aios	target: ubiquiti	Activado	Datos más recientes 22	Problemas	Gráficos 2
acap145-verbena	10.20.32.9.161	SNMP	class: network	target: aios	target: ubiquiti	Activado	Datos más recientes 22	Problemas	Gráficos 2
acap146-verbena	10.20.32.10.161	SNMP	class: network	target: aios	target: ubiquiti	Activado	Datos más recientes 22	Problemas	Gráficos 2
ACAP155-MURETE	10.20.32.11.161	SNMP	class: network	target: aios	target: ubiquiti	Activado	Datos más recientes 22	Problemas	Gráficos 2
acap166-escalera	10.20.87.25.161	SNMP	class: network	target: aios	target: ubiquiti	Activado	Datos más recientes 22	Problemas	Gráficos 2
acap170-pichales	10.20.32.12.161	SNMP	class: network	target: aios	target: ubiquiti	Activado	Datos más recientes 22	Problemas	Gráficos 2
acap188-santa cecilia	10.20.87.26.161	SNMP	class: network	target: aios	target: ubiquiti	Activado	Datos más recientes 22	Problemas	Gráficos 2
ACAP201-MORILLA	10.20.32.13.161	SNMP	class: network	target: aios	target: ubiquiti	Activado	Datos más recientes 22	Problemas	Gráficos 2
ACAP211-MAYORDOMO	10.20.32.14.161	SNMP	class: network	target: aios	target: ubiquiti	Activado	Datos más recientes 22	Problemas	Gráficos 2
acap217-misericordia	10.20.62.10.161	SNMP	class: network	target: aios	target: ubiquiti	Activado	Datos más recientes 22	Problemas	Gráficos 2
acap218-morilla	10.20.32.15.161	SNMP	class: network	target: aios	target: ubiquiti	Activado	Datos más recientes 22	Problemas	Gráficos 2
acbr09-zona franca-escalera ap	10.20.87.10.161	SNMP	class: network	target: aios	target: ubiquiti	Activado	Datos más recientes 22	Problemas	Gráficos 2
acbr10-zona franca-escalera	10.20.87.10.161	SNMP	class: network	target: aios	target: ubiquiti	Activado	Datos más recientes 22	Problemas	Gráficos 2
acbr12-reyes-santacecilia2	10.20.87.12.161	SNMP	class: network	target: aios	target: ubiquiti	Activado	Datos más recientes 22	Problemas	Gráficos 2
acbr15-murete	10.20.92.32.161	SNMP	class: network	target: aios	target: ubiquiti	Activado	Datos más recientes 12	Problemas	Gráficos 2

Pero existe otra forma de poder monitorizar los equipos de una forma más limpia y ordenada y esta es haciendo uso de los mapas, como podemos ver en la siguiente imagen, todos los equipos aparecen ordenados y podemos saber donde están conectados cada uno en este caso los servidores están conectados a repetidores y las antenas a ellos



Para crear un mapa en Zabbix tendremos que entrar en el apartado de monitorización y entrar en mapas y pinchar donde dice Crear mapa



Después tendremos que elegir el espacio que tendrá el mapa tanto de alto como ancho, ponerle el nombre al mapa, una imagen de fondo, elegir el propietario del mapa...

Mapas de la red

Mapa [Compartir](#)

* Propietario: Admin (Zabbix Administrator) x Seleccione

* Nombre:

* Anchura:

* Altura:

Imagen de fondo: Sin imagen

Escalar fondo: Ninguno Proporcionalmente

Asignación automática de iconos: <manual> [mostrar asignaciones de icono](#)

Icono resaltado: ☐

Marcar elementos al cambiar el estado del iniciador: ☐

Mostrar problemas: Expandir un único problema Número de problemas Número de problemas y expandir el más crítico

Etiquetas avanzadas: ☐

Tipo de etiqueta de elemento de mapa: Etiqueta

Ubicación de la etiqueta del elemento del mapa: Debajo

Mostrar etiquetas de elementos del mapa: Siempre Ocultar automáticamente

Mostrar etiquetas de enlaces: Siempre Ocultar automáticamente

Mostrar problema: Todo

Gravedad mínima: No clasificada Informativa Advertencia Promedio Alta Crítica

Mostrar problemas suprimidos: ☐

URLs

Nombre	URL	Grupo
		Equipo Eliminar

[Agregar](#)

Agregar Cancelar

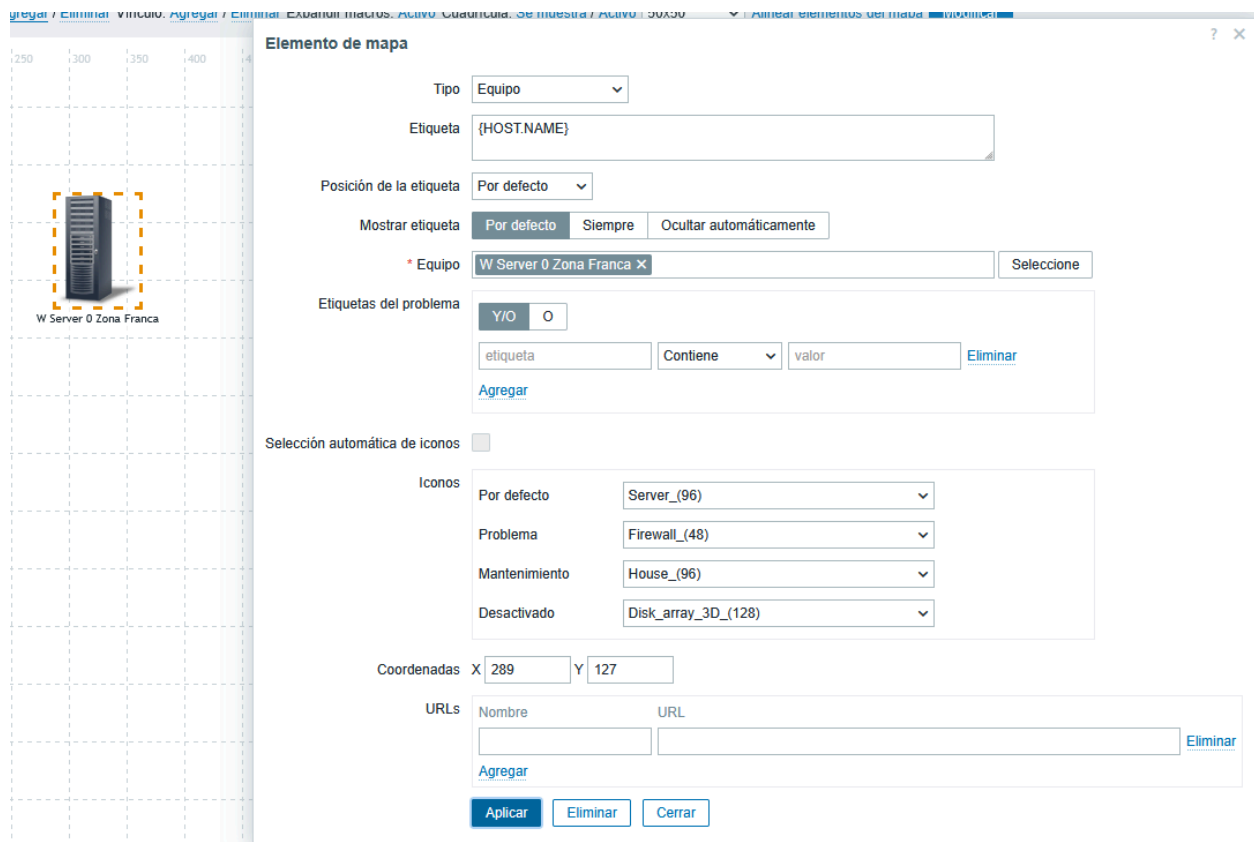
Una vez dentro, disponemos de varias opciones para añadir en el mapa, como pueden ser por ejemplo añadir equipos, formas, enlaces, aumentar el tamaño de la cuadrícula...

Mapas de la red



Si le damos a agregar elemento de mapa, nos aparecerá el siguiente menú en el que podremos añadir un equipo, en este nos aparecerán los distintos apartados como el equipo que vamos a monitorizar, los iconos, la etiqueta...

En el equipo tendremos que seleccionar el que queramos monitorizar que se encuentre en un grupo de equipos, la etiqueta si ponemos “{HOST.NAME}”, se pondrá el nombre que tenga por defecto el equipo, en los iconos podemos poner por ejemplo el del servidor si se trata de este, el de una antena, switch router... a su vez con los demás iconos (problema, mantenimiento, desactivado)



Si probamos con otro equipo como puede ser una antena, podemos hacer lo mismo incluso copiando el equipo que añadimos anteriormente y ahora solo cambiando el equipo y el icono, poniendo el correspondiente como es en este caso una antena

Elemento de mapa

Tipo: Equipo

Etiqueta: {HOST.NAME}

Posición de la etiqueta: Por defecto

Mostrar etiqueta: Por defecto, Siempre, Ocultar automáticamente

* Equipo: acap129-cananeo nieves X Seleccione

Etiquetas del problema: Y/O, O

etiqueta: Contiene, valor, Eliminar

Agregar

Selección automática de iconos: ☐

Iconos:

- Por defecto: Satellite_antenna_(96)
- Problema: Firewall_(48)
- Mantenimiento: House_(96)
- Desactivado: Disk_array_3D_(128)

Coordenadas X: 289 Y: 127

URLs: Nombre, URL, Eliminar

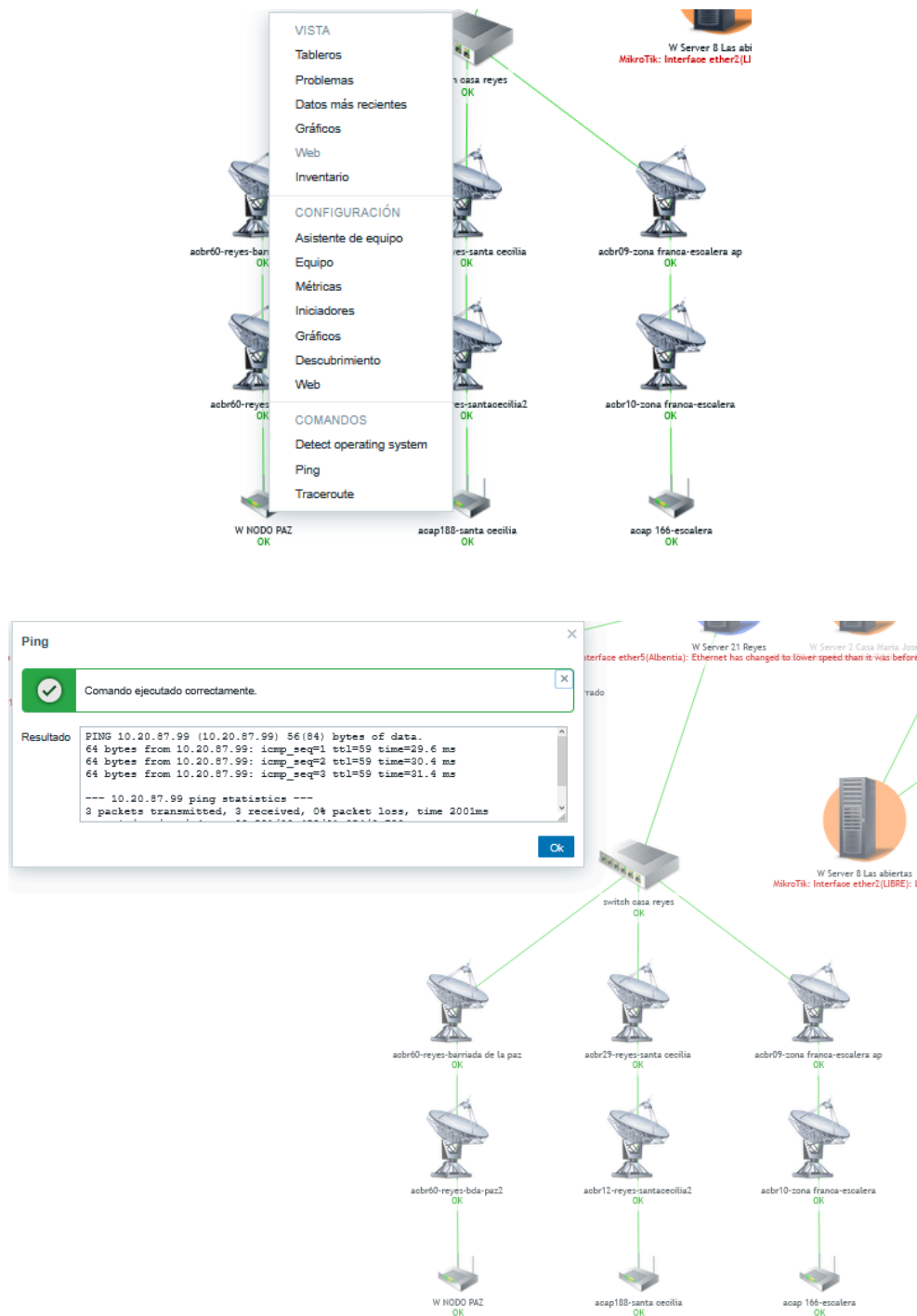
Agregar

Aplicar, Eliminar, Cerrar

Otra peculiaridad de los mapas de Zabbix es la opción de poder unir distintos dispositivos para saber de donde es cada uno y a quien esta conectado, para ello debemos usar la opción de antes de agregar vínculos



Como se comentaba anteriormente, existe la posibilidad de poder usar los comandos o el asistente de equipos que usábamos en la interfaz clásica como puede ser por ejemplo editar el equipo, hacerle un ping, traceroute, observar gráficos, métricas, entre otros.



18. Herramientas de apoyo

18.1. Control de versiones (GIT)

En este proyecto hemos usado GitHub como herramienta encargada del control de versiones ya que en ellas hemos estructurado todos los elementos del proyecto que nos han servido para llevar una correcta gestión del servidor, algunos de estos elementos son los siguientes:

- **Archivos de configuración clave** como “`zabbix_server.conf`” o “`zabbix_db.sql`” que son los archivos más importantes ya que en el primero se encontraba toda la configuración de Zabbix y en el segundo todo lo registrado en sus bases de datos
- **Scripts de automatización** como los scripts de **backup** y **actualización** del servidor que configuramos con el **cron** y registramos en **archivos de log**
- **Documentación y vídeos de instalación, administración y configuración** del servidor que son importantes para saber cómo hemos hecho cada apartado
- **Enlaces a fuentes oficiales** en las que hemos encontrado toda la información respecto al proyecto

18.2. Gestión de pruebas

Después de la fase de implementación y configuración, al haber tenido que añadir muchas cosas al servidor como los umbrales de aviso y las alertas por ejemplo, hemos podido hacer distintas pruebas en varios entornos en los cuales encontramos los siguientes:

18.2.1. Pruebas de funcionamiento (Alertas)

En este apartado, hemos tirado en ocasiones algunos de los equipos que estamos monitorizando para que tanto Telegram como GMAIL manden avisos de que al equipo le ha ocurrido un problema

18.2.2. Pruebas de rendimiento (Umbrales de aviso)

Hemos creado entornos críticos en los cuales hemos sometido a la máquina a sobrecargas de CPU, memoria RAM, disco... para comprobar si los umbrales de aviso que hemos configurado en cada dispositivo responden correctamente

19. Conclusiones

19.1. Conclusiones sobre el trabajo realizado

En este proyecto hemos logrado implementar un servidor Debian 12 con el servicio Zabbix instalado para poder monitorizar distintos entornos de prueba para comprobar si este servicio era rentable o no para poder utilizarlo en el departamento de informática y nos hemos dado cuenta de que sí lo es ya que en este proyecto hemos descubierto muchas cosas positivas para este las cuales pueden ser algunas de las siguientes

19.1.1. Monitorización de virtualización

Se ha logrado integrar un sistema de virtualización como es Proxmox que además para el departamento de informática es crítico y necesitaba de un servicio de monitorización para controlar su rendimiento y para controlar además los contenedores (LXC) y las VM que podemos tener instalada en él. Además se ha hecho la creación macros para poder gestionar los umbrales críticos de Proxmox como la RAM, CPU, disco y Swap

19.1.2. Control de la red con el protocolo SNMP

Hemos podido monitorizar a su vez usando el protocolo SNMP en los dispositivos de red de las aulas de 1º y 2º del Grado Superior para poder monitorizar a toda costa sus switches y así comprobar que este servicio se integra correctamente con Zabbix

19.1.3. Aplicación de políticas de seguridad

Se ha configurado distintas cuentas y roles de usuario para no administrar el frontend de Zabbix solo con la cuenta de administrador y así controlar quién y qué cambios hace en el servidor, además también se han restringido comandos para que los usuarios que no tengan permisos no puedan ejecutarlos sin ellos

19.1.4. Sistema de alertas redundante

Finalmente otro de los conceptos más importantes en el proyecto ha sido el de configurar dos métodos de aviso como puede ser Telegram y GMAIL para avisar a los usuarios que gestionan el servidor cuando

algo falla o supera las macros de los umbrales establecidos

19.2. Conclusiones personales

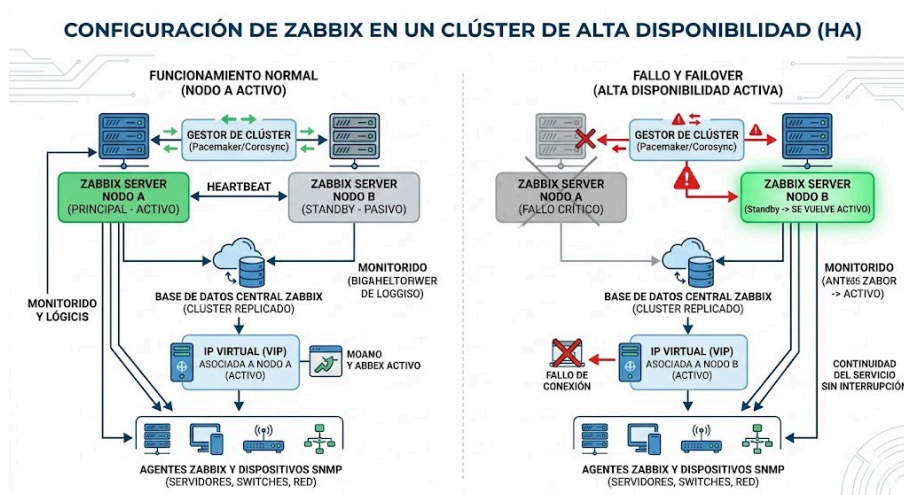
En conclusión, este trabajo ha supuesto un reto un poco complicado al principio pero después con todo el trabajo que se ha hecho durante el curso y además implantando el Zabbix en mi empresa he conseguido aprender muchísimo sobre este servicio de monitorización y además eso me ha ayudado mucho a avanzar en este proyecto. Lo más difícil pienso que fue integrar Proxmox con Zabbix mediante las API pero al final al ver una infraestructura tan grande y que al final funcione de esta forma da mucha satisfacción

19.3. Posibles ampliaciones y mejoras

Si hubiera más tiempo para gestionar este proyecto y añadir posibles mejoras futuras, me centraría en los 2 apartados siguientes:

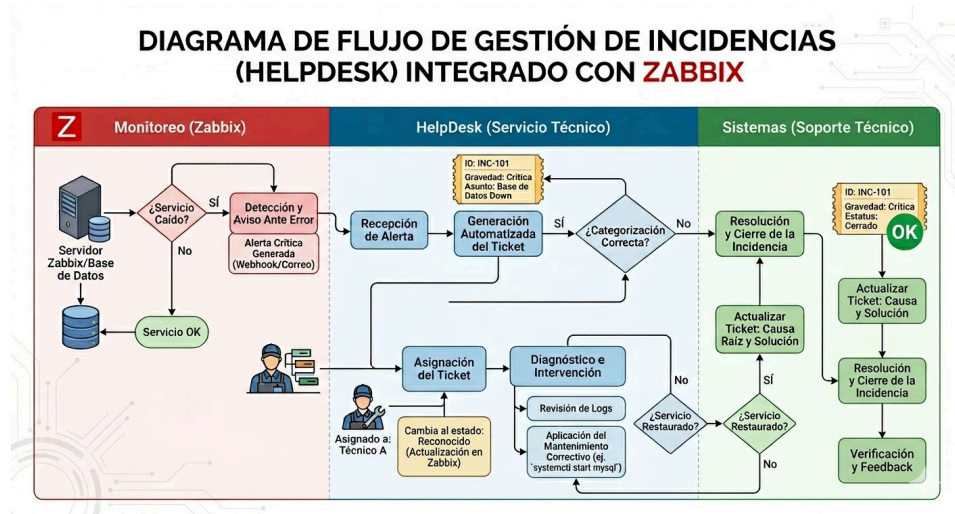
19.3.1. Configuración de Zabbix en un Clúster de alta disponibilidad

Esto haría que en caso de que el servidor principal de Zabbix tuviera algún fallo crítico como llegar a estropearse o perder la conexión, hubiese un nodo secundario que tome el control para seguir funcionando independientemente del fallo del servidor principal



19.3.2. Integración con un sistema de Tickets HelpDesk

Este es el sistema de generación de tickets que se comentaba en el apartado “[HelpDesk](#)” en el que decía que en caso de que hubiera una incidencia, Zabbix enviaba el aviso de que algún dispositivo había tenido un problema y HelpDesk generaba un ticket para que un técnico fuera a revisar la incidencia



BIBLIOGRAFÍA

- [Introducción](#)
- [¿Qué es Zabbix?](#)
- [Arquitectura](#)
- Requisitos
 - [Hardware](#)
 - [Software](#)
- [Ventajas](#)
- [Inconvenientes](#)
- [Seguridad](#)
- [Integraciones](#)
- [Automatización](#)
- [Métricas](#)
- Estado del arte
 - [Zabbix](#)
 - [Prometheus](#)
 - [Nagios](#)
 - [PRTG](#)
- Estudio de la viabilidad
 - [Zabbix About](#)
 - [Zabbix Features](#)
 - [Zabbix Requirements](#)
 - [Zabbix Installation Guide](#)
 - [Zabbix Download Debian](#)
 - [Zabbix Architecture](#)
 - [Zabbix Features](#)
 - [Zabbix Actions](#)
 - [Zabbix Integrations](#)
- Análisis de Requisitos
 - Análisis de Requisitos
 - [Guía de Requisitos](#)
 - [Especificaciones de Software recomendados](#)
 - Requisitos sistemas de monitorización
 - [Requisitos y arquitectura](#)
 - [Análisis de Requisitos](#)

- [Buenas prácticas de monitorización en Zabbix](#)
- Análisis de Riesgos y Vulnerabilidades
 - Metodologías oficiales
 - [Metodología oficial GE](#)
 - [Gestión de riesgos en seguridad de información](#)
 - Aplicación práctica en sistemas de monitorización
 - [Documentación zabbix](#)
 - [Análisis de riesgos en entornos digitales](#)
 - [Gestión de riesgos en sistemas de información](#)
- Diseño del Prototipo
 - Arquitectura
 - <https://standards.ieee.org>
 - [Zabbix documentation](#)
 - Topología
 - <https://www.cisco.com>
 - [Zabbix documentation](#)
 - Seguridad
 - <https://www.nist.gov/cyberframework>
 - [Zabbix documentation](#)
 - Despliegue
 - <https://www.virtualbox.org/manual>
 - [Zabbix documentation](#)
 - Integración
 - <https://www.zabbix.com/integrations>
 - [Zabbix documentation](#)
- Implementación
 - Instalación y configuración de los dispositivos de red
 - Agente Zabbix.-[Zabbix](#)
 - Protocolo SNMP.-[Fortinet](#)
- Administración
 - Usuarios y permisos.-[Zabbix](#)
 - Monitoreo y mantenimiento de la red.-[Zabbix](#)
 - Plan de mantenimiento preventivo y correctivo

- Administrador de tareas programadas con cron.-[Debian](#)
- Administrador de servicios de Linux.-[Debian](#)
- Implementación de backups y recuperación ante desastres (Base de datos).-[MariaDB Knowledge Base](#)
- Plan de contingencia y recuperación ante incidentes.-[Microsoft](#)
- Soporte para la aplicación
 - Integración de alertas con sistemas de tickets.-[Zabbix](#)
 - Sistema de gestión de incidencias HelpDesk.-[GLPI Project](#)
- Herramientas de apoyo
 - GitHub.-[GitHub](#)
- Conclusiones
 - Posibles mejoras
 - Clúster avanzado.-[Zabbix](#)
 - HelpDesk.-[Zabbix-GLPI](#)
- Bibliografía destacada
 - Documentación oficial de Zabbix.-[Zabbix](#)
 - Proxmox VE Wiki.-[Proxmox](#)
 - Telegram Bot API.-[Telegram](#)
 - Documentación oficial de Debian.-[Debian](#)